Fpt University

TRƯỜNG ĐẠI HỌC FPT

Software Design Document

Smart Cafe Management System

Project Code: SCMS

Document Code: Version 03

– Hanoi, October 2026 –

Table of contents

1 Introduction..... 3
1.1 Purpose..... 3
1.2 Definitions, Acronyms and Abbreviations..... 3
1.3 References..... 4
2 System Architecture..... 5
2.1 System Overview..... 5
2.2 Assumptions..... 5
2.3 Design Constraints..... 6
3 Software Architecture Design..... 7
3.1 Architectural Representation..... 7
3.2 Software Architecture..... 9
4 Detailed Component Design..... 25
4.1 Common Design..... 25
4.2 Login/Logout..... 32
4.3 Manage Roles & Permissions..... 38
4.4 Check In..... 42
4.5 Check Out..... 48
4.6 Manage Branch..... 52
4.7 Manage Branch Manager..... 59
4.8 Manage Employee Profiles..... 65
4.9 Manage Work Shifts..... 73
4.10 Manage Menu Categories..... 79
4.11 Raw Material Inventory Management Subsystem..... 87
5. Database Design..... 102
5.1 Database Design..... 102
5.2 Data File Design..... 124

1 Introduction

1.1 Purpose

This Software Design Description (SDD) provides the definitive technical specification and architectural blueprint for the Smart Cafe Management System (SCMS) - Group 2. Conforming to IEEE Std 1016-2009, this document translates the functional requirements and business rules (BR-01 through BR-51) defined in the Software Requirements Specification (SRS) into robust, modular, and maintainable software architectures.

The document defines the system's 4+1 architectural views (Process, Logical, Development, Deployment, and Use-Case scenarios), common cross-cutting infrastructure (exception handling, centralized logging, audit trails, WebSocket messaging, and transaction management), detailed component specifications (screen mockups, 5-column class tables, dynamic sequence diagrams), and complete requirements traceability matrices. It serves as the primary technical guide for developers, quality assurance engineers, system architects, and operations personnel.

1.2 Definitions, Acronyms and Abbreviations

The core operational and technical terms utilized throughout this document are summarized below:

No	Term / Acronym	Definition
1	SCMS	Smart Cafe Management System: Central multi-branch web management platform for cafe chains.
2	COO	Chief Operating Officer: Chain-level administrator managing master data, menus, and global reports.
3	BM	Branch Manager: Store manager responsible for shift schedules, cash control, and raw material inventory.
4	Cashier	Front-counter sales personnel operating the Point of Sale terminal for order and payment handling.
5	Staff	General store personnel (baristas, servers) who record attendance at the Kiosk via RFID/PIN.
6	POS	Point of Sale: Counter touchscreen software interface for order creation, customization, and checkout.
7	Kiosk	Standalone branch terminal allowing employees to clock in/out in under 0.5s via RFID or 4-digit PIN.

8	COGS	Cost of Goods Sold: Direct cost of raw ingredients used in preparing menu items, tracked for shrinkage.
9	RBAC	Role-Based Access Control: Security mechanism restricting API endpoints by user role and branch scope.
10	JWT	JSON Web Token (RFC 7519): Stateless cryptographically signed token carrying authenticated user claims.
11	WSS	WebSocket Secure: Encrypted bidirectional protocol pushing instant payment events to POS terminals.
12	SePay	Third-party payment gateway pushing automated bank balance fluctuation Webhooks to SCMS.
13	VietQR	National standard dynamic payment QR code (NAPAS 247) rendered for customer mobile banking.
14	ESC/POS	Standardized thermal printer command protocol driving POS receipt and barista preparation tickets.

1.3 References

This document derives from and complies with the following baseline specifications and industry standards:

No	Document / Standard	Version / Date	Source / Location
1	Software Requirements Specification (SRS) for SCMS	Version 1.0 (09/2026)	SCMS Group 2, FPT University
2	SCMS Business Process & Domain Model Specification	Version 1.0 (09/2026)	SCMS Group 2, FPT University
3	SCMS Database Design & Data Dictionary Document	Version 1.0 (10/2026)	SCMS Group 2, FPT University
4	IEEE Std 1016-2009: Software Design Descriptions	Standard (2009)	IEEE Computer Society
5	The '4+1' View Model of Software Architecture	IEEE Software (1995)	Philippe Kruchten
6	Spring Boot 3.x & React 18 Architecture Guides	Official Specs (2024)	VMware Tanzu & Meta Platforms, Inc.

2 System Architecture

2.1 System Overview

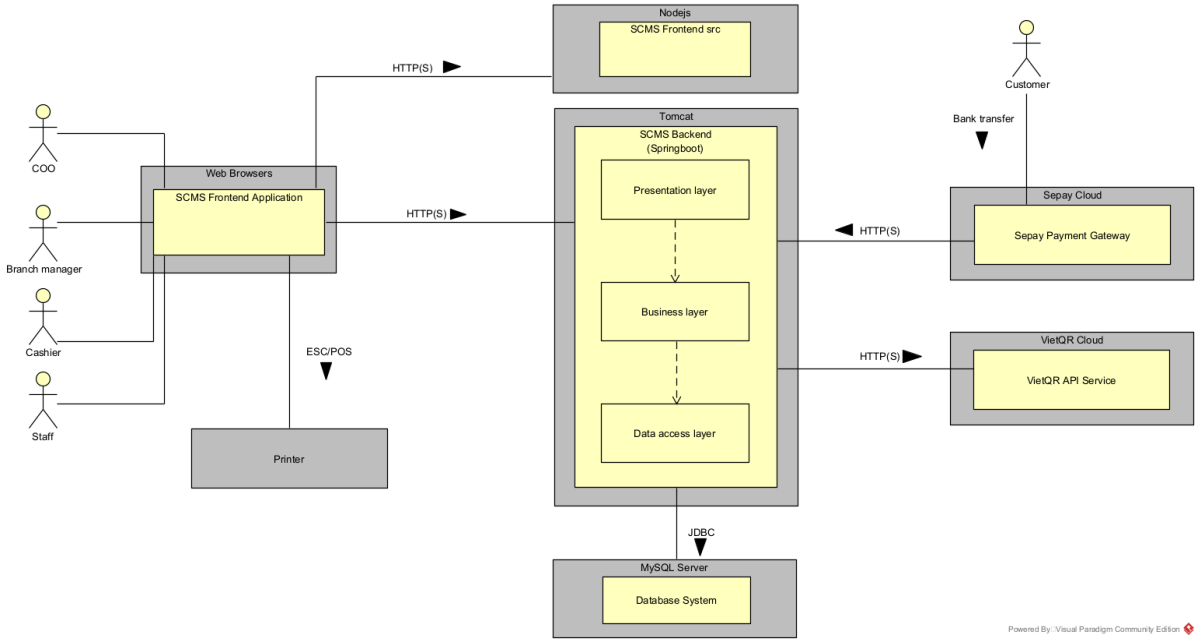


Figure 2-1: System Architectural Overview

As described in Figure 2-1, the system will have the following components:

- **User Groups / Actors**:
  - **COO, Branch Manager, Cashier, and Staff**: Internal users who interact with the system via a web browser.
  - **Customer**: External user who initiates bank transfers to make payments.
- **Web Browser**: Chrome or Edge used by internal users to access the application.
- **SCMS Frontend**: Developed as a React Single Page Application (SPA) bundled with Vite, serving the user interface over HTTP.
- **SCMS Backend**: Deployed on Tomcat, containing a REST controller, service layer, and data access layer, communicating with the browser via REST.
- **Database**: SQL Server or MySQL database connected to the SCMS backend via JDBC.
- **SePay Payment Gateway**: A third-party payment service that integrates with the SCMS backend via HTTP Webhook to process customer bank transfers.
- **VietQR API Service**: An external service that integrates with the SCMS backend via REST.
- **Printer**: Hardware peripheral connected to the SCMS backend via the ESC/POS protocol.

2.2 Assumptions

This system is designed based on the following assumptions:

- End-users (COO, branch managers, cashiers, staff) access the system using modern web browsers, specifically Chrome or Edge.

- The client-side environment runs a React SPA framework powered by Vite.
- The server-side deployment environment relies on Tomcat as the application server.
- The persistent storage layer relies on either SQL Server or MySQL accessed via JDBC.
- External services such as the SePay Payment Gateway and VietQR API Service are continuously available via network/webhooks.
- Hardware setup includes an ESC/POS-compatible receipt printer connected to the backend.

2.3 Design Constraints

This system should comply with the following items:

- The user interface must communicate with the backend services using HTTP, REST, and WebSocket (WSS) protocols.
- The backend architecture must strictly enforce a layered design pattern consisting of a REST controller layer, a service layer, and a data access layer.
- Database transactions must be executed via JDBC connectivity.
- Payment processing and transaction verification must integrate externally through HTTP Webhooks (SePay) and REST endpoints (VietQR API).
- Receipt and ticket printing functions must strictly support and utilize the ESC/POS communication protocol.

3 Software Architecture Design

3.1 Architectural Representation

Software architecture for the Smart Cafe Management System (SCMS) defines the foundational structure of the system, encompassing its software elements (React SPA frontend, Spring Boot backend RESTful services, and relational database persistence), their externally visible properties, and the mutual relationships between them. It establishes high-level modularity, security boundaries, and communication protocols (HTTPS/REST, WSS, JDBC, and ESC/POS) necessary to ensure high throughput, sub-second POS response times, and multi-branch data consistency.

To provide a clear, comprehensive description for all stakeholders, the system architecture is represented using Philippe Kruchten's 4+1 View Model. This framework decomposes the system into four complementary architectural views, organized around a central set of critical use case scenarios:

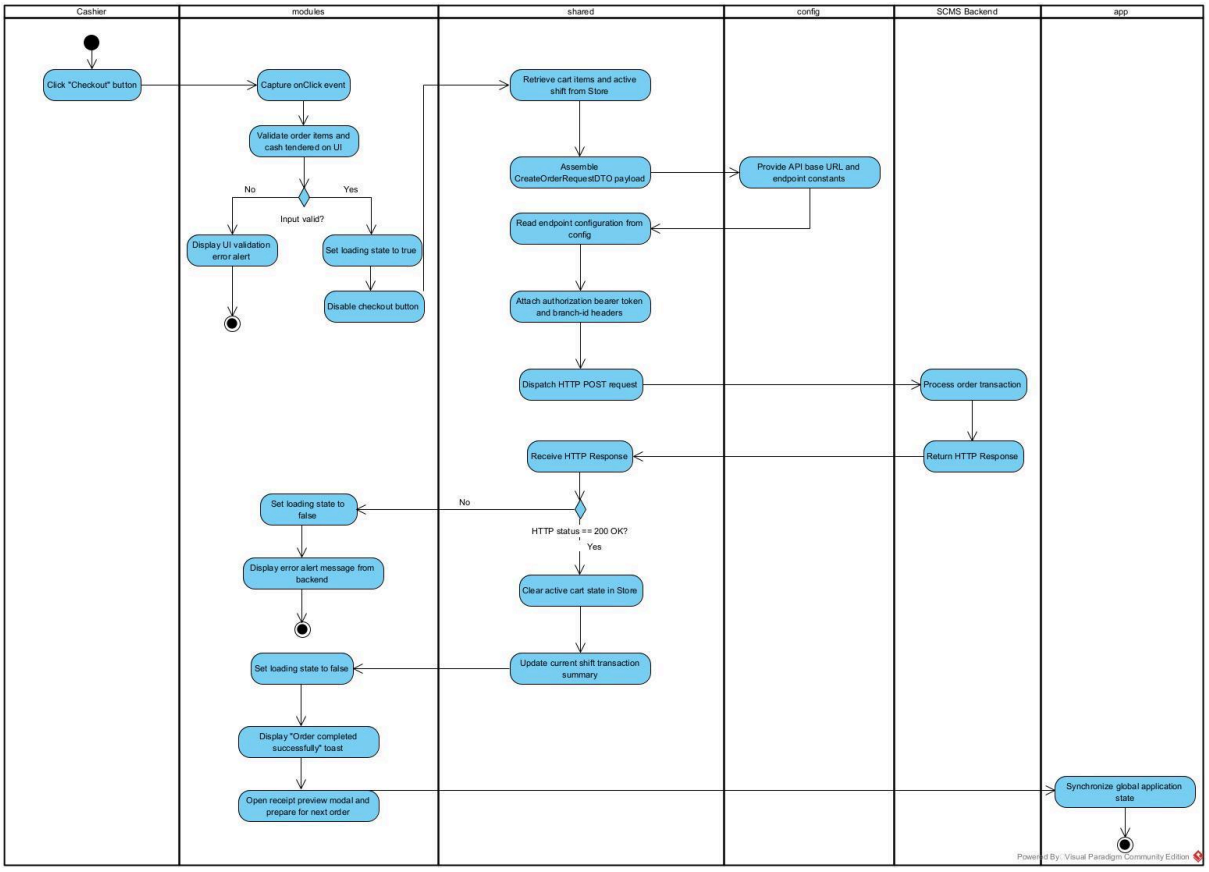
Architectural View	Focus & Architectural Role	Key Model Elements & Artifacts
Use-Case View	Forms the core that drives, validates, and illustrates the entire architectural design from the perspective of end users and external systems.	• Actors: COO, Branch Manager, Cashier, Staff, SePay Gateway, VietQR, Printer. • UML Use Case Diagrams and specifications for 33 use cases (UC01 through UC33). • Core operational scenarios: POS sales, cash shift lifecycle, and bulk inventory tracking.
Logical View	Describes the static structural decomposition of the software into functional layers, subsystems, packages, and object-oriented abstractions.	• Three-tier Spring MVC architecture (@RestController -> Service -> Repository -> Entity/DTO). • Subsystem Class Diagrams, Domain Models, and Entity Relationship Mappings. • 5-column Class Specifications defining attributes, method signatures, inputs, and outputs.

Process View	Addresses runtime concurrency, execution threads, transactional integrity, and dynamic inter-process communication.	• Dynamic UML Sequence Diagrams capturing end-to-end request-response interactions. • POS Checkout and automated VietQR / SePay Webhook reconciliation flows. • Cash Drawer Shift opening/closing lifecycle and physical variance reconciliation. • Full-duplex WebSocket Secure (WSS / STOMP) pub/sub event broadcasting.
Development View (Implementation)	Organizes the concrete codebase, internal package structures, dependency management, and software build configurations.	• Java/Spring Boot package hierarchy (controller, service, repository, entity, dto, security, config). • Frontend React/Vite modular directory structure (components, pages, hooks, services, utils). • Maven build dependencies (pom.xml) and NPM package manifests (package.json).
Deployment View	Maps software processes and execution artifacts onto physical hardware nodes, network topologies, and store peripherals.	• Host/Cloud Server hosting Tomcat application container and MySQL / SQL Server database. • Store-level hardware: Touchscreen POS terminals, Attendance Kiosks, and Mobile devices. • Hardware peripherals: ESC/POS 80mm thermal receipt/bar printers and USB RFID readers. • Secure network protocols: HTTPS, WebSocket Secure (WSS), JDBC, and Webhook listeners.

3.2 Software Architecture

3.2.1 Process View

3.2.1.1 Frontend Process View – POS Checkout Request Execution



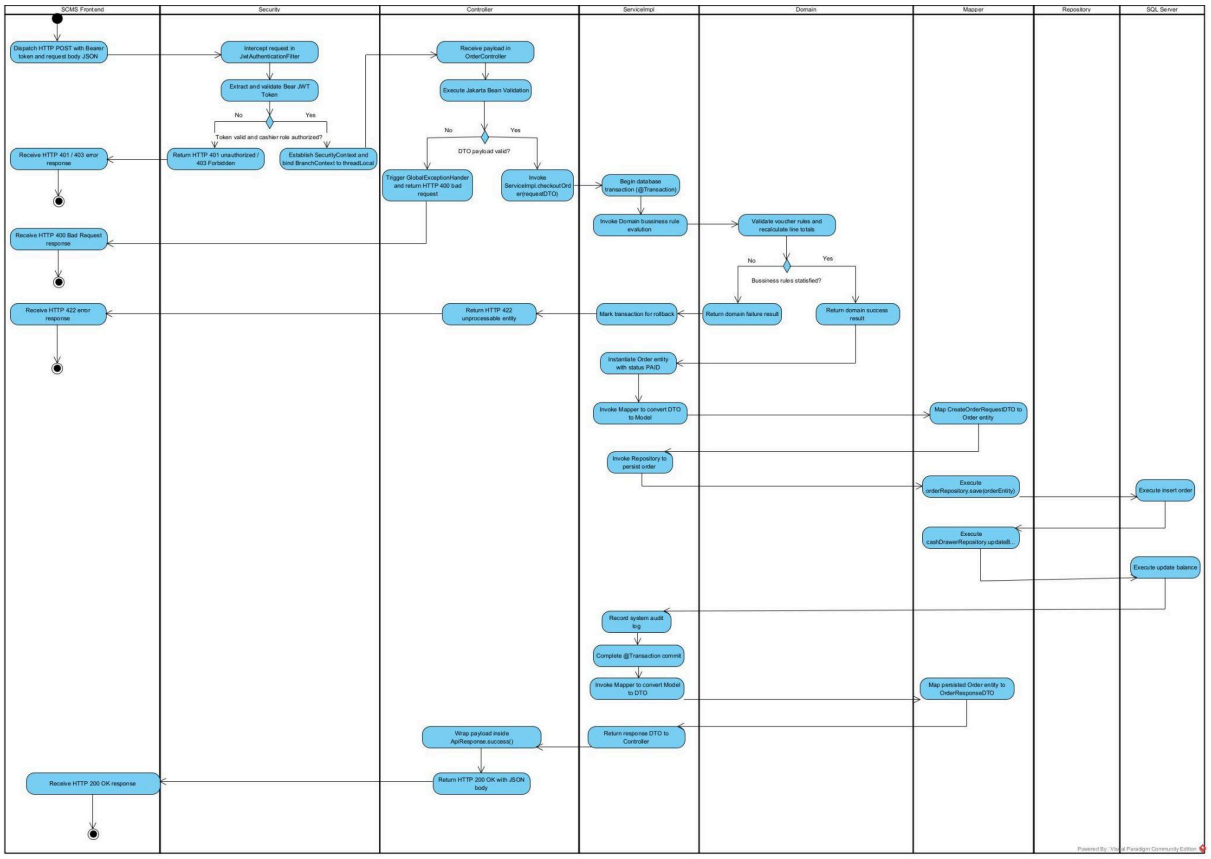
Description:

Step	Participant	Action Name	Detailed Description	Output / Next State
1	Cashier	Click "Checkout" button	Cashier reviews line items, enters customer cash tender, and clicks the primary checkout action.	Event dispatched to modules.
2	modules	Capture onClick event	The POS page component (modules/pos) captures the click event and initiates client-side form validation.	Triggers validation flow.
3	modules	Validate order items and cash tendered on UI	Evaluates basic constraints: verifies cart item count > 0 and confirms cash tendered is greater	Decision: Input valid?

			than or equal to total payable amount.	
4.1	modules	[Branch: No] Display UI validation error alert	If validation fails, an inline error toast or modal alert is rendered to notify the Cashier of invalid input.	Process Terminated (Final State).
4.2	modules	[Branch: Yes] Set loading state to true	If valid, sets local component state isLoading = true to display an active loading spinner.	UI enters loading state.
5	modules	Disable checkout button	Dynamically disables checkout button (disabled=true) to prevent accidental duplicate clicks or race conditions.	Button locked.
6	shared	Retrieve cart items and active shift from Store	Queries central Zustand store (shared/store) to pull snapshot data of selected items, modifiers, voucher codes, and current shift ID.	Cart snapshot data retrieved.
7	shared	Assemble CreateOrderRequestDTO payload	Serializes and structures snapshot data into a standardized CreateOrderRequestDTO JSON object.	DTO payload prepared.
8	config	Provide API base URL and endpoint constants	Exposes environment parameters (VITE_API_BASE_URL) and static route definitions (API_ENDPOINTS.POS_CHECKOUT).	Endpoint URL constants provided.
9	shared	Read endpoint configuration from config	The Axios HTTP client resolves the full target endpoint URL using constants supplied by config.	Target URL resolved.
10	shared	Attach authorization bearer token and branch-id headers	Axios request interceptor injects Bearer JWT token from authStore and active Branch-Id into HTTP request headers.	Headers injected.
11	shared	Dispatch HTTP POST request	Dispatches an asynchronous HTTP POST request containing DTO payload to backend server.	HTTP Request transmitted over network.
12	SCMS Backend	Process order transaction	Spring Boot backend ingests request, verifies security, evaluates business rules, and commits data to SQL Server.	Backend processing completed.
13	SCMS Backend	Return HTTP Response	Backend sends back HTTP response containing status code	Response sent to client.

			and response payload.	
14	shared	Receive HTTP Response	Axios response interceptor intercepts incoming response and evaluates status code.	Decision: HTTP status == 200 OK?
15.1	modules	[Branch: No] Set loading state to false	On HTTP failure (4xx or 5xx), unlocks UI loading spinner.	Spinner hidden.
15.2	modules	[Branch: No] Display error alert message from backend	Extracts error message from response envelope (e.g. ApiError.message) and renders error notification on screen.	Process Terminated (Final State).
16	shared	[Branch: Yes] Clear active cart state in Store	On HTTP 200 OK, dispatches clearCart() action in Zustand store to purge order lines and reset selected voucher.	Store cart state emptied.
17	shared	Update current shift transaction summary	Updates active register shift totals (increments total cash collected and completed order counter) in local store.	Shift summary updated.
18	modules	Set loading state to false	Resets component state (isLoading = false) to hide loading overlay.	UI interaction restored.
19	modules	Display "Order completed successfully" toast	Fires green success toast notification confirming payment completion and order persistence.	Toast displayed to user.
20	modules	Open receipt preview modal and prepare for next order	Triggers thermal receipt modal for optional customer printing and resets POS cash tender input fields.	Receipt modal opened.
21	app	Synchronize global application state	Root application container (app) ensures context synchronization across global providers and route listeners.	Process Terminated (Success Final State).

3.2.1.2 Backend Process View – POS Checkout Request Execution



Description:

Ste	Participant	Action Name	Detailed Description	Output / Next State

p				
1	SCMS Frontend	Dispatch HTTP POST with Bearer token and request body JSON	Dispatches an asynchronous HTTP POST request to /api/v1/pos/orders/checkout carrying Bearer JWT token in headers and CreateOrderRequestDTO JSON body.	Request enters Backend filter chain.
2	Security	Intercept request in JwtAuthenticationFilter	JwtAuthenticationFilter intercepts incoming HTTP request from the filter chain to inspect authorization headers.	Request intercepted for authentication.
3	Security	Extract and validate Bearer JWT Token	Extracts the Bearer token, verifies digital signature, checks expiration timestamp, and inspects granted user roles.	Decision: Token valid and cashier role authorized?
4.1	Security	[Branch: No] Return HTTP 401 unauthorized / 403 Forbidden	AuthenticationEntryPoint rejects unauthorized or invalid request, returning HTTP 401 Unauthorized or HTTP 403 Forbidden.	Error response transmitted to Frontend.
4.2	SCMS Frontend	[Branch: No] Receive HTTP 401 / 403 error response	Client receives 401/403 response, invalidates local session, and prompts user to re-authenticate.	Process Terminated (Error).
5	Security	[Branch: Yes] Establish SecurityContext and bind BranchContext to threadLocal	Populates SecurityContextHolder with user principal and binds current branch identifier (BranchId) to ThreadLocal context.	Contexts established; flow passed to Controller.
6	Controller	Receive payload in OrderController	OrderController deserializes JSON request body into CreateOrderRequestDTO instance.	DTO parsed into memory.
7	Controller	Execute Jakarta Bean Validation	Executes validation annotations (@Valid, @NotNull, @Positive) declared on the DTO fields.	Decision: DTO payload valid?
8.1	Controller	[Branch: No] Trigger	GlobalExceptionHandler intercepts validation	Error response transmitted to

		GlobalExceptionHandler and return HTTP 400 bad request	constraint violations and emits HTTP 400 Bad Request with field error details.	Frontend.
8.2	SCMS Frontend	[Branch: No] Receive HTTP 400 Bad Request response	Client receives 400 response and highlights invalid form input fields on the POS screen.	Process Terminated (Error).
9	Controller	[Branch: Yes] Invoke ServiceImpl.checkoutOrder(requestDTO)	Delegates the validated request payload to the application service layer for transactional orchestration.	Service execution initiated.
10	ServiceImpl	Begin database transaction (@Transactional)	Opens a declarative database transaction boundary (@Transactional) managing rollback and commit boundaries.	Transaction context started.
11	ServiceImpl	Invoke Domain business rule evaluation	Calls the pure domain logic engine to evaluate enterprise business rules without database dependencies.	Domain engine triggered.
12	Domain	Validate voucher rules and recalculate line totals	Evaluates voucher eligibility, minimum spend rules, discounts, and recalculates exact payable total and expected change.	Decision: Business rules satisfied?
13.1	Domain	[Branch: No] Return domain failure result	Domain logic returns a DomainFailureResult containing the business violation reason without throwing unmanaged exceptions.	Failure result passed to Service.
13.2	ServiceImpl	[Branch: No] Mark transaction for rollback	Service catches domain failure, marks active database transaction for rollback, and throws BusinessException.	Transaction marked for rollback.
13.3	Controller	[Branch: No] Return HTTP 422 unprocessable entity	GlobalExceptionHandler catches BusinessException and translates it into HTTP 422 Unprocessable Entity response.	Error response transmitted to Frontend.

13.4	SCMS Frontend	[Branch: No] Receive HTTP 422 error response	Client displays business rule violation alert (e.g., voucher expired or invalid total) to Cashier.	Process Terminated (Error).
14	Domain	[Branch: Yes] Return domain success result	Domain logic confirms that all business calculations, discounts, and item pricing are valid.	Success result passed to Service.
15	ServiceImpl	Instantiate Order entity with status PAID	Instantiates the JPA Order entity model, assigns reference invoice code, and sets operational status to PAID.	Model entity initialized.
16	ServiceImpl	Invoke Mapper to convert DTO to Model	Calls MapStruct mapper component to transform DTO values into the Order entity model hierarchy.	Mapping requested.
17	Mapper	Map CreateOrderRequestDTO to Order entity	Transforms DTO attributes and nested line items into Order and OrderItem JPA entity objects.	Entity graph populated.
18	ServiceImpl	Invoke Repository to persist order	Passes the mapped Order entity to the data access repository layer for database insertion.	Persistence requested.
19	Repository	Execute orderRepository.save(orderEntity)	Spring Data JPA translates the entity graph into parameterized SQL INSERT queries.	SQL commands prepared.
20	SQL Server	Execute insert order	Physical Microsoft SQL Server database executes INSERT INTO orders and order_items tables, generating primary keys.	Order records committed to disk.
21	Repository	Execute cashDrawerRepository.update()	Dispatches update query to increment the active shift cash drawer balance by the cash tender received.	Balance update query sent.
22	SQL Server	Execute update balance	Physical database executes UPDATE query on cash drawer balance and acknowledges successful commit.	Balance updated in database.

23	ServiceImpl	Record system audit log	Asynchronously records an audit log entry documenting order code, cashier identity, branch, and total value.	Audit entry saved.
24	ServiceImpl	Complete @Transaction commit	Completes and commits the application transaction boundary; database locks and connections are released.	Transaction committed successfully.
25	ServiceImpl	Invoke Mapper to convert Model to DTO	Calls mapper to convert persisted Order entity back into an outbound OrderResponseDTO.	Conversion requested.
26	Mapper	Map persisted Order entity to OrderResponseDTO	Transforms persisted entity properties into client-safe OrderResponseDTO, stripping database internals.	Response DTO constructed.
27	ServiceImpl	Return response DTO to Controller	Returns the assembled OrderResponseDTO back to OrderController.	DTO handed to Controller.
28	Controller	Wrap payload inside ApiResponse.success()	Wraps the response DTO into a standardized ApiResponse envelope with success flag and timestamp.	API response envelope constructed.
29	Controller	Return HTTP 200 OK with JSON body	Sends HTTP 200 OK status code along with JSON response envelope over network to client.	HTTP 200 transmitted.
30	SCMS Frontend	Receive HTTP 200 OK response	Client receives success response, unlocks UI, and proceeds with cart clearing and receipt modal rendering.	Process Terminated (Success Final State).

3.2.2 Logical View

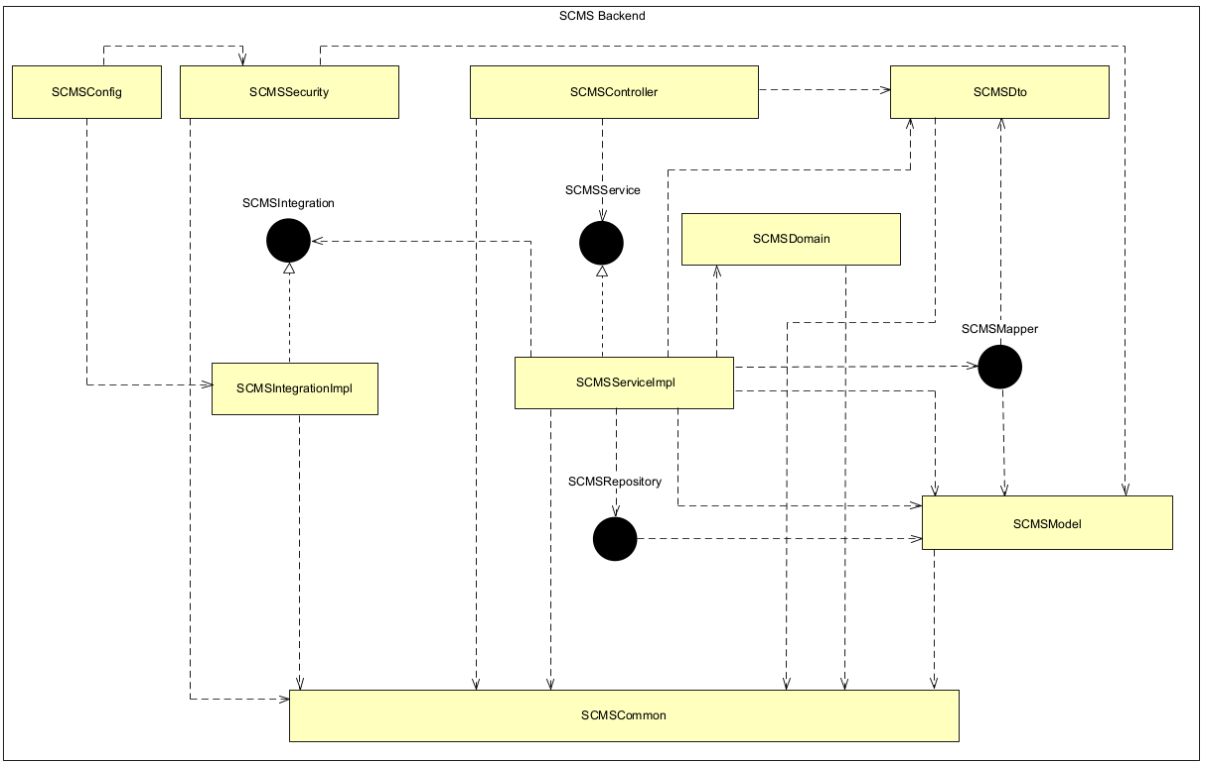


Figure 3-1: SCMS BE Logical Overview

1. Controller
    - Controller — entry point receiving requests from the UI and returning results. It imports dto, service, security (authorization) and common (error codes); a GlobalExceptionHandler centralizes error handling in one place.
2. DTO
    - DTO — defines the data entering/leaving the API (Request/Response) so entities are never exposed. A near-pure data layer; it imports only common (for validation annotations / ApiError/ Enum).
3. Service
    - Service — orchestrates each use case and manages transactions via @Transactional. It imports domain (business rules), repository + model (data), dto, mapper (DTO conversion), integration (external systems) and common.
4. Domain
    - Domain — holds the pure business rules (cash balance, shrinkage, voucher, webhook matching, order state) in plain Java, independent of any framework. It returns result objects instead of throwing errors (the service decides on errors); it never touches repository, integration, BranchContext or audit.
5. Mapper
    - Mapper — Converts between entities and DTOs. It imports model and dto.
6. Repository

    - Repository — Handles database access and applies branch filtering in one central place, preventing data leaks between branches. It imports model and common (like read BranchContext).
7. Model
    - Model — The JPA entities mapped to database tables according to the ERD. It uses only JPA annotations and It can import common (Enum), making it the most stable foundation.
8. Integration
    - Integration — Communicates with SePay, VietQR and the ESC/POS printer using the Port/Adapter pattern, so changing a provider only requires replacing the Adapter. It can import common (IntegrationException/ErrorCode)
9. Security
    - Security — Handles JWT authentication and RBAC authorization, and sets up BranchContext for each request. It imports repository (to load users), common, and model (the single exception, confined to CustomUserDetailsServiceImpl)
10. Common
    - Common — Shared utilities (exceptions, ErrorCode, annotations, utils, constants, audit, BranchContext) with no business logic. Everyone may import it, but it imports no one, which prevents dependency cycles.
11. Config
    - Config — Framework configuration and wiring (the composition root). It enables Spring features (security filter chain, JPA, CORS, OpenAPI, async/scheduling) and registers shared beans such as PasswordEncoder. It imports only security (to attach the JWT filter and UserDetailsService to the SecurityFilterChain). no other package imports config;

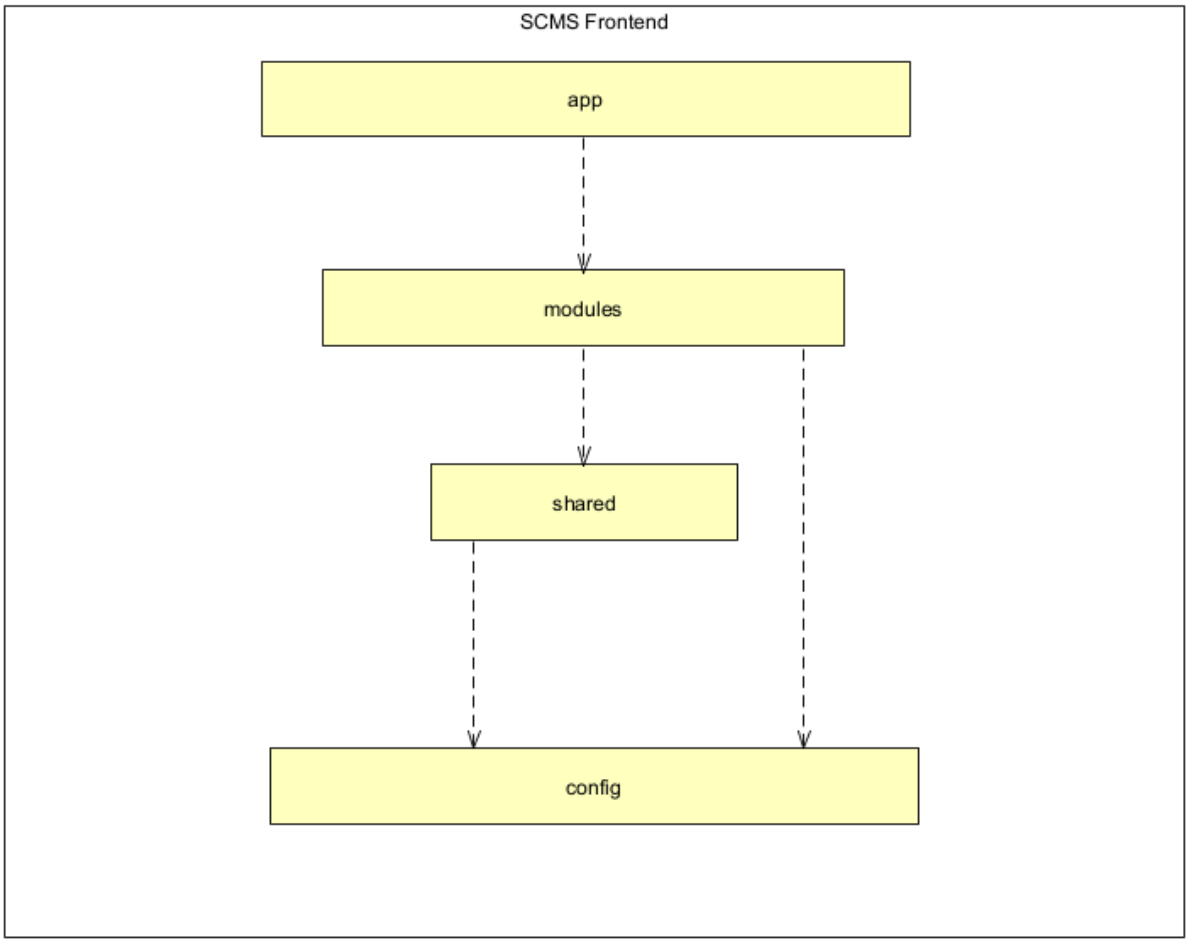
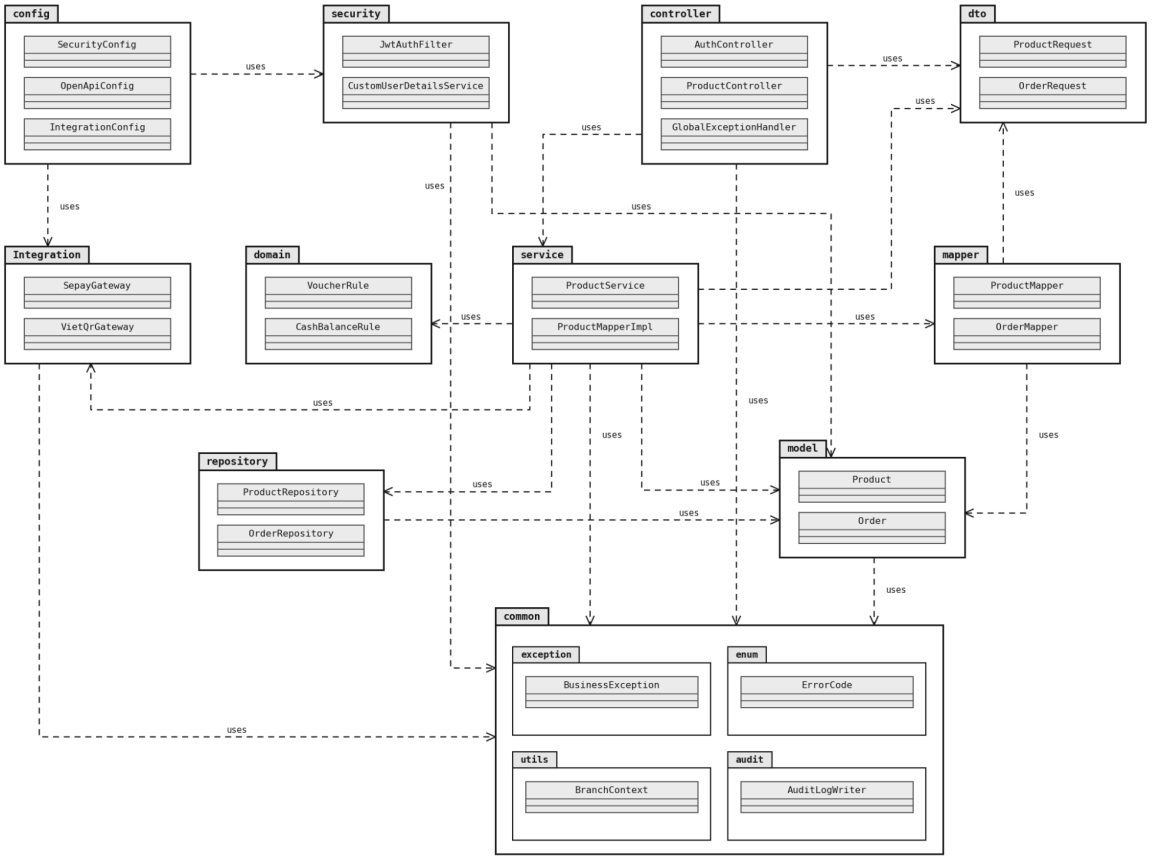


Figure 3-2: SCMS FE Logical Overview

No	Package	Description
01	app	Houses core application bootstrap logic, global routing (router), top-level context wrappers (providers), and page frame structures (layouts).
02	modules	Organized by business domains, encapsulating feature-specific pages, domain components, React Query API hooks, custom business hooks, and client-side calculations.
03	shared	Contains cross-cutting, domain-agnostic components and utilities reusable across all feature modules (such as base UI components, global Zustand stores, custom hooks, and API clients).
04	config	Stores system-wide configuration files including environment variables (env), static constants, and application route definitions (routes).

3.2.3 Development View

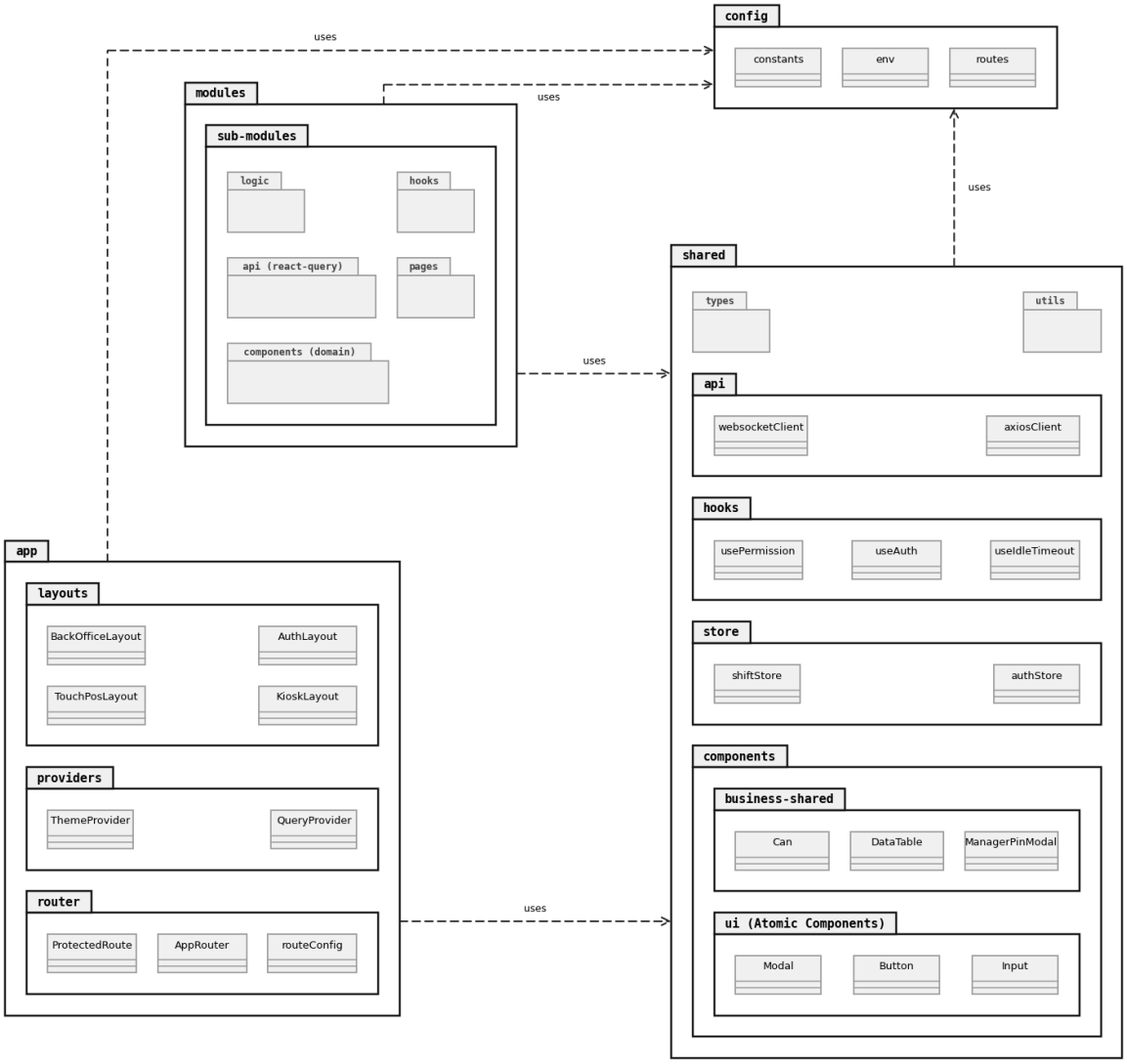
3.2.3.1 Spring Boot Package Diagram



No	Package	Description
01	config	Contains core Spring Boot configuration classes (e.g., SecurityConfig, AppConfig) for setting up beans, CORS, and system-wide parameters.
02	security	Manages security mechanisms, authentication filters (e.g., JwtTokenFilter), Role-Based Access Control (RBAC), and user details services (CustomUserDetailsService).
03	common	Contains system-wide shared components such as centralized exception handling (GlobalExceptionHandler), enums, annotations, and multi-branch context (BranchContext).
04	controller	The presentation layer (RestController). Receives HTTP requests from clients, coordinates business logic, and returns formatted responses.

05	dto	Contains Data Transfer Objects (such as request/response models) used to safely transport data between the client and server without exposing database entities directly.
06	service	The business logic layer. Implements core application workflows and rules, consisting of both service interfaces and implementation classes (ServiceImpl).
07	domain	Holds the pure business rules (cash balance, shrinkage, voucher, webhook matching, order state) in plain Java, independent of any framework. It returns result objects instead of throwing errors (the service decides on errors); it never touches repository, integration, BranchContext or audit.
08	mapper	Contains data mapping components (e.g., MapStruct) responsible for converting objects back and forth between Model (Entities) and DTOs.
09	repository	The data access layer, housing JPA Repository interfaces to interact directly with the database.
10	model	Contains entity classes mapped directly to relational database tables within the system.
11	integration	Contains services dedicated to connecting with external third-party systems or hardware peripherals (e.g., SePay payment gateway, VietQR service, ESC/POS receipt printers).

3.2.3.2 React Package Diagram



No	Package	Description
01	app	Houses core application bootstrap logic, global routing (router), top-level context wrappers (providers), and page frame structures (layouts).
02	config	Stores system-wide configuration files including environment variables (env), static constants, and application route definitions (routes).
03	shared	Contains cross-cutting, domain-agnostic components and utilities reusable across all feature modules (such as base UI components, global Zustand stores, custom hooks, and API clients).
04	shared/components	Encapsulates both generic atomic UI elements (ui (Atomic Components)) and higher-level reusable

		business widgets (business-shared like data tables and authorization guards).
05	shared/store	Manages global application states (e.g., authentication session in authStore, active POS shift status in shiftStore) using Zustand.
06	shared/hooks	Provides shared custom React hooks for authorization, permission checking, and system idle timeouts.
07	shared/api	Houses centralized HTTP (axiosClient) and WebSocket (websocketClient) instances with configured interceptors for authentication and real-time events.
08	modules	Organized by business domains, encapsulating feature-specific pages, domain components, React Query API hooks, custom business hooks, and client-side calculations.

3.2.4 Deployment View

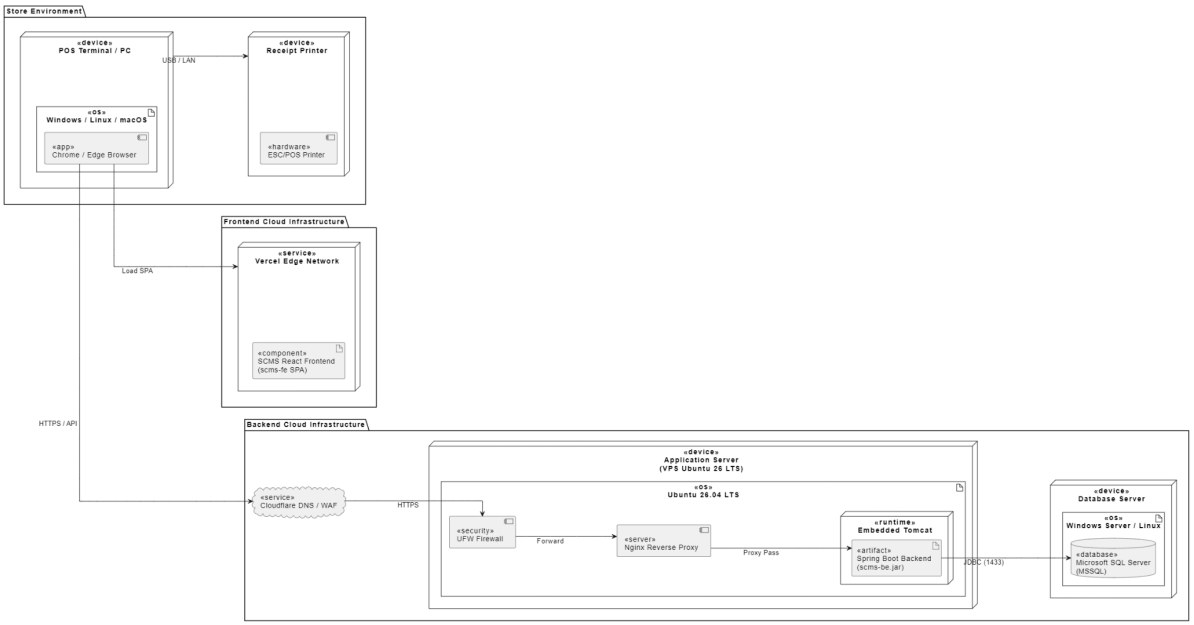


Figure 3-3: SCMS Deployment structure

Configuration Description

No	Component / Node	Description
01	POS Terminal / PC	Physical client hardware (PC/Terminal) located at the store counter used by cashier staff to perform daily checkout transactions and shift management.
02	Receipt Printer	Local hardware peripheral (ESC/POS compatible) connected directly to the POS Terminal via USB/LAN for printing sales receipts.
03	Vercel Edge Network	Global serverless cloud infrastructure hosting and delivering the React single-page application (scms-fe) static build assets.
04	Cloudflare DNS / WAF	Cloud security layer providing DNS resolution, Web Application Firewall (WAF) filtering, and SSL/TLS termination to protect backend endpoints.
05	Application Server	Cloud Virtual Private Server (VPS) running Ubuntu 26.04 LTS, serving as the central execution host for application business logic.
06	UFW Firewall	Operating system-level firewall configured on Ubuntu VPS to restrict inbound traffic strictly to authorized HTTP/HTTPS ports.
07	Nginx Reverse Proxy	High-performance Web/Proxy server configured to handle incoming API requests and securely forward them (proxy_pass) to the local Spring Boot runtime.

08	Spring Boot Backend	Runnable Java application (scms-be.jar) executed inside an embedded Tomcat container to process RESTful API requests, business validation, and authorization.
09	Database Server	Dedicated cloud/standalone server running Microsoft SQL Server (MSSQL) to handle persistence and data management operations independently.

4 Detailed Component Design

4.1 Common Design

4.1.1 Architectural Objectives & Scope

The common design infrastructure is engineered to enforce consistency, high availability, sub-second POS execution, and enterprise security across all store branches. It eliminates code duplication by abstracting cross-cutting concerns into dedicated reusable packages: com.scms.common (shared utilities, exception handlers, DTOs, and context binders) on the backend, and src/shared (Zustand stores, Axios interceptors, atomic UI components, and helper utilities) on the frontend.

4.1.2 Error Handling Design

To ensure system resilience and prevent the leakage of sensitive infrastructure details, SCMS adopts a centralized, non-leaking exception handling architecture powered by Spring Boot's @RestControllerAdvice. Internal stack traces, raw SQL exceptions, and framework errors are intercepted and converted into predictable, standardized client envelopes.

4.1.2.1 Exception Hierarchy

All application exceptions derive from an abstract BaseException, segregated into specialized technical and business categories:

- BusinessException: Domain rule violations (e.g., VoucherExpiredException, ShiftClosedException).
- ResourceNotFoundException: Entity lookup failures (e.g., ProductNotFoundException, BranchNotFoundException).
- ValidationException: Payload constraint failures (Jakarta Bean Validation @Valid).
- UnauthorizedException: Missing, expired, or invalid JWT Bearer tokens.
- ForbiddenException: Role-Based Access Control (RBAC) violations or cross-branch data access attempts.
- ConflictException: Unique constraint collisions (e.g., duplicate barcode, concurrent shift conflict).
- IntegrationException: Third-party communication failures (SePay Webhook, VietQR, ESC/POS Printer).

4.1.2.2 Centralized Exception Handling (GlobalExceptionHandler)

The GlobalExceptionHandler class intercepts all exceptions thrown out of @RestController endpoints. It extracts contextual error codes, formats field-level validation errors, logs failures with the active traceId, and returns a sanitized ApiErrorResponse entity. Uncaught general exceptions are caught by a global fallback handler that masks internal details with a friendly error message and HTTP 500 status.

4.1.2.3 Standard Error Response Envelope (ApiErrorResponse)

Every failed HTTP request produces an immutable JSON envelope containing: success (false), status (HTTP status code), errorCode (standardized business enum), message (human-readable description), path (requested URI), timestamp (ISO-8601 UTC), traceId (correlation ID), and an optional fieldErrors array for form validation violations.

Table 4.1-1: HTTP Status Code Mapping Matrix

HTTP Status	Scenario / Exception Type	Error Code Example	Client Action
400 Bad Request	Malformed JSON syntax, invalid query parameters, failed Bean Validation (@Valid).	INVALID_INPUT_PAYLOAD	Highlight invalid fields on UI form.
401 Unauthorized	Missing, expired, or cryptographically invalid JWT Bearer token.	AUTH_TOKEN_EXPIRED	Clear local auth state; redirect to /login.
403 Forbidden	User lacks required role authority or attempts cross-branch data access.	ACCESS_DENIED	Display 'Insufficient Permissions' alert.
404 Not Found	Target resource ID does not exist within the active branch scope.	RESOURCE_NOT_FOUND	Render 404 Empty State / Toast notification.
409 Conflict	State conflicts, duplicate barcode/SKU, or concurrent update collisions.	RESOURCE_CONFLICT	Prompt user to reload latest data.
422 Unprocessable	Syntactically valid request violates domain logic (e.g., voucher minimum spend).	BUSINESS_RULE_VIOLATION	Display specific business rule rejection alert.
500 Internal Error	Unexpected infrastructure / database failure. Internal stack trace masked.	INTERNAL_SERVER_ERROR	Display friendly error; prompt user with traceId.

503 Unavailable	External integration unavailable (e.g., SePay API offline, printer disconnected).	GATEWAY_UNAVAILABLE	Offer offline/cash fallback to Cashier.

4.1.3 Logging & Distributed Tracing Design

To enable deterministic error diagnosis, forensic auditing, and sub-second bug reproduction in multi-user and multi-branch environments, SCMS implements structured logging combined with Mapped Diagnostic Context (MDC) distributed tracing.

4.1.3.1 Distributed Tracing & Correlation ID (traceId)

Every incoming HTTP request is assigned a unique Correlation Identifier (traceId / UUIDv4) upon entering the application container via TraceContextFilter (OncePerRequestFilter). The traceId is bound to the thread execution context using SLF4J MDC.put('traceId', traceId) and MDC.put('branchId', branchId). It is automatically emitted in every log line, returned in the X-Trace-Id HTTP response header, and embedded inside ApiErrorResponse payloads, allowing support staff to instantly grep logs.

Table 4.1-2: Logging Standards & Levels Governance

Log Level	Intended Usage	Environment	Retention	Target Destination
DEBUG	In-depth troubleshooting, method parameters, SQL execution timings.	Dev / Staging	3 days	Rolling file: debug.log
INFO	Business lifecycle events: Shift opened, Order completed, User login, SePay Webhook.	Production	30 days	Rolling file: app.log / ELK
WARN	Handled business rejections, retryable connection timeouts, invalid logins.	Production	60 days	Rolling file: app.log
ERROR	Unhandled exceptions, SQL connection drops, payment mismatches (includes full stack trace).	Production	180 days	Rolling file: error.log / Sentry

4.1.3.2 Structured JSON Log Format & Sensitive Data Masking

In production, Logback emits single-line JSON records using LogstashEncoder for automated ingestion into ELK / CloudWatch. An automated masking filter scrubs sensitive data before persistence: user passwords and PINs are replaced with [PROTECTED_CREDENTIAL], JWT tokens are truncated to eyJhbGciOi...[MASKED], and bank account numbers are masked as ******1234.

4.1.4 Message Processing & User Feedback Design

A unified communication contract between backend REST endpoints and the React frontend guarantees that users receive consistent, localized, and actionable feedback.

4.1.4.1 Standard API Response Envelope (ApiResponse<T>)

All successful HTTP endpoints return an immutable ApiResponse<T> containing: success (true), status (200), code ('SUCCESS'), message (contextual confirmation), data (generic payload object), timestamp, and traceId.

Table 4.1-3: Frontend Feedback Channels & Display Placement Matrix

Channel	UI Component	Screen Placement	Trigger Condition	Auto-dismiss
Ephemeral Toast	Toast / Notification	Top-Right or Bottom-Right	Async success (e.g., 'Order checkout completed'), network warnings.	Yes (3 - 5s)
Inline Form Error	FormHelperText	Directly beneath input element	Client validation failure, duplicate name, missing mandatory field.	No (until fixed)
Modal Dialog	Modal / AlertConfirm	Center Viewport (Modal Overlay)	Destructive actions (Cancel shift, Void order, Delete product).	No (User click)
Global Banner	SystemBanner	Top Sticky Bar (Full Width)	Server maintenance alert, offline network status, shift variance.	No (Persistent)
Error Screen	ErrorBoundary	Full Content Area	Fatal React rendering crash, 404 route	No (Reload btn)

			missing, 500 API down.	

4.1.5 UI Standards & Layout Guidelines

The SCMS user interface is engineered for high operational ergonomics, touch friendliness at the POS station, and clean administrative oversight for managers and the COO.

4.1.5.1 Viewport Standards & Breakpoints

- POS Touchscreen Station: Fixed 1920x1080 (Full HD) standard, minimum supported 1366x768. Touch targets have a minimum dimension of 48px x 48px with 8px spacing to eliminate mis-clicks during peak hours.
- Administrative Shell: Responsive layout supporting Desktop (>= 1200px), Tablet (768px - 1199px), and Mobile management (< 768px).

4.1.5.2 Standard Layout Shells

- Administrative Shell (COO / Branch Manager): Features a collapsible left sidebar navigation (260px expanded, 80px icon-only), fixed top header bar (64px) housing branch selector and user profile, and fluid content container (padding 24px).
- POS Cashier Shell (Touchscreen POS): Two-panel split layout featuring a 65% width product catalog & category tab ribbon on the left, and a 35% fixed 420px shopping cart, modifier summary, voucher entry, and Cash/QR tender action panel on the right.

Table 4.1-4: Design System Tokens, Color Palette & Typography

Category	Token / Element	Standard Specification
Color Palette	Primary Brand	Deep Coffee #6F4E37 / Emerald #10B981 (Accent)
Color Palette	Background / Surface	Canvas #F9FAFB, Card Surface #FFFFFF, Border #E5E7EB
Color Palette	Feedback Colors	Success #10B981, Error/Danger #EF4444, Warning #F59E0B, Info #3B82F6
Typography	Font Family	'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif
Typography	Font Sizes	Heading 1: 28px (bold), Heading 2: 22px (semi-bold), Body: 14px, Small: 12px
Interactive	Touch Target	Minimum 48px x 48px on POS screens, 8px padding between clickable buttons

Tables	Data Table Standard	Zebra striping optional, sticky header, row height 52px, right-aligned numbers

4.1.6 Common Utility Classes & Components

Reusable cross-cutting logic is isolated inside com.scms.common.utils (Backend) and src/shared/utils (Frontend) adhering strictly to the DRY principle:

- DateTimeUtil (Backend): Standardizes timestamps across database and JSON APIs to ISO-8601 UTC (yyyy-MM-dd'T'HH:mm:ss.SSS'Z') and provides thread-safe local timezone conversion (Asia/Ho_Chi_Minh).
- MoneyUtil (Backend): Enforces rounding rules for Vietnamese Dong (VND: zero decimal places, HALF_UP rounding) and formats numeric values to currency representations (e.g., 125.000 ₫).
- SecurityUtil (Backend): Static accessor providing convenient access to SecurityContextHolder for extracting active user ID, username, and branch scope.
- ValidationUtil (Backend): Regular expression helpers for validating Vietnamese phone numbers, POS barcodes, and tax IDs.
- formatCurrency & formatDate (Frontend): Pure functional helpers wrapping browser Intl.NumberFormat and date-fns for fast, reactive UI rendering.

4.1.7 Transaction Management Architecture

Transaction management enforces ACID guarantees across all state-modifying operations, particularly inventory deductions, cash shift reconciliations, and POS order payments.

- Strict Layer Enforcement: @Transactional boundaries are declared exclusively at the Service Layer (@Service / ServiceImpl). Controllers and Repositories are strictly prohibited from initiating transaction scopes.
- Exception Rollback Policy: All transactions rollback automatically on unchecked exceptions (RuntimeException) and explicitly on checked exceptions via @Transactional(rollbackFor = Exception.class).

Table 4.1-5: Transaction Classification & Strategies Matrix

Transaction Category	Configuration Strategy	Isolation & Locking	Use Case Applications
Read-Only Transactions	@Transactional(readOnly = true)	Read Committed (Snapshot)	Product catalog lookups, sales reports, inventory inquiries. Disables Hibernate dirty checking; eliminates write-locks.

Standard Business Write	@Transactional(rollbackFor = Exception.class)	Read Committed	Order checkout, shift opening/closing, employee profile modification. Atomic single-branch write.
High-Contention / Stock Deduct	@Transactional + @Lock(LockModeType.PESSIMISTIC_WRITE)	Pessimistic Write Lock (SELECT ... WITH (UPDLOCK))	Bulk inventory deduction during rush hours, preventing negative stock levels.
Isolated Sub-Transactions	@Transactional(propagation = Propagation.REQUIRES_NEW)	Independent Transaction Scope	Audit logging (AuditLogService), failed login attempt counters, external payment webhook logging. Commits even if parent fails.

4.1.7.1 Database Engine Concurrency Control (Microsoft SQL Server)

Microsoft SQL Server is configured with READ_COMMITTED_SNAPSHOT ON at the database level. Readers do not block writers, and writers do not block readers, guaranteeing sub-second response times for POS queries during active order processing. Concurrently modified entities (InventoryStock, CashDrawerShift) include a @Version private Long version; optimistic locking column to prevent lost updates.

4.1.8 Security Architecture: Authentication, Authorization & Encryption

- Authentication Architecture: Employs stateless JSON Web Tokens (JWT) with a Dual-Token lifecycle: short-lived Access Tokens (30 minutes) carrying identity and branch claims, and long-lived Refresh Tokens (7 days) stored in secure HTTP-only SameSite cookies.
- Role-Based Access Control (RBAC): Enforces hierarchical authorization (ROLE_COO > ROLE_BRANCH_MANAGER > ROLE_CASHIER = ROLE_STAFF) via declarative Spring Security @PreAuthorize annotations.
- Multi-Branch Isolation (BranchContext): SecurityContext extracts the active branchId from the JWT, binding it to a ThreadLocal BranchContext for the request lifespan. SCMSRepository instances append automatic branch scoping (WHERE branch_id = :branchId) to prevent cross-branch data leaks.
- Cryptography & Data Protection: Passwords hashed with BCrypt (work factor 12); network transmission secured via TLS 1.3 / HTTPS and WSS; database physical files protected with SQL Server Transparent Data Encryption (TDE).

4.2 Login/Logout

This section presents the detailed design of the Login/Logout feature. The COO and Branch Managers authenticate on the User Login screen (SCR-01) with username and password; the backend validates the credentials with a BCrypt hash comparison, applies the temporary lock policy after 5 consecutive failures, issues a JWT access token that carries the user's role and branch scope (BR-01, BR-04), and the client routes the user through the Role Dispatcher (SCR-03). Logout revokes the token and records an audit entry.

4.2.1 Screen Design

4.2.1.1 User Login Screen (SCR-01)

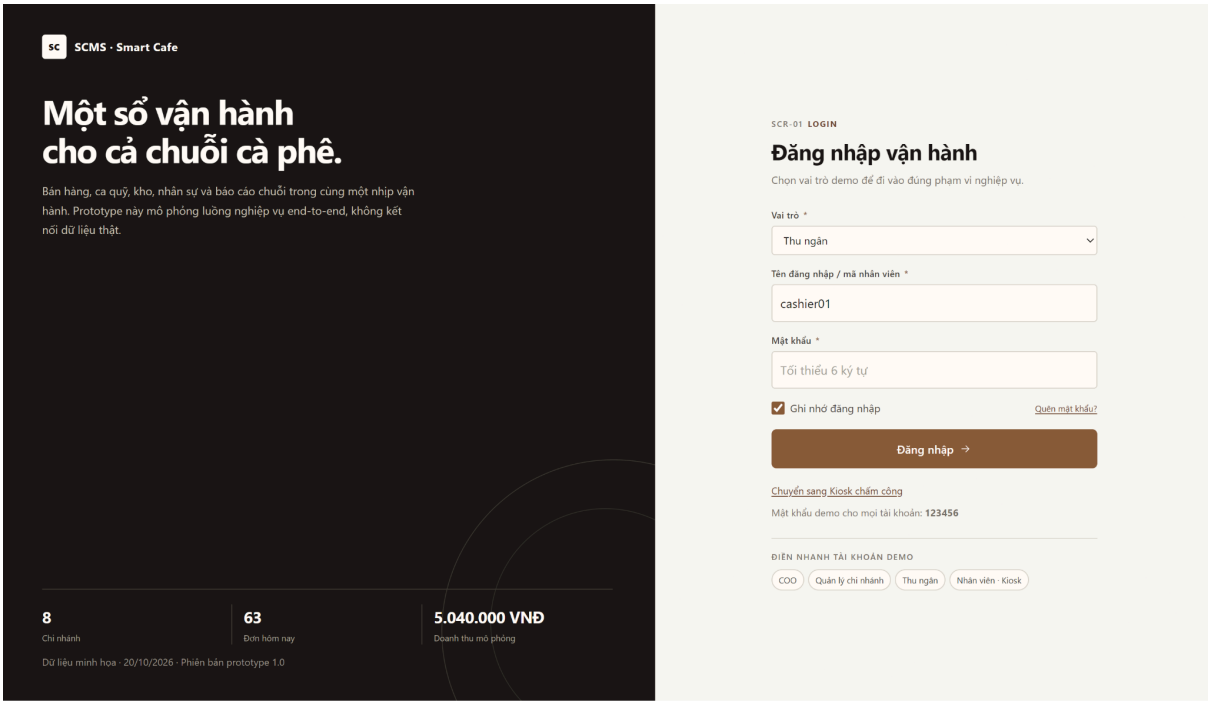


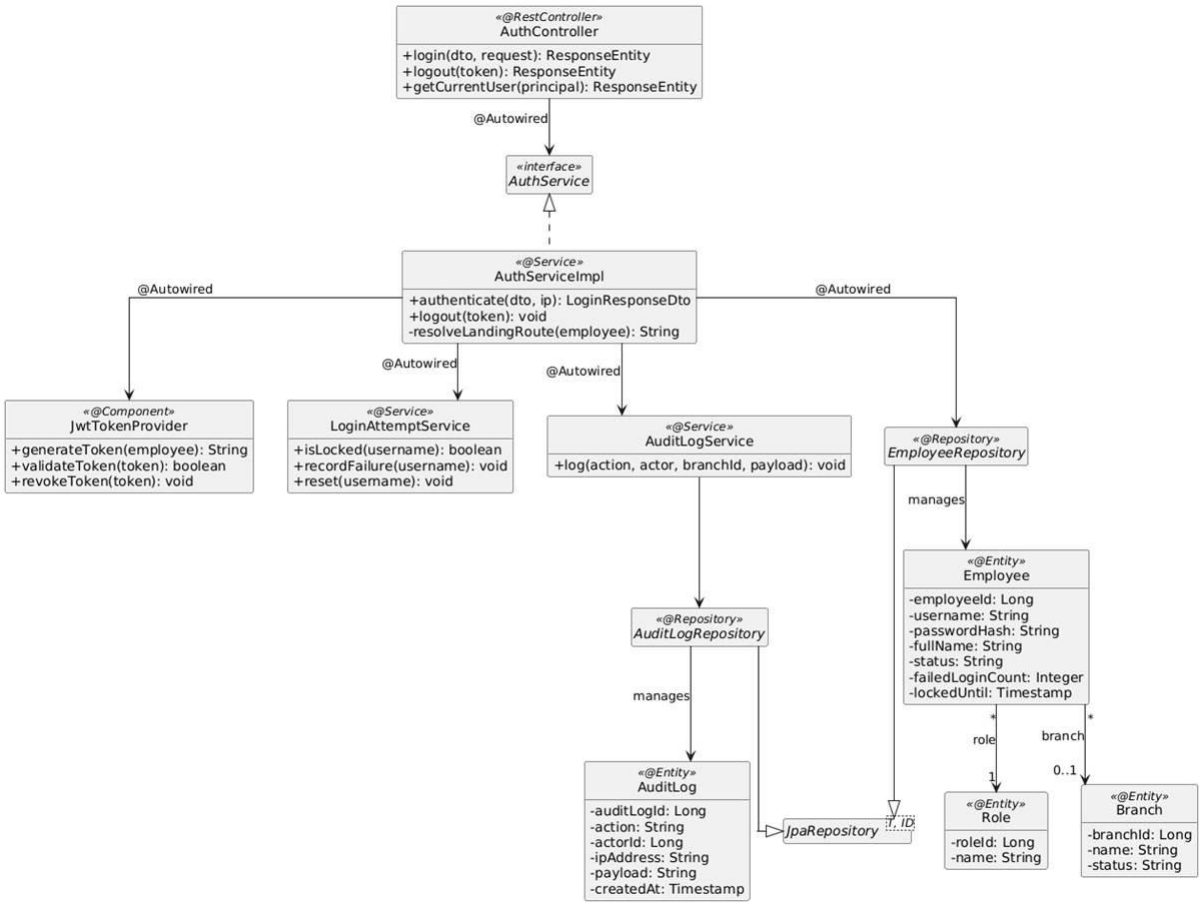
Table 4-1: User Login Screen (SCR-01) Definition

No	Object/Control Name	Type	Required	Length	Description
1	cboRoleSelector	Dropdown	Yes	-	Selects the intended working module (Cashier, Branch Manager, COO, Staff). Used only as a routing hint; the real role is always resolved from the database (BR-01).
2	txtUsername	Textbox	Yes	Max 30	Username or employee ID (e.g., manager_d1). Trimmed before submission.

3	txtPassword	Password	Yes	6 - 32	Masked password input. Minimum 6 characters; sent only over HTTPS.
4	chkRememberMe	Checkbox	No	-	Keeps the refresh token in browser storage so the session survives a page reload. Default: checked.
5	lnkForgotPassword	Link	No	-	Shows instructions to contact the Branch Manager or COO for a password reset.
6	btnLogin	Button	Yes	-	Calls POST /api/v1/auth/login. On success stores the JWT and redirects to SCR-03; on failure displays MSG09 or the account-locked message.
7	lnkSwitchToKiosk	Link	No	-	Switches the terminal to the Attendance Kiosk screen (SCR-02).

4.2.2 Class/Component Design

4.2.2.1 Classes/Component Structure



4.2.2.2 Classes/Component Description

4.2.2.2.1 AuthController Class

Class	AuthController
Description	Spring REST Controller exposing the authentication endpoints: login, logout and current-user profile.
Base Class	None (@RestController, @RequestMapping("/api/v1/auth"))
Constructor	+AuthController(AuthService authService)
Prototype	public class AuthController
Source File	AuthController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
authService	AuthService - Service interface injected via @Autowired.
Methods	
Name	**Input / Output / Description**

login	Input: @RequestBody @Valid LoginRequestDto dto, HttpServletRequest request Output: ResponseEntity<LoginResponseDto> Description: POST /login. Authenticates the credentials and returns access token, role, branchId and default landing route (UC01 main flow).
logout	Input: @RequestHeader("Authorization") String token Output: ResponseEntity<Void> Description: POST /logout. Revokes the current token and records a LOGOUT audit entry.
getCurrentUser	Input: @AuthenticationPrincipal UserPrincipal principal Output: ResponseEntity<CurrentUserDto> Description: GET /me. Returns the profile, role and data scope used by SCR-03.

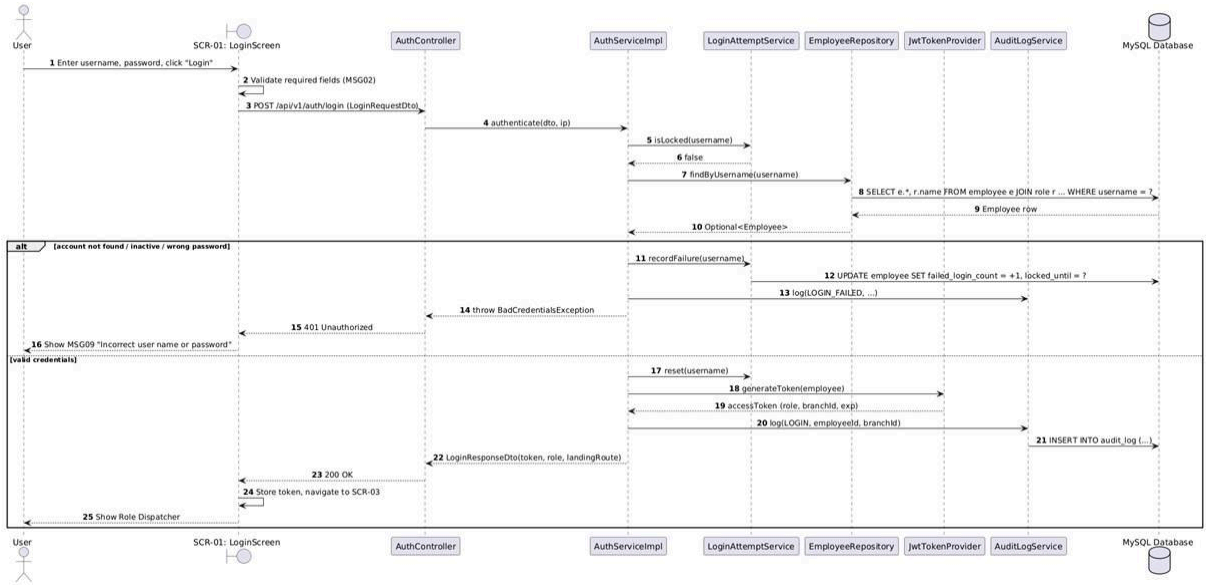
4.2.2.2.2 AuthServiceImpl Class

Class	AuthServiceImpl
Description	Service implementation performing credential validation, lock policy, token issuing and session termination.
Base Class	None (implements AuthService)
Constructor	+AuthServiceImpl(EmployeeRepository employeeRepo, PasswordEncoder passwordEncoder, JwtTokenProvider jwtTokenProvider, LoginAttemptService loginAttemptService, AuditLogService auditLogService)
Prototype	public class AuthServiceImpl implements AuthService
Source File	AuthServiceImpl.java
Namespace	com.scms.service.impl
Attributes	
Name	**Type & Description**
employeeRepo	EmployeeRepository - Loads the employee account by username.
passwordEncoder	PasswordEncoder - BCrypt encoder used to verify the password hash.
jwtTokenProvider	JwtTokenProvider - Generates, validates and revokes JWT tokens.
loginAttemptService	LoginAttemptService - Counts failed attempts and locks the account after 5 consecutive failures.
auditLogService	AuditLogService - Writes LOGIN / LOGOUT / LOGIN_FAILED entries (SCR-17).
Methods	
Name	**Input / Output / Description**

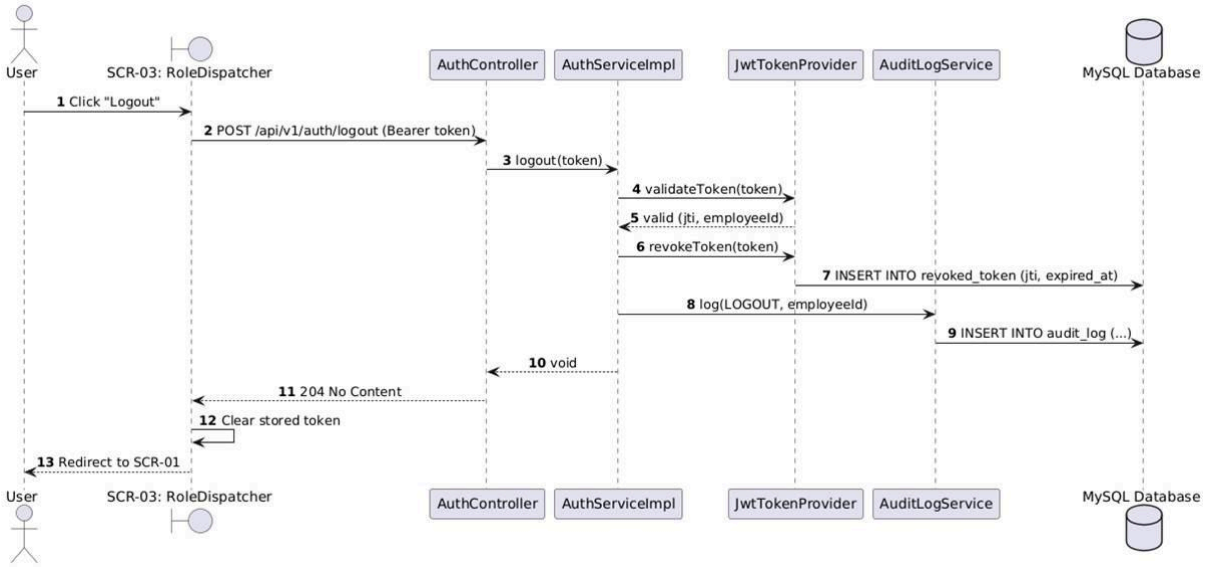
authenticate	Input: LoginRequestDto dto, String ipAddress Output: LoginResponseDto Description: Checks lock state, loads the account, verifies status = ACTIVE and password hash, resets the failure counter, resolves role and branch scope, issues the token and logs the login. Throws BadCredentialsException (MSG09) or AccountLockedException.
logout	Input: String token Output: void Description: Validates the token, adds its jti to the revoked-token store and logs the logout.
resolveLandingRoute	Input: Employee employee Output: String Description: Maps the role to the default screen: COO -> SCR-10, Branch Manager -> SCR-07, Cashier -> SCR-04.

4.2.3 Object Interactions (Sequence Diagrams)

4.2.3.1 SD-01: User Login (UC01)



4.2.3.2 SD-02: User Logout (UC01)



4.3 Manage Roles & Permissions

This section presents the detailed design of the Manage Roles & Permissions feature (TAB-09 of SCR-16). Only the COO can configure the role matrix (BR-02). The COO selects a role, the system loads its permission matrix (screen/feature x action), the COO toggles permissions and saves. The backend validates the change against security policies (COO role locked, dependent permissions), persists the matrix, evicts the permission cache so the new rules apply immediately to all sessions of that role, and writes a diff payload to the Audit Log (BR-51).

4.3.1 Screen Design

4.3.1.1 Roles & Permissions Matrix Tab (SCR-16 / TAB-09)

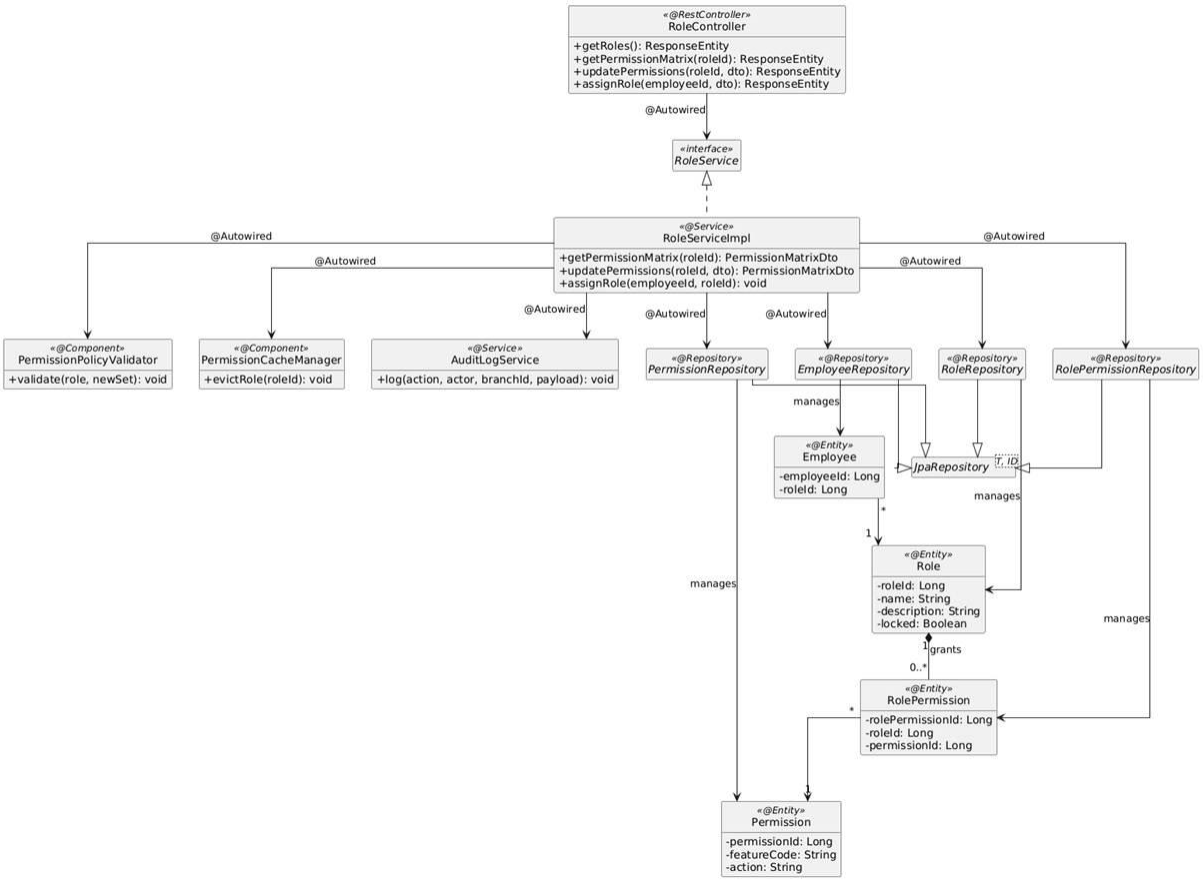
Table 4-3: Roles & Permissions Matrix Tab (SCR-16 / TAB-09) Definition

No	Object/Control Name	Type	Required	Length	Description
1	tabSwitcher	Tabs	-	-	Switches between TAB-10 (Bank & SePay Webhook Config) and TAB-09 (Roles & Permissions Matrix).
2	cboRole	Dropdown	Yes	-	Lists system roles (Staff, Cashier, Branch Manager, COO) loaded from GET /api/v1/roles. Changing the value reloads the matrix.
3	lblRoleDescription	Label	-	Max 200	Scope description of the selected role.
4	lblBoundUsers	Label	-	-	Number of active accounts bound to the role; shows the impact of the change.
5	tblPermissionMatrix	Data Grid	Yes	-	Rows = screens/features, columns = View, Create, Edit, Delete, Approve, Cancel/Refund. Each cell is a checkbox bound to a permission code (e.g. ORDER_REFUND).
6	pnlPolicyNotice	Alert Box	-	-	Explains locked rules (COO matrix read-only, dependent permissions) and shows validation errors returned by the server.

7	btnSavePermissions	Button	Yes	-	Calls PUT /api/v1/roles/{roleId}/permissions; on success shows MSG03, on policy violation shows the error and reverts unsaved cells.
8	btnResetDefault	Button	No	-	Restores the default permission template of the selected role (requires confirmation).
9	btnCancelChanges	Button	No	-	Discards local edits and reloads the stored matrix.

4.3.2 Class/Component Design

4.3.2.1 Classes/Component Structure



4.3.2.2 Classes/Component Description

4.3.2.2.1 RoleController Class

Class	RoleController

Description	Spring REST Controller exposing role listing, permission matrix retrieval/update and role assignment to employees. Secured with @PreAuthorize("hasRole('COO')").
Base Class	None (@RestController, @RequestMapping("/api/v1/roles"))
Constructor	+RoleController(RoleService roleService)
Prototype	public class RoleController
Source File	RoleController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
roleService	RoleService - Service interface injected via @Autowired.
Methods	
Name	**Input / Output / Description**
getRoles	Input: None Output: ResponseEntity<List<RoleDto>> Description: GET endpoint returning all roles with description and bound user count.
getPermissionMatrix	Input: @PathVariable Long roleId Output: ResponseEntity<PermissionMatrixDto> Description: GET /{roleId}/permissions. Returns every feature with the granted actions of the role.
updatePermissions	Input: @PathVariable Long roleId, @RequestBody @Valid UpdatePermissionMatrixDto dto Output: ResponseEntity<PermissionMatrixDto> Description: PUT /{roleId}/permissions. Validates and saves the new matrix (UC02 main flow, BR-02).
assignRole	Input: @PathVariable Long employeeId, @RequestBody AssignRoleDto dto Output: ResponseEntity<Void> Description: PUT /assignments/{employeeId}. Assigns a role to an employee account.

4.3.2.2.2 RoleServiceImpl Class

Class	RoleServiceImpl
Description	Service implementation managing the RBAC matrix: loading, policy validation, persistence, cache eviction and auditing.
Base Class	None (implements RoleService)
Constructor	+RoleServiceImpl(RoleRepository roleRepo, PermissionRepository permissionRepo, RolePermissionRepository rolePermissionRepo, EmployeeRepository employeeRepo, PermissionPolicyValidator policyValidator, PermissionCacheManager permissionCache, AuditLogService auditLogService)
Prototype	public class RoleServiceImpl implements RoleService

Source File	RoleServiceImpl.java
Namespace	com.scms.service.impl
Attributes	
Name	**Type & Description**
roleRepo	RoleRepository - Reads roles.
permissionRepo	PermissionRepository - Reads the catalog of permission codes (feature + action).
rolePermissionRepo	RolePermissionRepository - Persists role-permission mappings.
employeeRepo	EmployeeRepository - Updates the role of an employee.
policyValidator	PermissionPolicyValidator - Checks security constraints of a permission set.
permissionCache	PermissionCacheManager - Evicts cached authorities so changes take effect immediately.
auditLogService	AuditLogService - Writes PERMISSION_UPDATE entries with before/after diff.
Methods	
Name	**Input / Output / Description**
getPermissionMatrix	Input: Long roleId Output: PermissionMatrixDto Description: Loads all permissions and marks the ones granted to the role.
updatePermissions	Input: Long roleId, UpdatePermissionMatrixDto dto Output: PermissionMatrixDto Description: @Transactional. Loads the old set, calls policyValidator.validate(), replaces role_permission rows, evicts the cache, logs the diff. Throws PermissionPolicyException on violation (AT1).
assignRole	Input: Long employeeId, Long roleId Output: void Description: Changes the role of an employee and logs ROLE_ASSIGN.

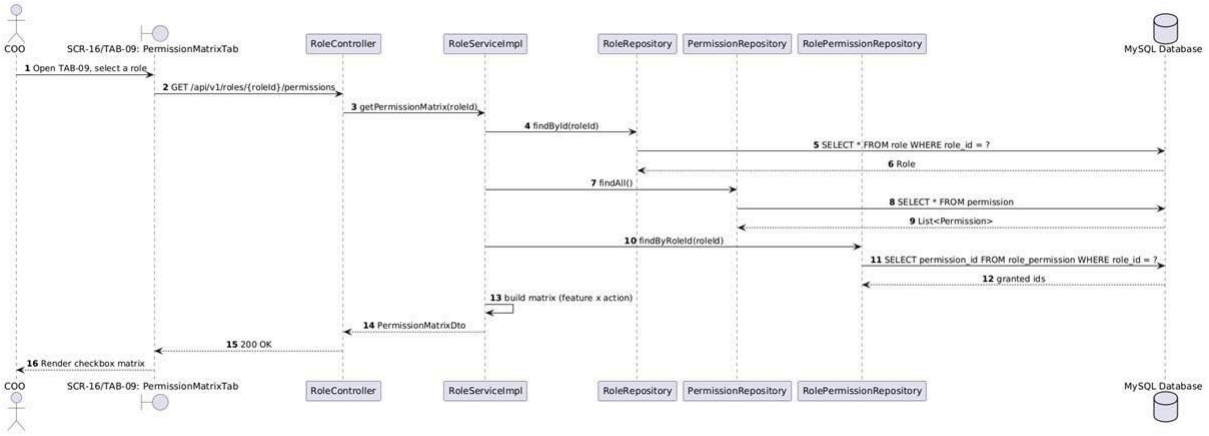
4.3.2.2.3 PermissionPolicyValidator Class

Class	PermissionPolicyValidator
Description	Component that enforces RBAC security constraints before saving.
Base Class	None (@Component)
Constructor	+PermissionPolicyValidator()
Prototype	public class PermissionPolicyValidator
Source File	PermissionPolicyValidator.java
Namespace	com.scms.security
Attributes	
Name	**Type & Description**

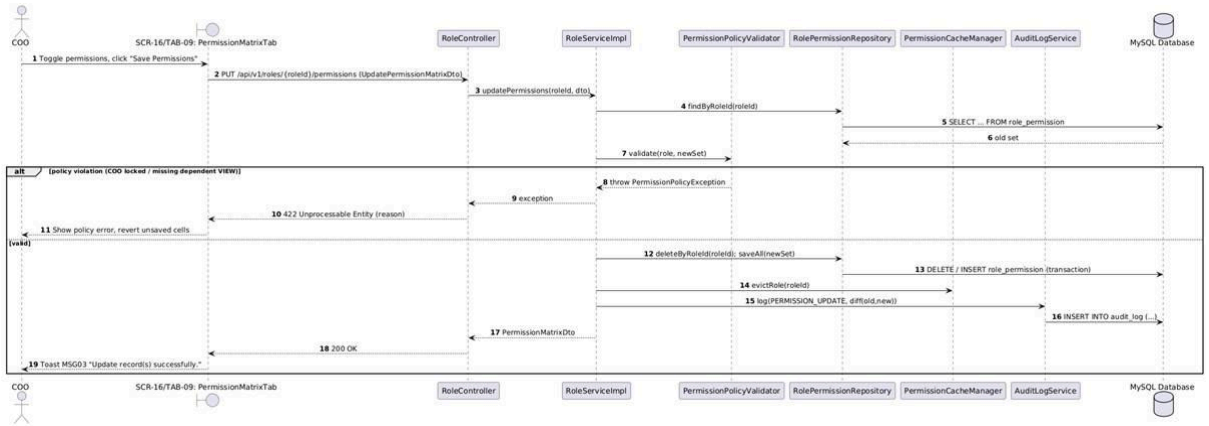
-	None (stateless component)
Methods	
Name	**Input / Output / Description**
validate	Input: Role role, Set<Permission> newSet Output: void Description: Rejects any change on the COO role (BR-02), requires VIEW when CREATE/EDIT/DELETE/APPROVE is granted, and forbids granting cross-branch permissions to branch roles (BR-04). Throws PermissionPolicyException with a reason message.

4.3.3 Object Interactions (Sequence Diagrams)

4.3.3.1 SD-01: View Role Permission Matrix (UC02)



4.3.3.2 SD-02: Update Role Permissions (UC02)



4.4 Check In

This section presents the detailed design of the Check In feature executed on the Attendance Kiosk (SCR-02). Staff and Cashiers authenticate either by swiping an RFID card (BR-06) or, as a fallback, by selecting their name and entering a 4-digit PIN (BR-08). The backend verifies the PIN hash with lockout protection, matches the current time against the employee's shift assignment for today (UC08), records the check-in timestamp with the

punctuality status and returns the result banner. An unscheduled check-in is still recorded but flagged for manager review (AT2).

4.4.1 Screen Design

4.4.1.1 Attendance Kiosk Screen - Check In (SCR-02)

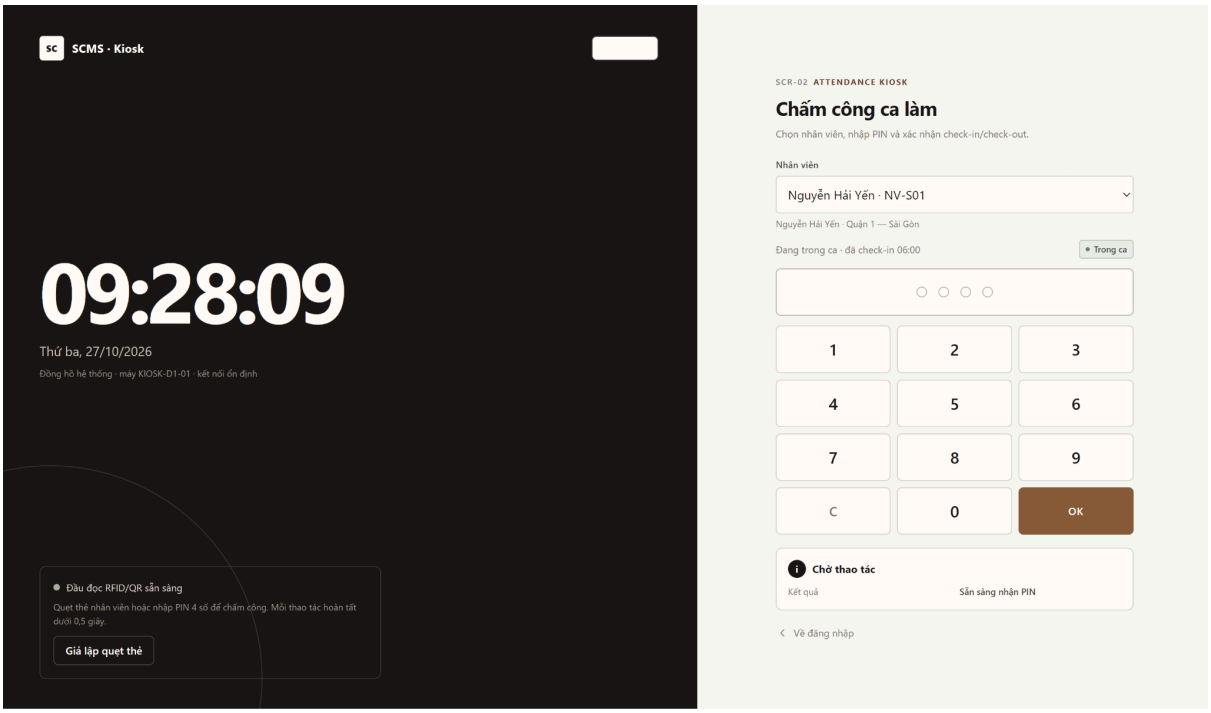


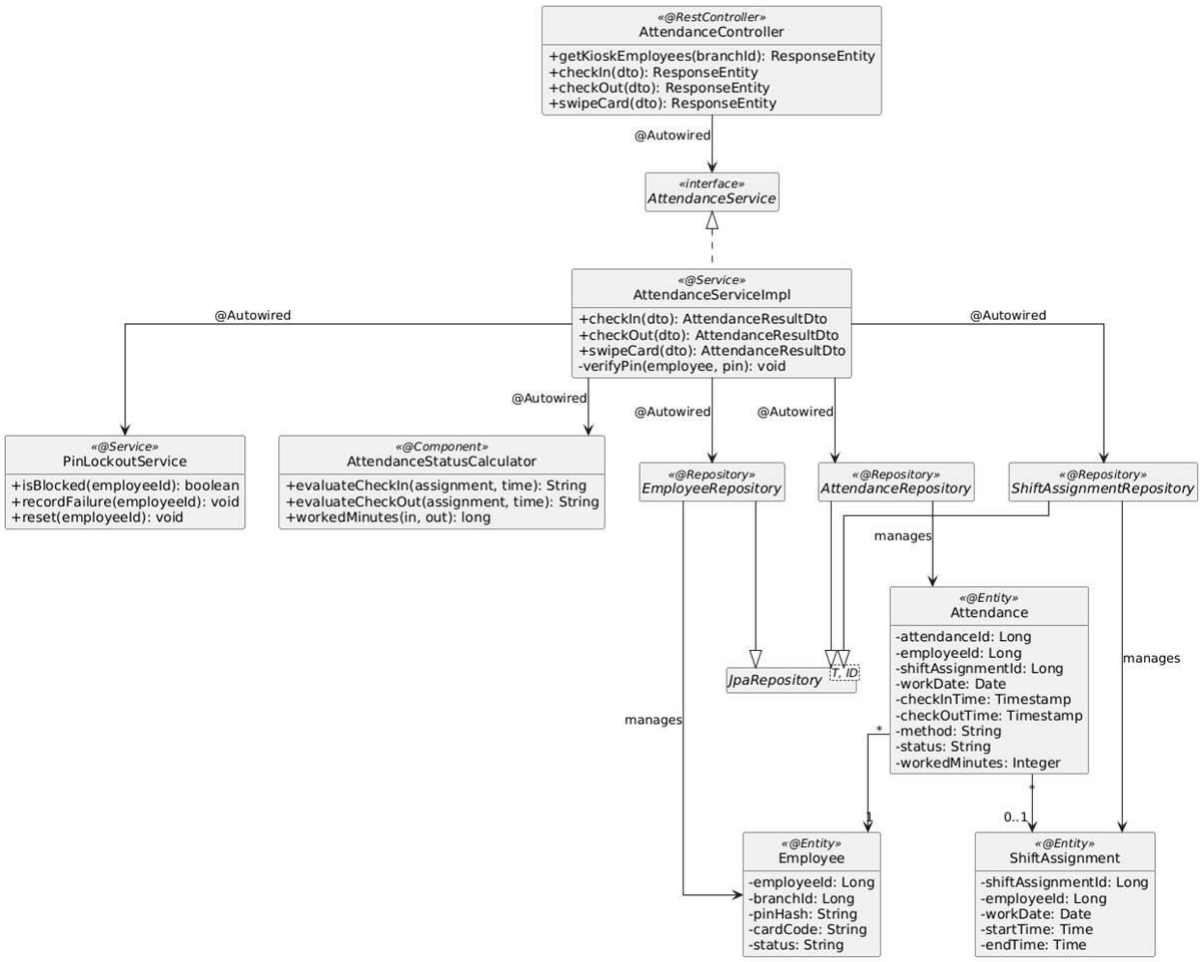
Table 4-4: Attendance Kiosk Screen - Check In (SCR-02) Definition

No	Object/Control Name	Type	Required	Length	Description
1	lblSystemClock	Label	-	-	Real-time clock synchronised with the server time (GET /api/v1/system/time every 60s).
2	btnBackToLogin	Button	No	-	Returns to SCR-01 User Login.
3	pnlRfidScanArea	Component	-	-	Captures keyboard-wedge input from the USB RFID/QR reader and calls POST /api/v1/attendance/card-swipe (BR-06).
4	lblReaderStatus	Label	-	Max 50	Reader connection status, e.g. "Ready for check-in/out".

5	cboEmployee	Dropdown	Yes	-	Active employees of the branch scheduled today (Full Name + Employee ID); used by the PIN fallback (BR-08).
6	txtPin	Password	Yes	4	Masked PIN display; exactly 4 digits.
7	btnNumpad[0-9]	Button Grid	-	-	Large 44x44 px touch keys to enter the PIN.
8	btnClear	Button	No	-	Clears the entered PIN digits.
9	btnSubmitOk	Button	Yes	-	Submits POST /api/v1/attendance/check-in with employeeId and PIN.
10	lblResultBanner	Label / Box	-	Max 150	Shows the latest result: name, ID, Check-in, timestamp, On Time / Late or warning "No scheduled shift found for today".

4.4.2 Class/Component Design

4.4.2.1 Classes/Component Structure



4.4.2.2 Classes/Component Description

4.4.2.2.1 AttendanceController Class

Class	AttendanceController
Description	Spring REST Controller serving the Attendance Kiosk (SCR-02): employee list for the PIN fallback, PIN check-in/check-out and RFID card swipe.
Base Class	None (@RestController, @RequestMapping("/api/v1/attendance"))
Constructor	+AttendanceController(AttendanceService attendanceService)
Prototype	public class AttendanceController
Source File	AttendanceController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
attendanceService	AttendanceService - Service interface injected via @Autowired.
Methods	
Name	**Input / Output / Description**

getKioskEmployees	Input: @RequestParam Long branchId Output: ResponseEntity<List<KioskEmployeeDto>> Description: GET /kiosk/employees. Active Staff/Cashier of the branch scheduled today (Employee Selector).
checkIn	Input: @RequestBody @Valid PinAttendanceRequestDto dto Output: ResponseEntity<AttendanceResultDto> Description: POST /check-in. PIN-based check-in (UC03).
checkOut	Input: @RequestBody @Valid PinAttendanceRequestDto dto Output: ResponseEntity<AttendanceResultDto> Description: POST /check-out. PIN-based check-out (UC04).
swipeCard	Input: @RequestBody @Valid CardSwipeRequestDto dto Output: ResponseEntity<AttendanceResultDto> Description: POST /card-swipe. RFID swipe: records check-in if no open record, otherwise check-out (BR-06).

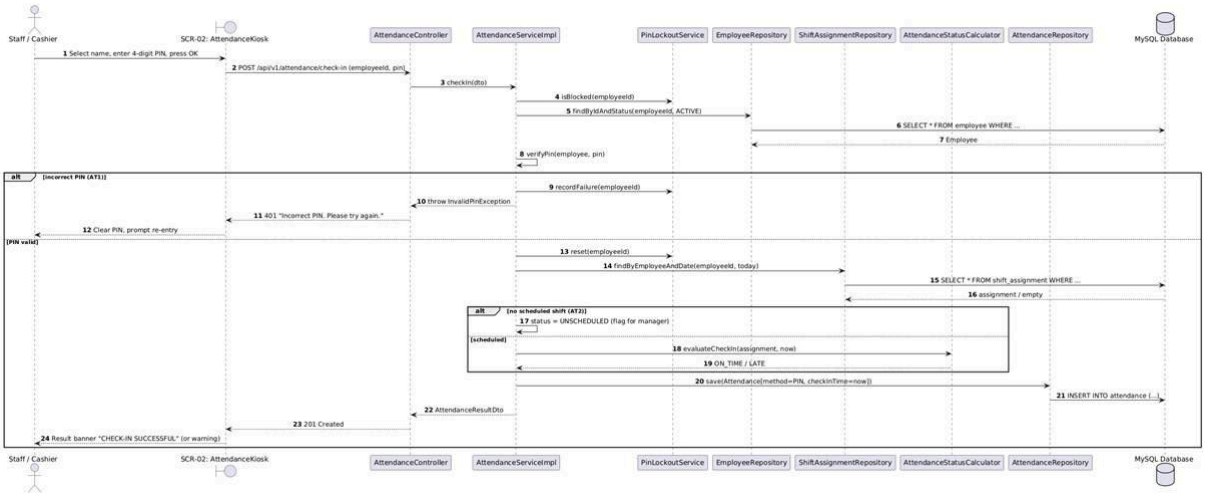
4.4.2.2.2 AttendanceServiceImpl Class

Class	AttendanceServiceImpl
Description	Service implementation handling kiosk authentication and attendance recording.
Base Class	None (implements AttendanceService)
Constructor	+AttendanceServiceImpl(EmployeeRepository employeeRepo, AttendanceRepository attendanceRepo, ShiftAssignmentRepository assignmentRepo, PasswordEncoder pinEncoder, PinLockoutService pinLockoutService, AttendanceStatusCalculator statusCalculator)
Prototype	public class AttendanceServiceImpl implements AttendanceService
Source File	AttendanceServiceImpl.java
Namespace	com.scms.service.impl
Attributes	
Name	**Type & Description**
employeeRepo	EmployeeRepository - Loads the employee and PIN hash / card code.
attendanceRepo	AttendanceRepository - Persists and queries attendance records.
assignmentRepo	ShiftAssignmentRepository - Finds the scheduled shift of the employee for today (UC08 data).
pinEncoder	PasswordEncoder - BCrypt encoder comparing the 4-digit PIN with its hash.
pinLockoutService	PinLockoutService - Blocks an employee for 5 minutes after 5 consecutive wrong PINs.
statusCalculator	AttendanceStatusCalculator - Computes On Time / Late / Early Leave status and worked minutes.
Methods	

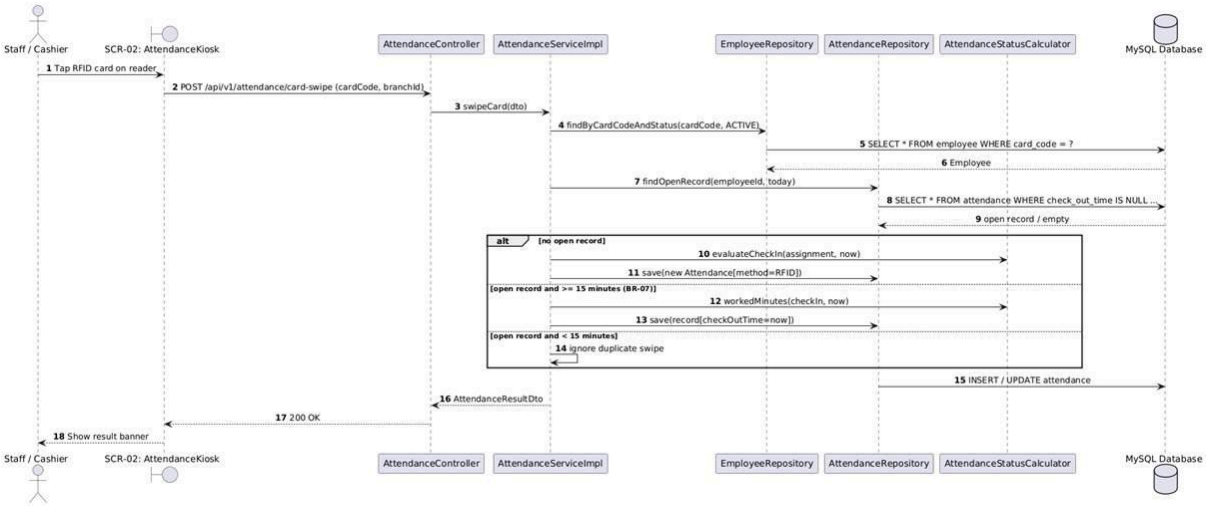
Name	Input / Output / Description
checkIn	Input: PinAttendanceRequestDto dto Output: AttendanceResultDto Description: @Transactional. Checks lockout, verifies PIN (AT1), rejects a duplicate open check-in, finds today's ShiftAssignment, evaluates status (On Time/Late) or flags UNSCHEDULED (AT2), saves the Attendance with method = PIN.
swipeCard	Input: CardSwipeRequestDto dto Output: AttendanceResultDto Description: Finds the employee by cardCode; if an open record exists and >= 15 minutes elapsed (BR-07) performs check-out, else performs check-in with method = RFID.
verifyPin	Input: Employee employee, String pin Output: void Description: Compares the PIN with pinHash; on failure increments the lockout counter and throws InvalidPinException ("Incorrect PIN. Please try again.").

4.4.3 Object Interactions (Sequence Diagrams)

4.4.3.1 SD-01: Check In by PIN (UC03)



4.4.3.2 SD-02: Check In / Out by RFID Card Swipe (UC03)



4.5 Check Out

This section presents the detailed design of the Check Out feature. It reuses the Attendance Kiosk components of UC03. The employee authenticates by PIN (or RFID card, see 4.4.3 SD-02); the system verifies that an open check-in record exists for the current day, enforces the 15-minute minimum interval (BR-07), records the check-out timestamp, computes the total worked time and early-leave status against the assigned shift, and displays the result. If no open check-in exists the employee is asked to contact the Branch Manager for a manual adjustment (BR-10).

4.5.1 Screen Design

4.4.1.1 Attendance Kiosk Screen - Check In (SCR-02)

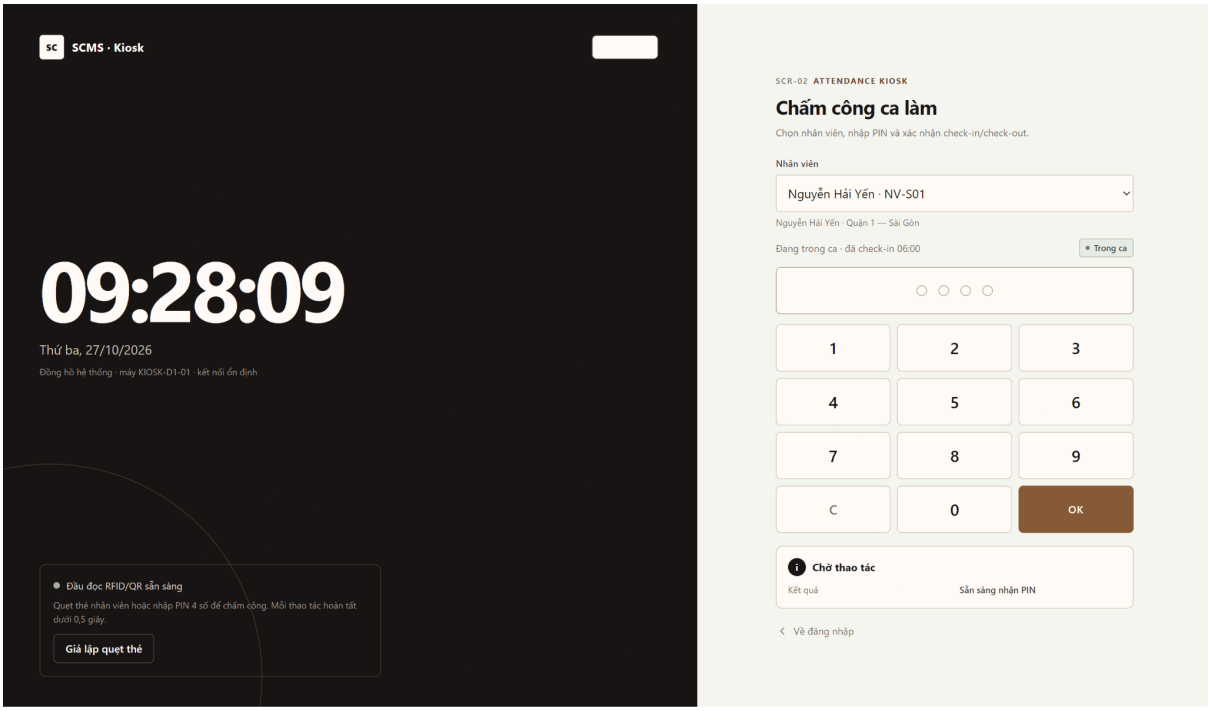
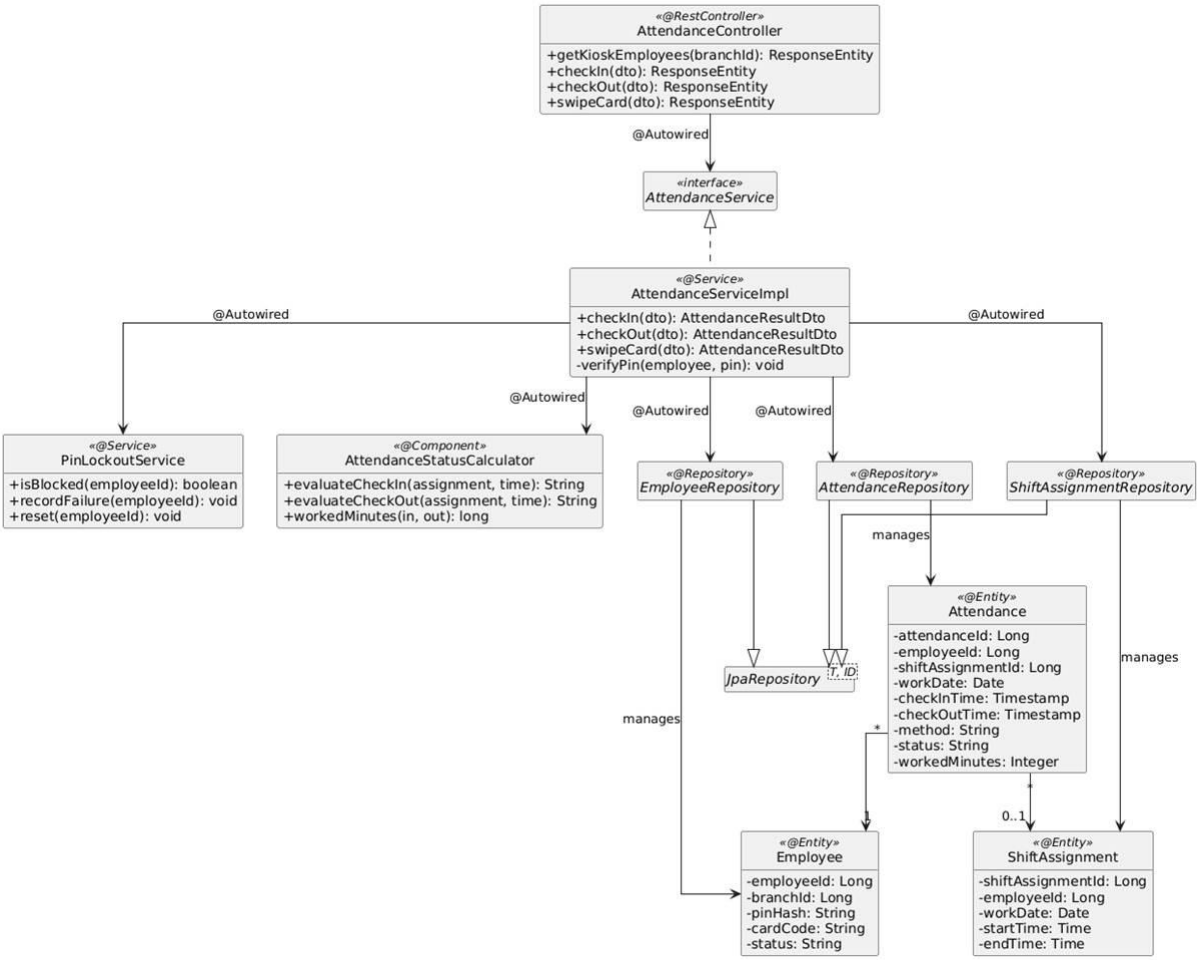


Table 4-4: Attendance Kiosk Screen - Check In (SCR-02) Definition

No	Object/Control Name	Type	Required	Length	Description
1	lblSystemClock	Label	-	-	Real-time clock synchronised with the server time (GET /api/v1/system/time every 60s).
2	btnBackToLogin	Button	No	-	Returns to SCR-01 User Login.
3	pnlRfidScanArea	Component	-	-	Captures keyboard-wedge input from the USB RFID/QR reader and calls POST /api/v1/attendance/card-swipe (BR-06).
4	lblReaderStatus	Label	-	Max 50	Reader connection status, e.g. "Ready for check-in/out".
5	cboEmployee	Dropdown	Yes	-	Active employees of the branch scheduled today (Full Name + Employee ID); used by the PIN fallback (BR-08).
6	txtPin	Password	Yes	4	Masked PIN display; exactly 4 digits.
7	btnNumpad[0-9]	Button Grid	-	-	Large 44x44 px touch keys to enter the PIN.
8	btnClear	Button	No	-	Clears the entered PIN digits.
9	btnSubmitOk	Button	Yes	-	Submits POST /api/v1/attendance/check-in with employeeId and PIN.
10	lblResultBanner	Label / Box	-	Max 150	Shows the latest result: name, ID, Check-in, timestamp, On Time / Late or warning "No scheduled shift found for today".

4.5.2 Class/Component Design

4.5.2.1 Classes/Component Structure



4.5.2.2 Classes/Component Description

4.5.2.2.1 AttendanceServiceImpl Class

Class	AttendanceServiceImpl
Description	Service implementation (shared with UC03). Methods specific to the check-out flow are described below.
Base Class	None (implements AttendanceService)
Constructor	+AttendanceServiceImpl(EmployeeRepository employeeRepo, AttendanceRepository attendanceRepo, ShiftAssignmentRepository assignmentRepo, PasswordEncoder pinEncoder, PinLockoutService pinLockoutService, AttendanceStatusCalculator statusCalculator)
Prototype	public class AttendanceServiceImpl implements AttendanceService
Source File	AttendanceServiceImpl.java
Namespace	com.scms.service.impl

Attributes	
Name	**Type & Description**
employeeRepo	EmployeeRepository - Loads the employee and PIN hash / card code.
attendanceRepo	AttendanceRepository - Persists and queries attendance records.
assignmentRepo	ShiftAssignmentRepository - Finds the scheduled shift of the employee for today (UC08 data).
pinEncoder	PasswordEncoder - BCrypt encoder comparing the 4-digit PIN with its hash.
pinLockoutService	PinLockoutService - Blocks an employee for 5 minutes after 5 consecutive wrong PINs.
statusCalculator	AttendanceStatusCalculator - Computes On Time / Late / Early Leave status and worked minutes.
Methods	
Name	**Input / Output / Description**
checkOut	Input: PinAttendanceRequestDto dto Output: AttendanceResultDto Description: @Transactional. Verifies PIN, loads the open attendance record of today (AT1 if none -> NoOpenCheckInException), checks interval >= 15 minutes (BR-07), sets checkOutTime, workedMinutes and final status, saves the record.
findOpenRecord	Input: Long employeeId, LocalDate workDate Output: Optional<Attendance> Description: Returns the attendance row with checkOutTime IS NULL (also covers overnight shifts started the previous day).

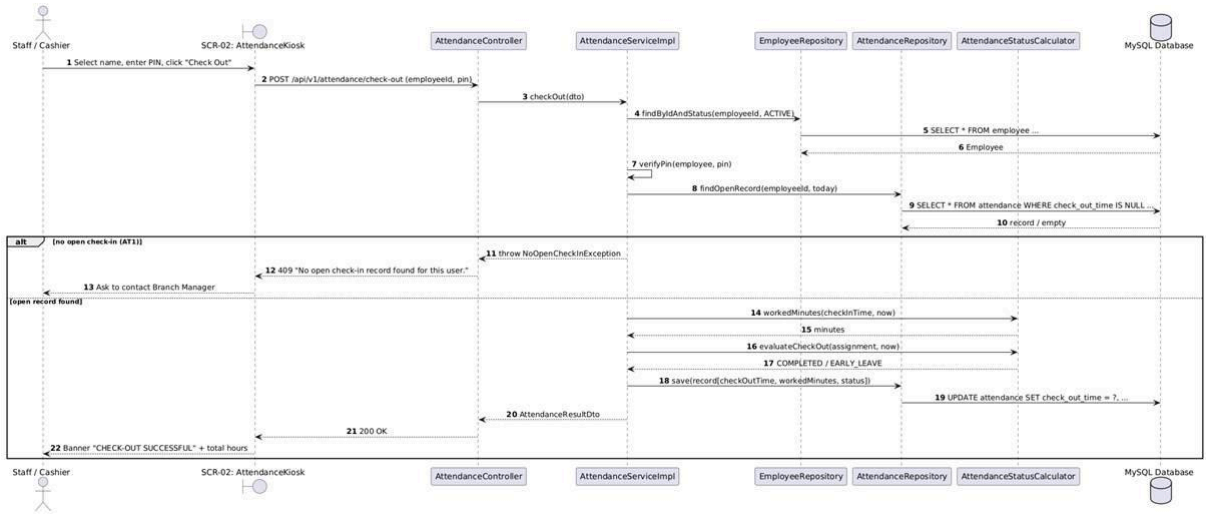
4.5.2.2.2 AttendanceStatusCalculator Class

Class	AttendanceStatusCalculator
Description	Component computing punctuality and worked time.
Base Class	None (@Component)
Constructor	+AttendanceStatusCalculator()
Prototype	public class AttendanceStatusCalculator
Source File	AttendanceStatusCalculator.java
Namespace	com.scms.service.support
Attributes	
Name	**Type & Description**
-	None (stateless component)
Methods	
Name	**Input / Output / Description**
evaluateCheckOut	Input: ShiftAssignment assignment, LocalDateTime time Output: String

	Description: Returns EARLY_LEAVE when time < shift end - tolerance, otherwise COMPLETED.
workedMinutes	Input: LocalDateTime checkIn, LocalDateTime checkOut Output: long Description: Duration in minutes between check-in and check-out used for payroll.

4.5.3 Object Interactions (Sequence Diagrams)

4.5.3.1 SD-01: Check Out by PIN (UC04)



4.6 Manage Branch

This section presents the detailed design of the Manage Branch feature on SCR-13 Branches & Managers. The COO (the only authorised role, BR-11) views the branch list, creates a new branch, edits its profile (code, name, address, hotline, opening hours, table count, tax code) or changes its operational status (Open, Temporarily Closed, Maintenance, Inactive). Branches are never hard-deleted; deactivation is a status change so that historical orders, shifts and inventory stay intact. Every change is written to the Audit Log.

4.6.1 Screen Design

4.6.1.1 Branches & Managers Screen (SCR-13)

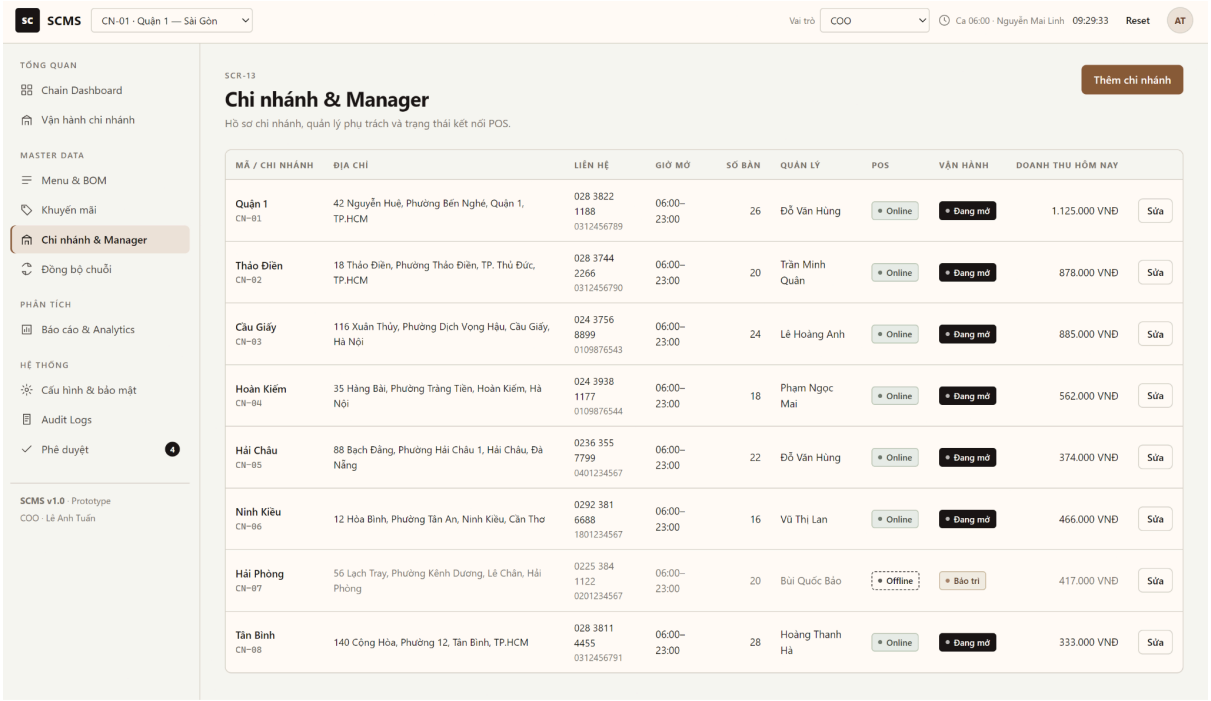


Table 4-6: Branches & Managers Screen (SCR-13) Definition

No	Object/Control Name	Type	Required	Length	Description
1	btnAddBranch	Button	No	-	Opens POP-13A in create mode.
2	txtSearchBranch	Textbox	No	Max 50	Live search (300 ms debounce) by branch code or name.
3	tblBranches	Data Grid	Yes	-	Branch Code, Name, Address, Phone, Assigned Manager, Manager Account, POS Status, Action. Paginated, loaded from GET /api/v1/branches.
4	lblPosStatus	Label	-	20	Online / Offline connection status of the branch POS.

5	btnEditBranch	Button	No	-	Opens POP-13A pre-filled with the selected branch.
6	btnNavSidebar	Menu	-	-	HQ sidebar navigation (SCR-10 ... SCR-17).

4.6.1.2 Create / Edit Branch Profile Popup (POP-13A)

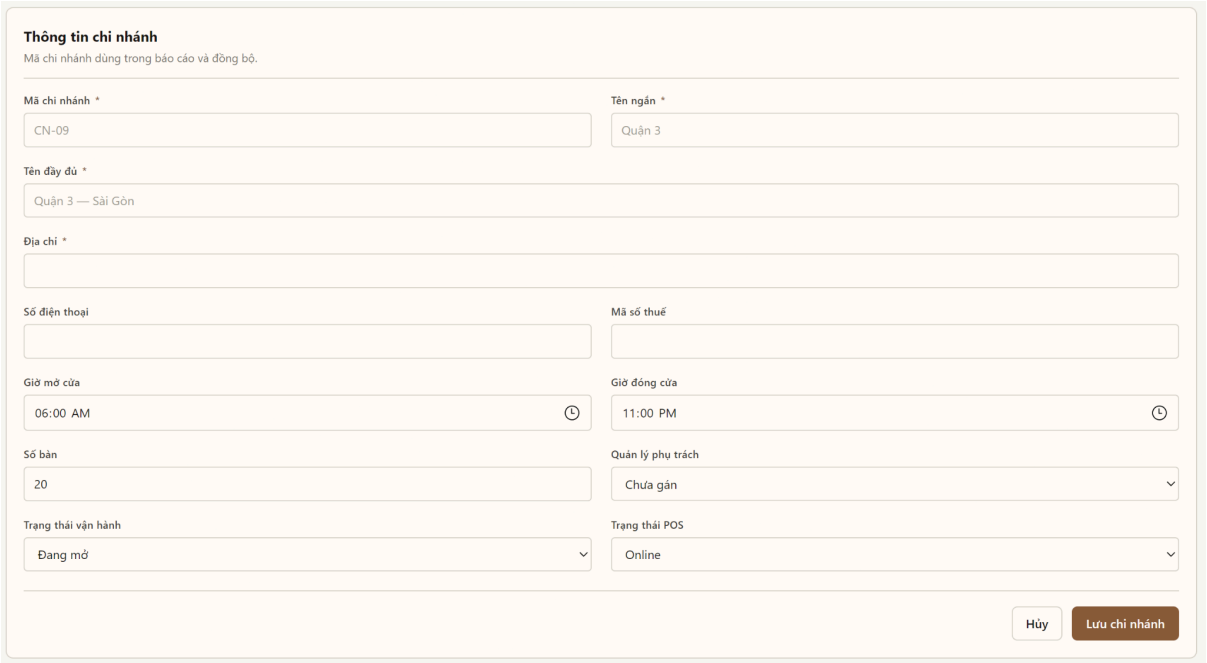


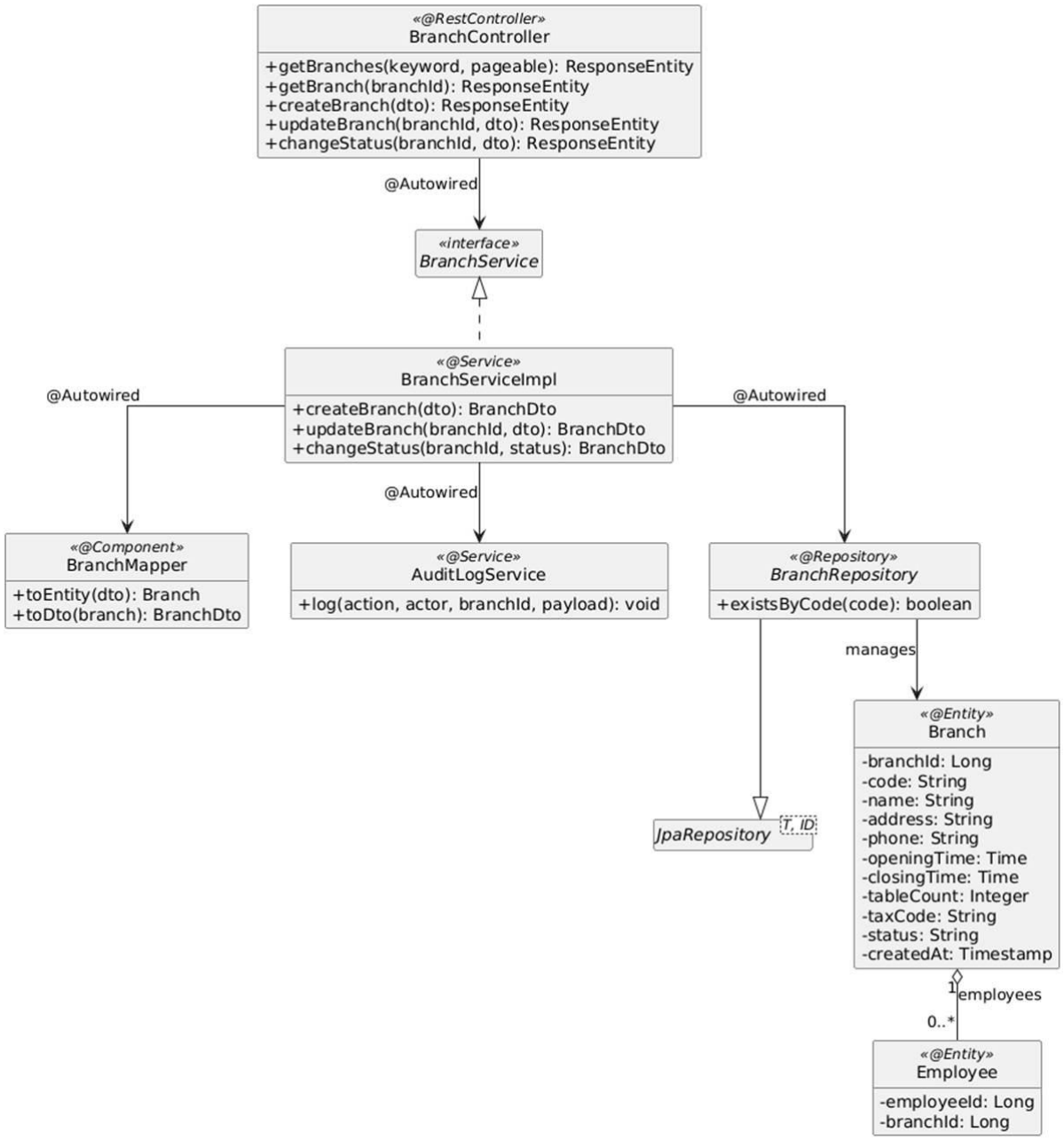
Table 4-7: Create / Edit Branch Profile Popup (POP-13A) Definition

No	Object/Control Name	Type	Required	Length	Description
1	txtBranchCode	Textbox	Yes	Max 10	Unique branch code (e.g. CN04). Read-only in edit mode.
2	txtBranchName	Textbox	Yes	Max 100	Branch display name.
3	txtHotline	Textbox	No	10 - 11	Digits only, starting with 0.

4	txtAddress	Textbox	Yes	Max 255	Physical address.
5	tpOpeningTime / tpClosingTime	Time Picker	Yes	5	Operating hours; closing time must be later than opening time.
6	txtTableCount	Number	No	3	Number of tables at the branch (>= 0).
7	txtTaxCode	Textbox	No	Max 20	Branch tax code.
8	cboStatus	Dropdown	Yes	-	Open / Temporarily Closed / Maintenance / Inactive.
9	lblAssignedManager	Dropdown	No	-	Displays the bound Branch Manager; assignment itself is done in UC06.
10	btnSaveBranch	Button	Yes	-	POST /api/v1/branches (create) or PUT /api/v1/branches/{id} (update); shows MSG04 / MSG03.
11	btnDeactivate	Button	No	-	Confirmation popup then PATCH /api/v1/branches/{id}/status.
12	btnCancel	Button	No	-	Closes the popup without saving.

4.6.2 Class/Component Design

4.6.2.1 Classes/Component Structure



4.6.2.2 Classes/Component Description

4.6.2.2.1 BranchController Class

Class	BranchController
Description	Spring REST Controller exposing CRUD-style branch management endpoints for the COO (@PreAuthorize("hasRole('COO')"), BR-11).
Base Class	None (@RestController, @RequestMapping("/api/v1/branches"))
Constructor	+BranchController(BranchService branchService)

Prototype	public class BranchController
Source File	BranchController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
branchService	BranchService - Service interface injected via @Autowired.
Methods	
Name	**Input / Output / Description**
getBranches	Input: @RequestParam String keyword, Pageable pageable Output: ResponseEntity<Page<BranchDto>> Description: GET endpoint listing branches with assigned manager and POS status.
getBranch	Input: @PathVariable Long branchId Output: ResponseEntity<BranchDetailDto> Description: GET /{branchId}. Loads one branch for the edit form.
createBranch	Input: @RequestBody @Valid CreateBranchDto dto Output: ResponseEntity<BranchDto> Description: POST endpoint creating a branch (UC05 main flow).
updateBranch	Input: @PathVariable Long branchId, @RequestBody @Valid UpdateBranchDto dto Output: ResponseEntity<BranchDto> Description: PUT endpoint updating the branch profile.
changeStatus	Input: @PathVariable Long branchId, @RequestBody ChangeBranchStatusDto dto Output: ResponseEntity<BranchDto> Description: PATCH /{branchId}/status. Changes operational status / deactivates.

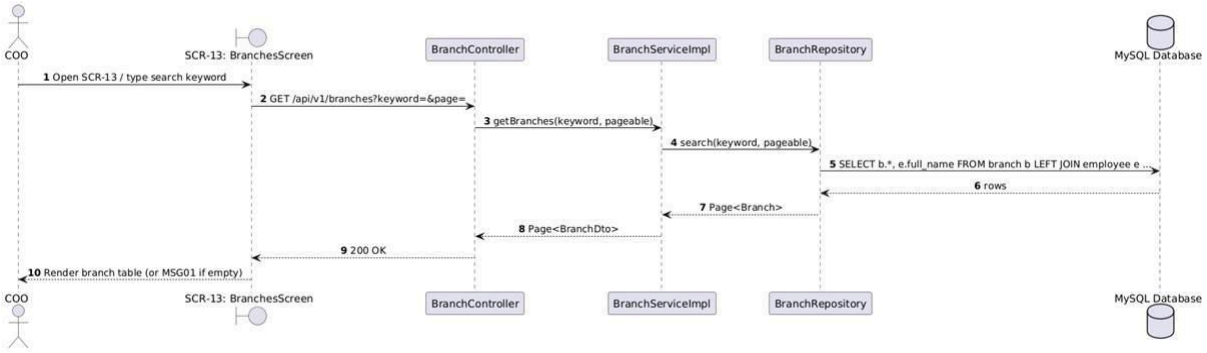
4.6.2.2.2 BranchServiceImpl Class

Class	BranchServiceImpl
Description	Service implementation validating and persisting branch profiles.
Base Class	None (implements BranchService)
Constructor	+BranchServiceImpl(BranchRepository branchRepo, BranchMapper branchMapper, AuditLogService auditLogService)
Prototype	public class BranchServiceImpl implements BranchService
Source File	BranchServiceImpl.java
Namespace	com.scms.service.impl
Attributes	
Name	**Type & Description**
branchRepo	BranchRepository - Persists branch records.

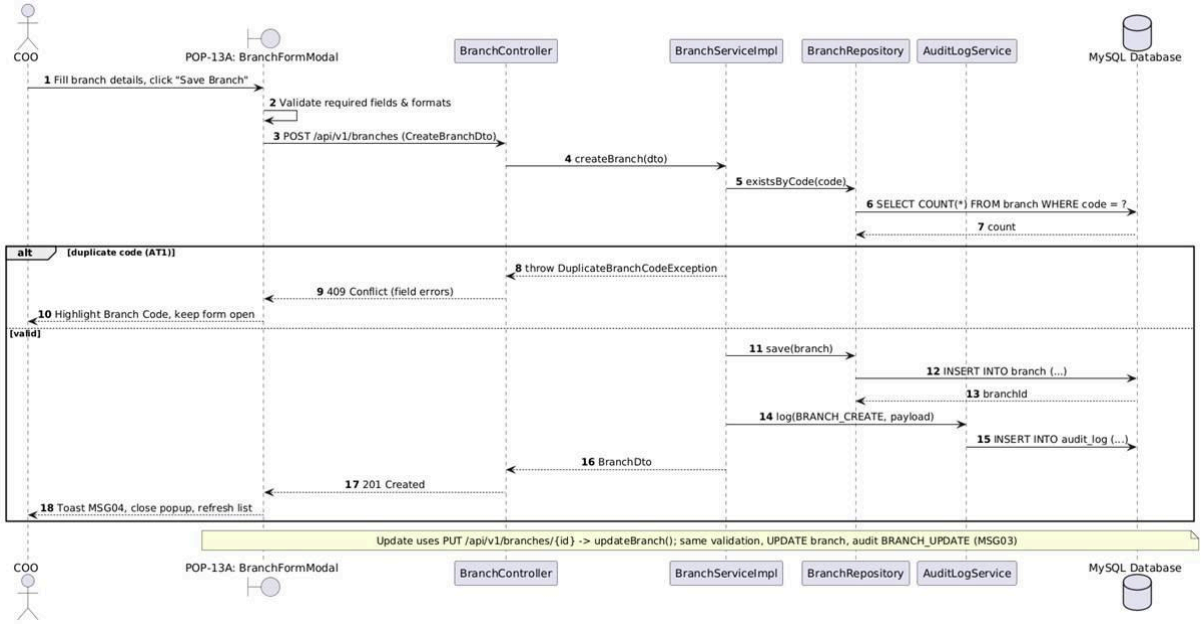
branchMapper	BranchMapper - Maps DTOs <-> Branch entity.
auditLogService	AuditLogService - Logs BRANCH_CREATE / BRANCH_UPDATE / BRANCH_STATUS with diff payload.
Methods	
Name	**Input / Output / Description**
createBranch	Input: CreateBranchDto dto Output: BranchDto Description: Checks existsByCode (AT1 -> DuplicateBranchCodeException), validates opening < closing time, saves with createdAt, logs audit.
updateBranch	Input: Long branchId, UpdateBranchDto dto Output: BranchDto Description: Loads the branch (404 if missing), applies changes, saves, logs before/after diff.
changeStatus	Input: Long branchId, BranchStatus status Output: BranchDto Description: Updates status (soft deactivation - no hard delete) and logs the operation.

4.6.3 Object Interactions (Sequence Diagrams)

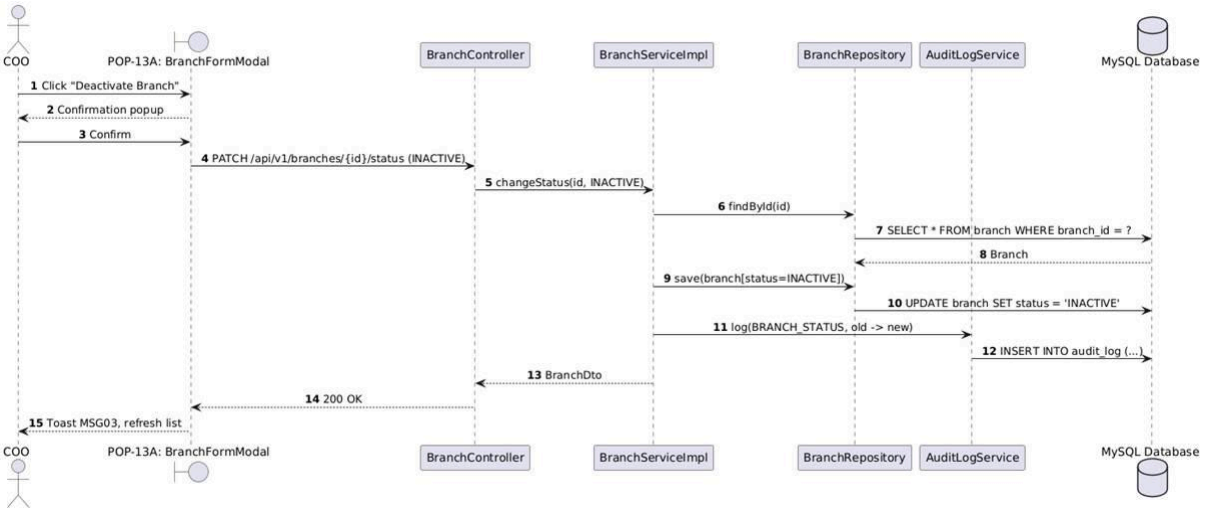
4.6.3.1 SD-01: View Branch List (UC05)



4.6.3.2 SD-02: Create / Update Branch (UC05)



4.6.3.3 SD-03: Deactivate / Change Branch Status (UC05)



4.7 Manage Branch Manager

This section presents the detailed design of the Manage Branch Manager feature. From SCR-13 the COO opens POP-13B to create a Branch Manager account or edit an existing one: profile, credentials (username, password hashed with BCrypt), assigned branch scope, approval limit and status. Each manager belongs to exactly one branch; only the COO can reassign it, the reassignment is written to the Audit Log and historical data remains with the original branch (BR-12). Username and email must be unique (AT1).

4.7.1 Screen Design

4.7.1.1 Create / Edit Branch Manager Account Popup (POP-13B)

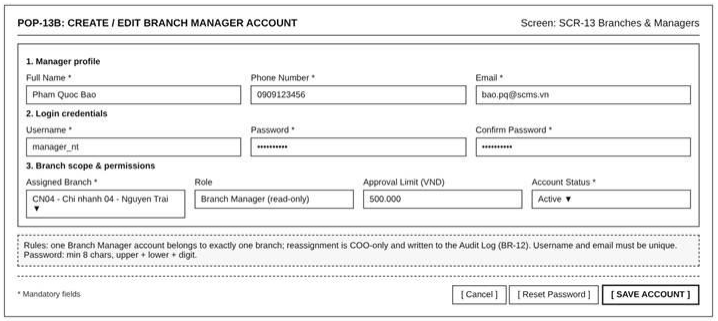


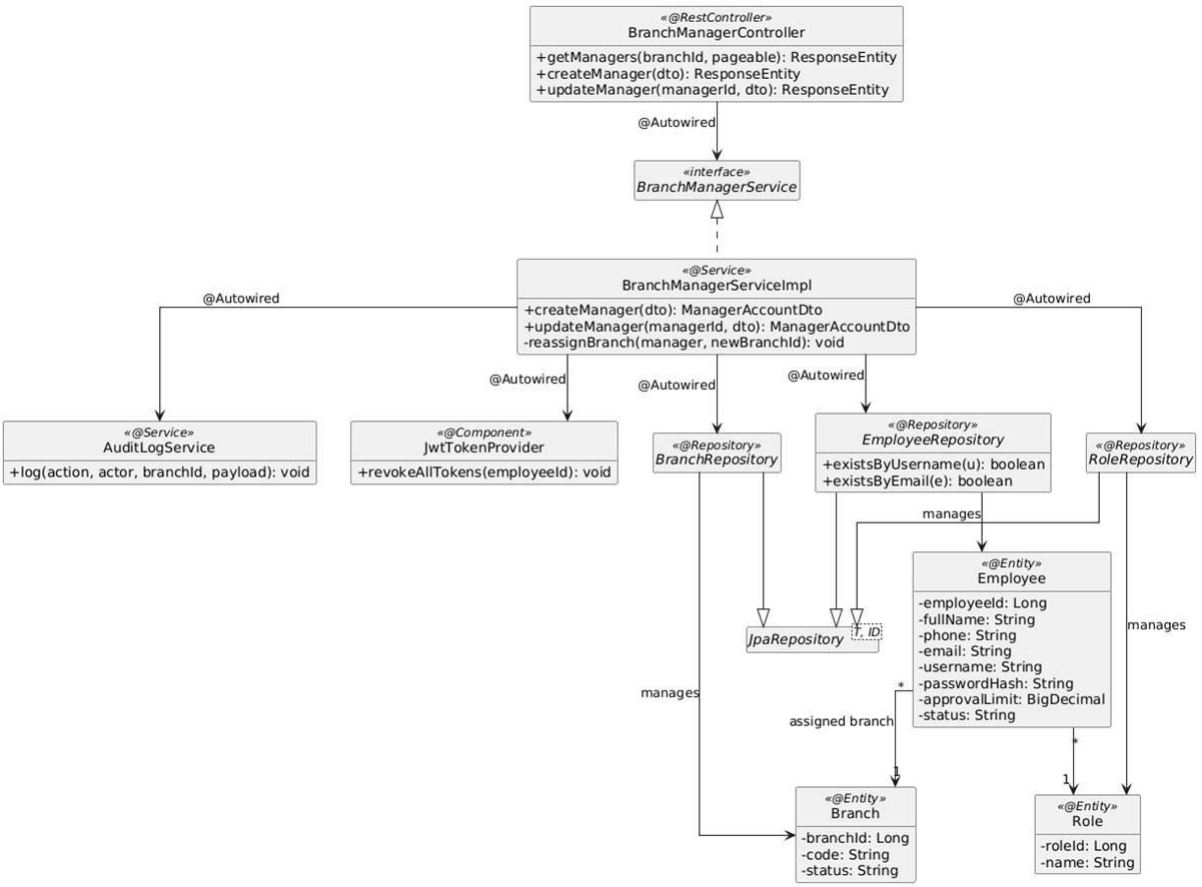
Table 4-8: Create / Edit Branch Manager Account Popup (POP-13B) Definition

No	Object/Control Name	Type	Required	Length	Description
1	txtFullName	Textbox	Yes	Max 100	Manager full name.
2	txtPhone	Textbox	Yes	10 - 11	Digits only, starting with 0.
3	txtEmail	Textbox	Yes	Max 100	Unique, valid e-mail format.
4	txtUsername	Textbox	Yes	4 - 30	Unique login username. Read-only in edit mode.
5	txtPassword / txtConfirmPassword	Password	Yes (create)	8 - 32	Strong password (upper, lower, digit). Optional in edit mode; stored as BCrypt hash.

6	cboAssignedBranch	Dropdown	Yes	-	Active branches loaded from GET /api/v1/branches?status=OPEN. Changing it in edit mode triggers a reassignment (BR-12).
7	lblRole	Label	-	-	Fixed role Branch Manager.
8	txtApprovalLimit	Number	No	15	Approval threshold (VND) for refunds / cash adjustments (BR-35).
9	cboAccountStatus	Dropdown	Yes	-	Active / Locked / Inactive.
10	btnResetPassword	Button	No	-	Generates a temporary password for the manager.
11	btnSaveAccount	Button	Yes	-	POST /api/v1/branch-managers or PUT /api/v1/branch-managers/{id}; shows MSG04 / MSG03 or "Username/Email already exists in the system."
12	btnCancel	Button	No	-	Closes the popup without saving.

4.7.2 Class/Component Design

4.7.2.1 Classes/Component Structure



4.7.2.2 Classes/Component Description

4.7.2.2.1 BranchManagerController Class

Class	BranchManagerController
Description	Spring REST Controller for Branch Manager account management, restricted to the COO.
Base Class	None (@RestController, @RequestMapping("/api/v1/branch-managers"))
Constructor	+BranchManagerController(BranchManagerService managerService)
Prototype	public class BranchManagerController
Source File	BranchManagerController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
managerService	BranchManagerService - Service interface injected via @Autowired.
Methods	
Name	**Input / Output / Description**

getManagers	Input: @RequestParam Long branchId, Pageable pageable Output: ResponseEntity<Page<ManagerAccountDto>> Description: GET endpoint listing manager accounts and their branches.
createManager	Input: @RequestBody @Valid CreateManagerAccountDto dto Output: ResponseEntity<ManagerAccountDto> Description: POST endpoint creating an account bound to a branch (UC06 main flow).
updateManager	Input: @PathVariable Long managerId, @RequestBody @Valid UpdateManagerAccountDto dto Output: ResponseEntity<ManagerAccountDto> Description: PUT endpoint updating profile, status, approval limit, password or assigned branch.

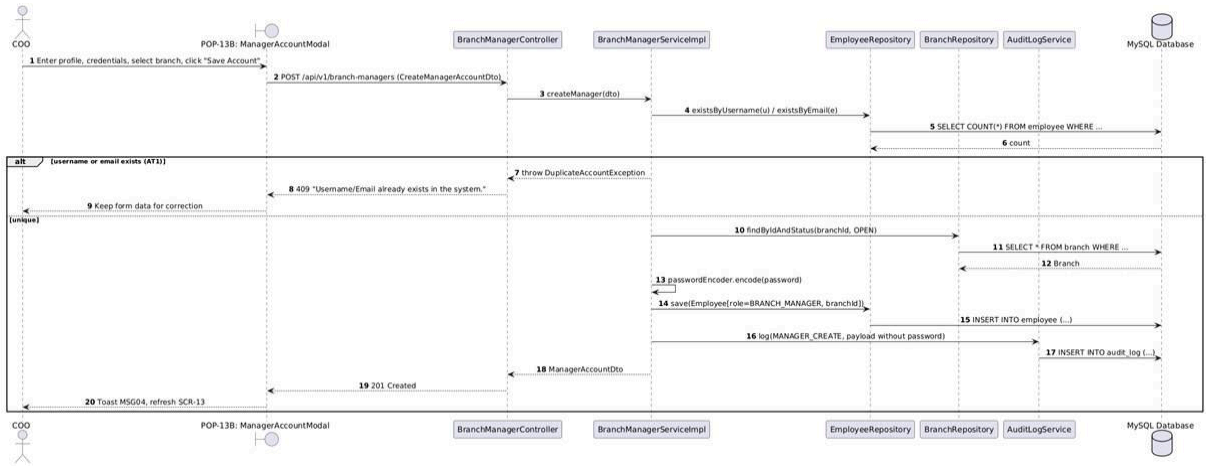
4.7.2.2.2 BranchManagerServiceImpl Class

Class	BranchManagerServiceImpl
Description	Service implementation validating and persisting Branch Manager accounts and their branch scope.
Base Class	None (implements BranchManagerService)
Constructor	+BranchManagerServiceImpl(EmployeeRepository employeeRepo, BranchRepository branchRepo, RoleRepository roleRepo, PasswordEncoder passwordEncoder, AuditLogService auditLogService)
Prototype	public class BranchManagerServiceImpl implements BranchManagerService
Source File	BranchManagerServiceImpl.java
Namespace	com.scms.service.impl
Attributes	
Name	**Type & Description**
employeeRepo	EmployeeRepository - Persists the manager account (Employee with role BRANCH_MANAGER).
branchRepo	BranchRepository - Validates the target branch.
roleRepo	RoleRepository - Loads the BRANCH_MANAGER role.
passwordEncoder	PasswordEncoder - Hashes the password (BCrypt).
auditLogService	AuditLogService - Logs MANAGER_CREATE / MANAGER_UPDATE / MANAGER_REASSIGN.
Methods	
Name	**Input / Output / Description**
createManager	Input: CreateManagerAccountDto dto Output: ManagerAccountDto Description: @Transactional. Checks unique username/email (AT1), password strength, branch exists and is active, hashes the password, saves the Employee with role BRANCH_MANAGER and branchId, logs audit.

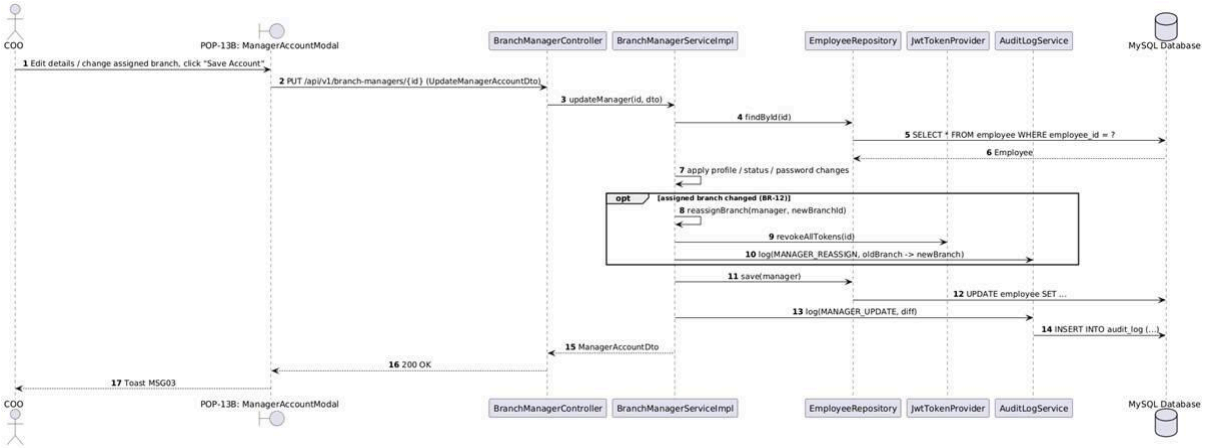
updateManager	Input: Long managerId, UpdateManagerAccountDto dto Output: ManagerAccountDto Description: Applies profile/status changes; if branchId changed calls reassignBranch(); re-hashes password when provided.
reassignBranch	Input: Employee manager, Long newBranchId Output: void Description: Moves the account to the new branch, revokes active tokens and logs MANAGER_REASSIGN with old/new branch (BR-12).

4.7.3 Object Interactions (Sequence Diagrams)

4.7.3.1 SD-01: Create Branch Manager Account (UC06)



4.7.3.2 SD-02: Update / Reassign Branch Manager (UC06)



4.8 Manage Employee Profiles

This section presents the detailed design of the Manage Employee Profiles feature on TAB-03 of SCR-09 Employee & Shifts. The Branch Manager views the staff directory of his/her own branch only (BR-04), adds employees, edits profile data (name, phone, position, hourly wage, attendance PIN, RFID card) and deactivates terminated employees. Hard delete is prohibited (BR-13); PINs are 4 digits, unique within the branch and stored hashed; an RFID card can belong to only one active employee (BR-05); a new hourly wage applies only to shifts after the change (BR-14).

4.8.1 Screen Design

4.8.1.1 Profiles & Hourly Wage Tab (SCR-09 / TAB-03)

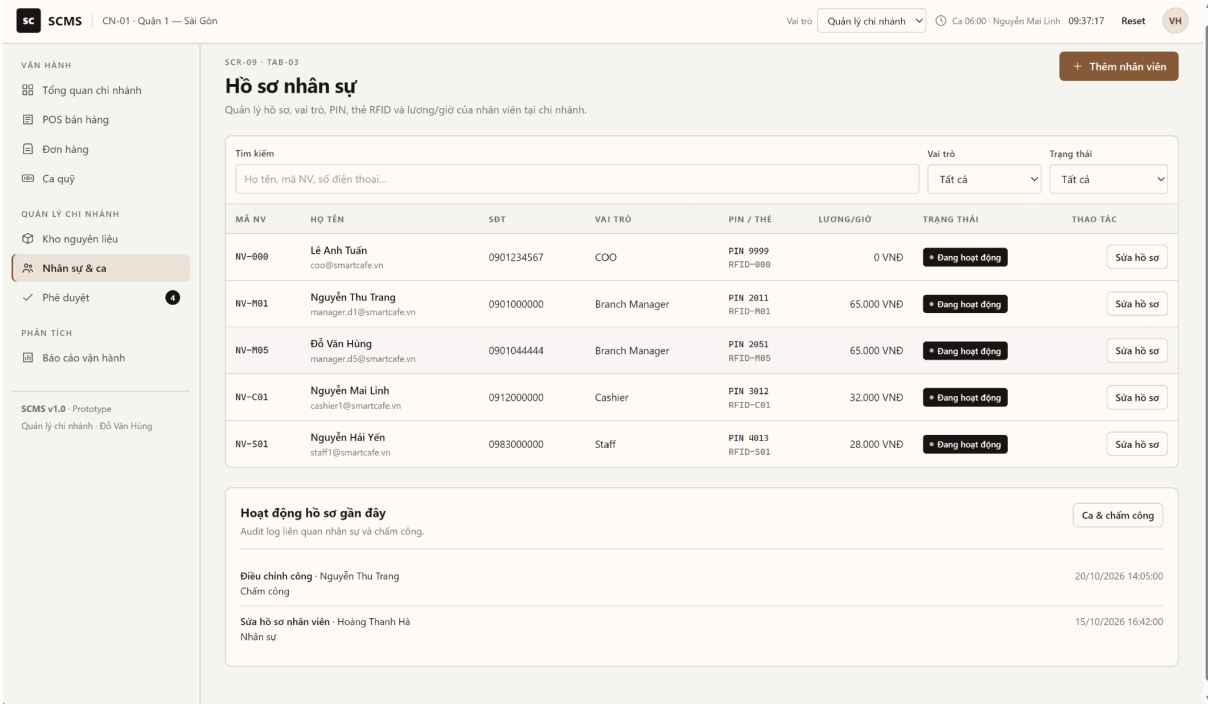


Table 4-9: Profiles & Hourly Wage Tab (SCR-09 / TAB-03) Definition

No	Object/Control Name	Type	Required	Length	Description
1	btnAddEmployee	Button	No	-	Opens POP-09A in create mode.
2	tabSwitcher	Tabs	-	-	Switches between TAB-04 (Shifts & Attendance Logs) and TAB-03 (Profiles & Hourly Wage).
3	txtSearchEmployee	Textbox	No	Max 50	Live search by employee ID, name or phone.

4	cboPositionFilter	Dropdown	No	-	Filters by position: All, Cashier, Staff (Barista), Staff (Waiter).
5	cboStatusFilter	Dropdown	No	-	Filters by employment status: Active / Inactive / All.
6	tblProfiles	Data Grid	Yes	-	Employee ID, Full Name, Phone, Position, Hourly Wage, PIN / RFID (masked), Status, Action. Data from GET /api/v1/employees (current branch).
7	btnEditProfile	Button	No	-	Opens POP-09A pre-filled with the selected employee.
8	btnDeactivate	Button	No	-	Confirmation popup, then PATCH /api/v1/employees/{id}/status (Inactive, BR-13).

4.8.1.2 Add / Edit Employee Profile Popup (POP-09A)

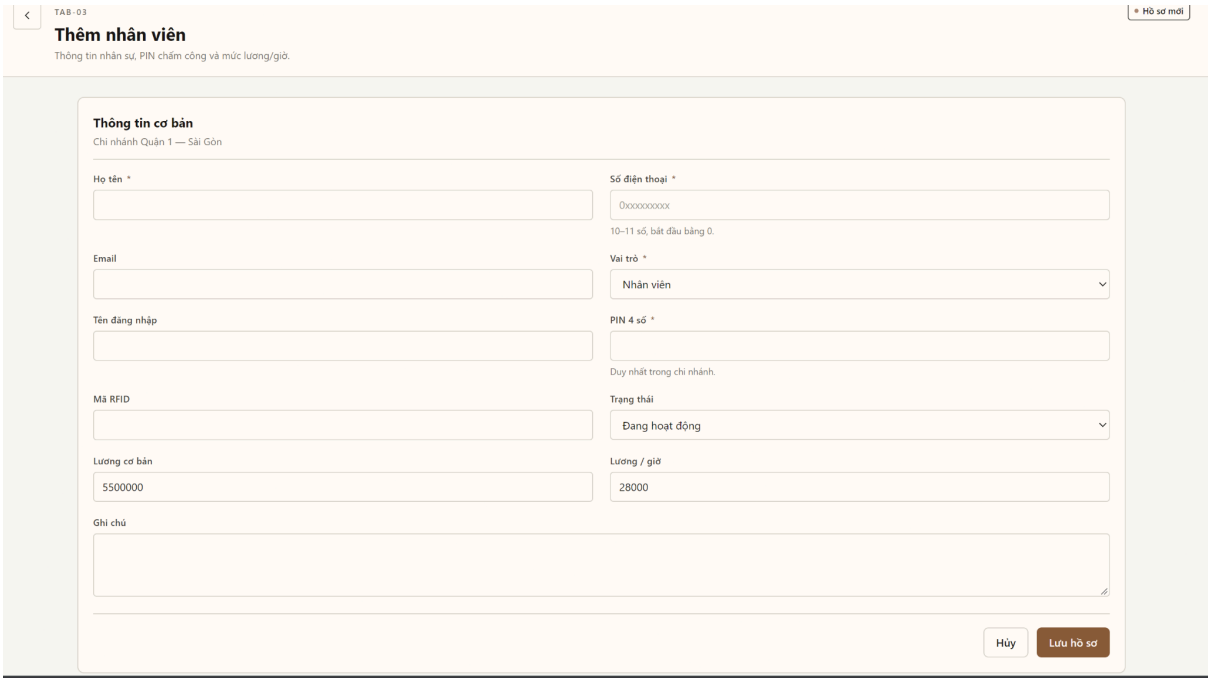


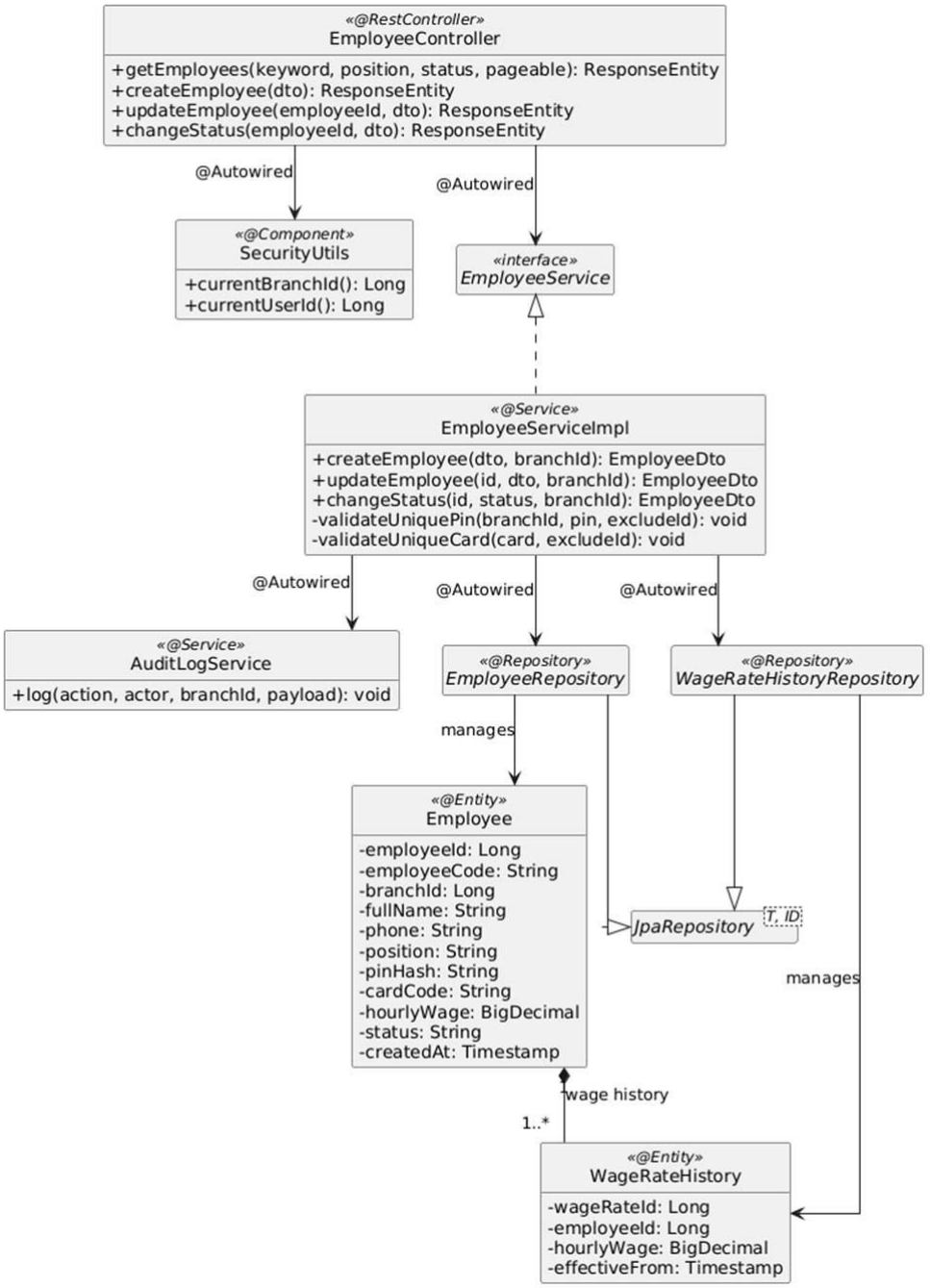
Table 4-10: Add / Edit Employee Profile Popup (POP-09A) Definition

No	Object/Control Name	Type	Required	Length	Description
1	txtFullName	Textbox	Yes	Max 100	Employee full name.
2	txtPhone	Textbox	Yes	10 - 11	Digits only, starting with 0.
3	cboPosition	Dropdown	Yes	-	Cashier / Staff; determines the role bound to the profile.
4	txtHourlyWage	Number	Yes	10	Hourly wage rate (VND, positive integer). Effective for shifts after save (BR-14).
5	txtPin	Password	Yes	4	Attendance PIN, exactly 4 digits, unique within the branch.
6	txtRfidCode	Textbox	No	Max 15	RFID card code; unique among active employees (BR-05).

7	cboStatus	Dropdown	Yes	-	Active / Inactive.
8	dtStartDate	Date Picker	No	10	Start working date (DD/MM/YYYY).
9	btnSaveEmployee	Button	Yes	-	POST /api/v1/employees or PUT /api/v1/employees/{id}; shows MSG04 / MSG03 or "PIN code already in use by another staff member."
10	btnCancel	Button	No	-	Closes without saving.

4.8.2 Class/Component Design

4.8.2.1 Classes/Component Structure



4.8.2.2 Classes/Component Description

4.8.2.2.1 EmployeeController Class

Class	EmployeeController
Description	Spring REST Controller for branch employee profile management. The branch scope is taken from the authenticated principal, never from the request (BR-04).
Base Class	None (@RestController, @RequestMapping("/api/v1/employees"))

Constructor	+EmployeeController(EmployeeService employeeService, SecurityUtils securityUtils)
Prototype	public class EmployeeController
Source File	EmployeeController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
employeeService	EmployeeService - Service interface injected via @Autowired.
securityUtils	SecurityUtils - Resolves current user and branchId from the JWT.
Methods	
Name	**Input / Output / Description**
getEmployees	Input: @RequestParam String keyword, String position, String status, Pageable pageable Output: ResponseEntity<Page<EmployeeDto>> Description: GET endpoint listing employees of the manager's branch.
createEmployee	Input: @RequestBody @Valid CreateEmployeeDto dto Output: ResponseEntity<EmployeeDto> Description: POST endpoint adding an employee (UC07 main flow).
updateEmployee	Input: @PathVariable Long employeeId, @RequestBody @Valid UpdateEmployeeDto dto Output: ResponseEntity<EmployeeDto> Description: PUT endpoint updating the profile, wage, PIN or RFID.
changeStatus	Input: @PathVariable Long employeeId, @RequestBody ChangeEmployeeStatusDto dto Output: ResponseEntity<EmployeeDto> Description: PATCH /{employeeId}/status. Deactivates / reactivates (no hard delete, BR-13).

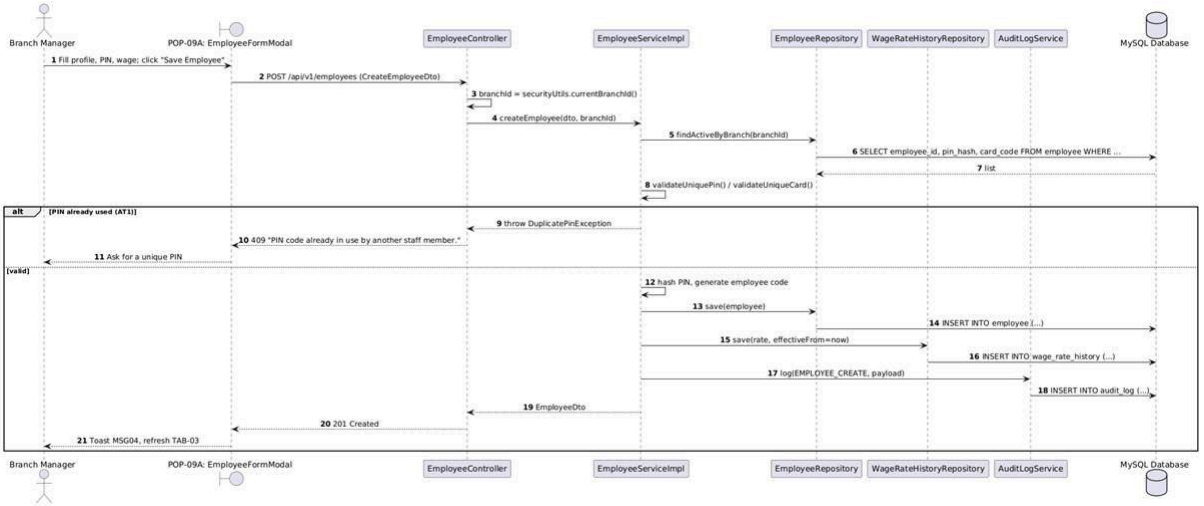
4.8.2.2.2 EmployeeServiceImpl Class

Class	EmployeeServiceImpl
Description	Service implementation validating and persisting employee profiles inside the branch scope.
Base Class	None (implements EmployeeService)
Constructor	+EmployeeServiceImpl(EmployeeRepository employeeRepo, WageRateHistoryRepository wageHistoryRepo, PasswordEncoder pinEncoder, AuditLogService auditLogService)
Prototype	public class EmployeeServiceImpl implements EmployeeService
Source File	EmployeeServiceImpl.java
Namespace	com.scms.service.impl

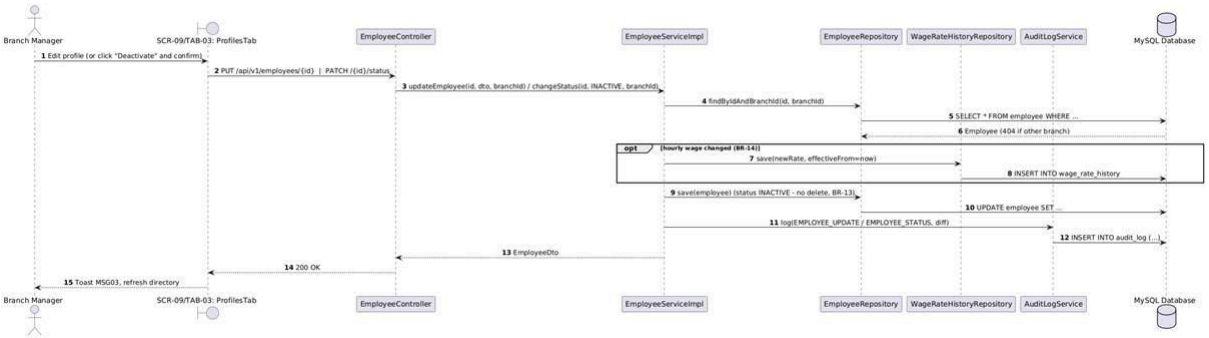
Attributes	
Name	**Type & Description**
employeeRepo	EmployeeRepository - Persists employee profiles.
wageHistoryRepo	WageRateHistoryRepository - Stores hourly wage versions with effectiveFrom (BR-14).
pinEncoder	PasswordEncoder - Hashes attendance PINs.
auditLogService	AuditLogService - Logs EMPLOYEE_CREATE / EMPLOYEE_UPDATE / EMPLOYEE_STATUS.
Methods	
Name	**Input / Output / Description**
createEmployee	Input: CreateEmployeeDto dto, Long branchId Output: EmployeeDto Description: @Transactional. Calls validateUniquePin() (AT1) and validateUniqueCard(), hashes PIN, generates employee code, saves the profile and the first wage record, logs audit.
updateEmployee	Input: Long employeeId, UpdateEmployeeDto dto, Long branchId Output: EmployeeDto Description: Ensures the employee belongs to branchId, re-validates PIN/RFID when changed, inserts a new wage history row if the rate changed, saves and logs diff.
changeStatus	Input: Long employeeId, EmployeeStatus status, Long branchId Output: EmployeeDto Description: Switches status to INACTIVE/ACTIVE; clears RFID binding on deactivation.
validateUniquePin	Input: Long branchId, String pin, Long excludeId Output: void Description: Matches the raw PIN against the hashes of active employees of the branch; throws DuplicatePinException.

4.8.3 Object Interactions (Sequence Diagrams)

4.8.3.1 SD-01: Add New Employee Profile (UC07)



4.8.3.2 SD-02: Update / Deactivate Employee Profile (UC07)



4.9 Manage Work Shifts

This section presents the detailed design of the Manage Work Shifts feature on TAB-04 of SCR-09. The Branch Manager configures shift templates (Morning, Afternoon, Evening or custom time frames), picks a date range and assigns active employees. Before saving, the system checks every assignment for overlapping shifts of the same employee on the same day (BR-15); conflicts are returned to the screen and highlighted (AT1). The saved assignments become the roster used by the Kiosk to validate check-in (UC03).

4.9.1 Screen Design

4.9.1.1 Shifts & Attendance Logs Tab (SCR-09 / TAB-04)

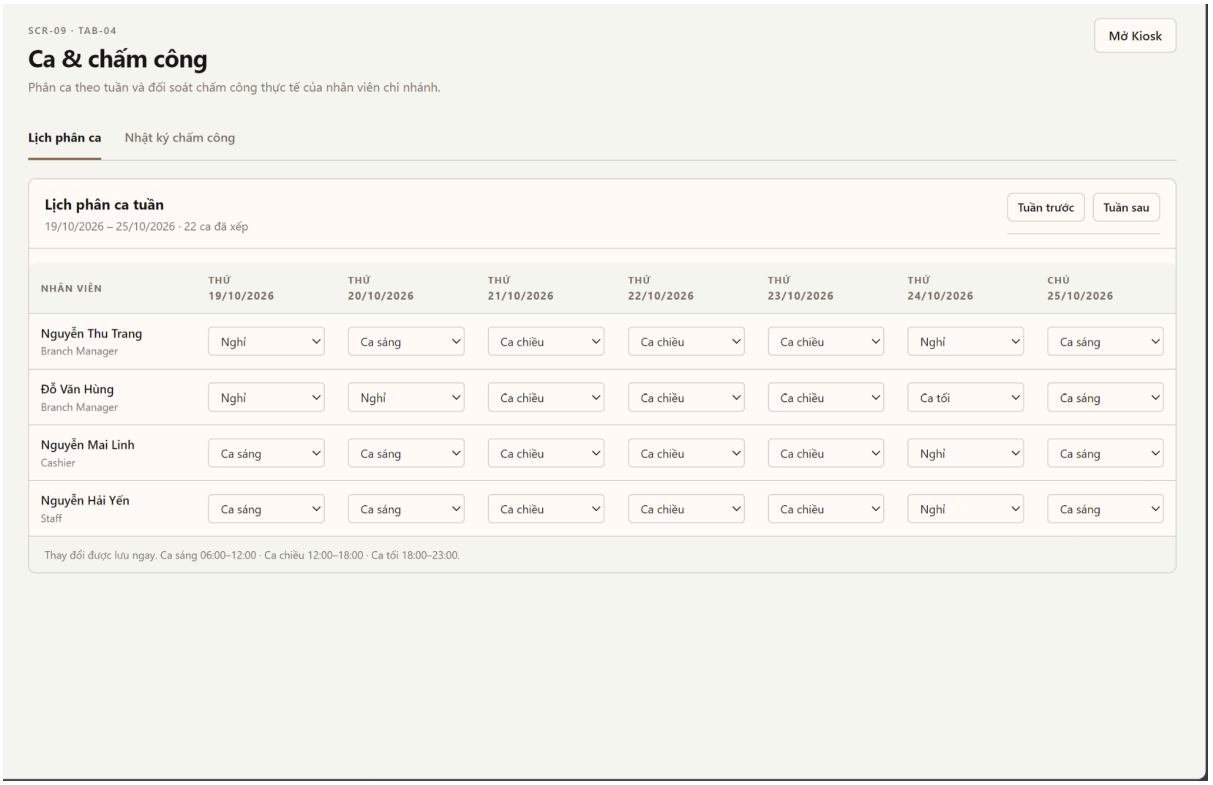


Table 4-11: Shifts & Attendance Logs Tab (SCR-09 / TAB-04) Definition

No	Object/Control Name	Type	Required	Length	Description
1	btnAddShift	Button	No	-	Opens POP-09B to create a shift schedule.
2	tabSwitcher	Tabs	-	-	Switches between TAB-04 and TAB-03.

3	tblWeeklySchedule	Data Grid	Yes	-	Roster matrix (employee x Monday..Sunday) from GET /api/v1/work-schedules?weekStart=. Clicking a cell opens POP-09B for that day.
4	cboWeekPicker	Date Picker	No	-	Selects the week to display (previous / next week).
5	tblKioskLogs	Data Grid	-	-	Today's check-in/check-out logs from SCR-02 (read-only).
6	btnAdjustAttendance	Button	No	-	Opens the attendance adjustment dialog (BR-10).

4.9.1.2 Create / Edit Work Shift Schedule Popup (POP-09B)

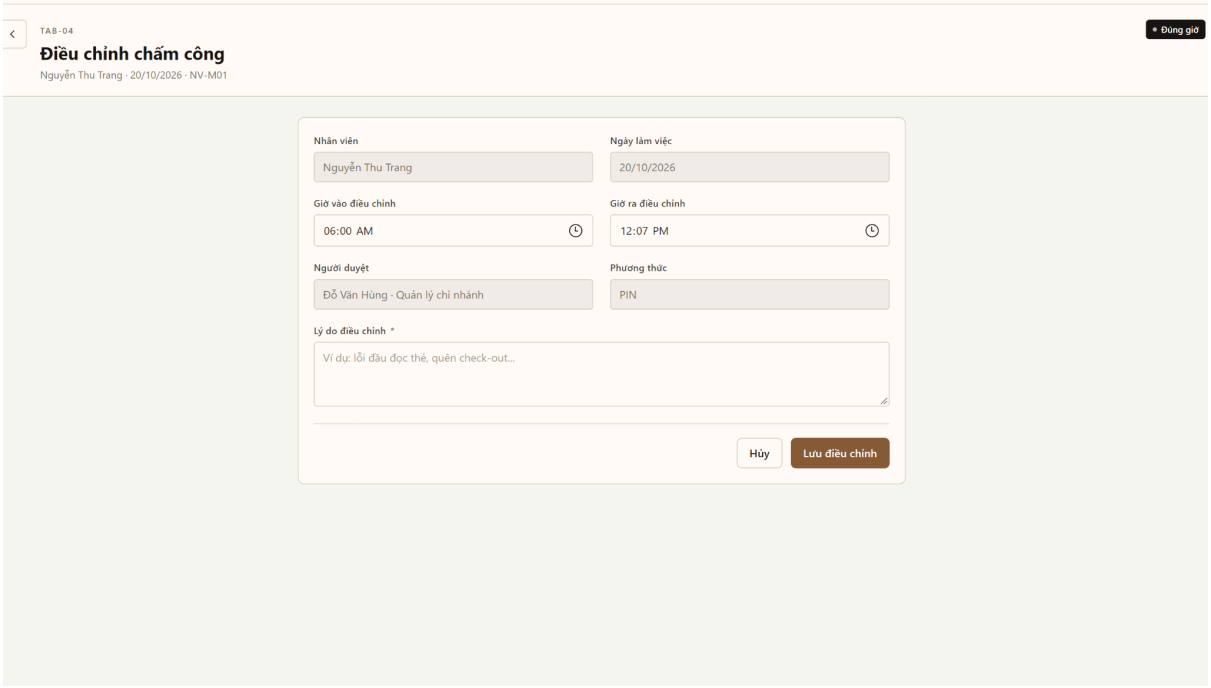


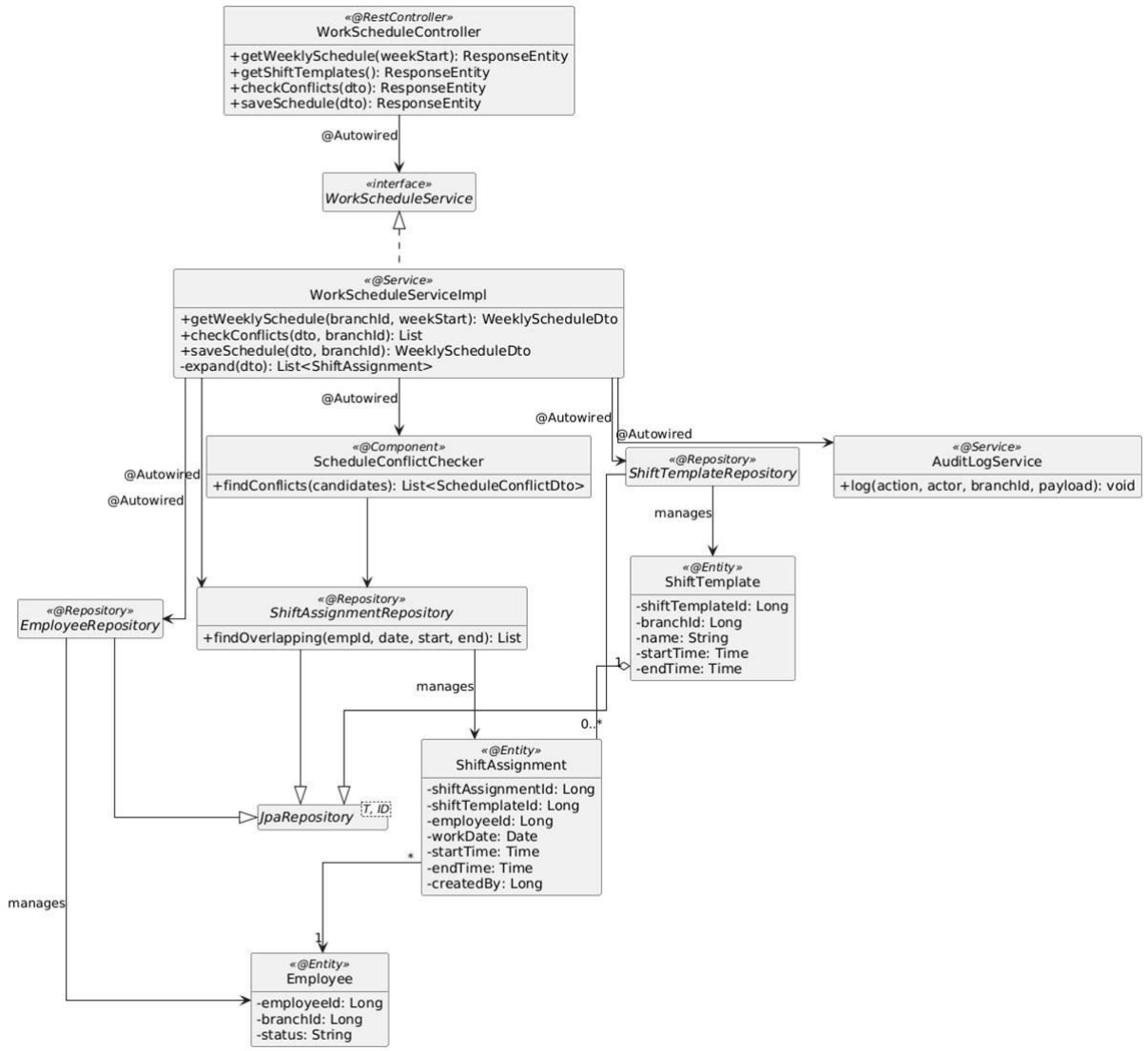
Table 4-12: Create / Edit Work Shift Schedule Popup (POP-09B) Definition

No	Object/Control Name	Type	Required	Length	Description

1	cboShiftTemplate	Dropdown	Yes	-	Morning / Afternoon / Evening / Custom; pre-fills start and end time.
2	tpStartTime / tpEndTime	Time Picker	Yes	5	Shift time frame within branch opening hours; end > start (overnight not allowed).
3	dtApplyFrom / dtApplyTo	Date Picker	Yes	10	Date range of the schedule (from <= to, not in the past).
4	chkRepeatDays	Checkbox Group	No	-	Weekdays on which the shift is generated.
5	tblAssignEmployees	Data Grid	Yes	-	Active employees of the branch with a Select checkbox and an availability column filled by POST /api/v1/work-schedules /check-conflicts.
6	pnlConflictWarning	Alert Box	-	-	Lists conflicting employees and shift times (AT1, BR-15).
7	btnSaveSchedule	Button	Yes	-	POST /api/v1/work-schedules ; on success shows MSG04 and refreshes the roster.
8	btnCancel	Button	No	-	Closes without saving.

4.9.2 Class/Component Design

4.9.2.1 Classes/Component Structure



4.9.2.2 Classes/Component Description

4.9.2.2.1 WorkScheduleController Class

Class	WorkScheduleController
Description	Spring REST Controller for shift templates and employee shift assignments of a branch.
Base Class	None (@RestController, @RequestMapping("/api/v1/work-schedules"))
Constructor	+WorkScheduleController(WorkScheduleService scheduleService, SecurityUtils securityUtils)
Prototype	public class WorkScheduleController
Source File	WorkScheduleController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**

scheduleService	WorkScheduleService - Service interface injected via @Autowired.
securityUtils	SecurityUtils - Resolves branchId of the current manager.
Methods	
Name	**Input / Output / Description**
getWeeklySchedule	Input: @RequestParam LocalDate weekStart Output: ResponseEntity<WeeklyScheduleDto> Description: GET endpoint returning the roster of the manager's branch for a week.
getShiftTemplates	Input: None Output: ResponseEntity<List<ShiftTemplateDto>> Description: GET /templates. Shift templates of the branch.
checkConflicts	Input: @RequestBody @Valid SaveScheduleDto dto Output: ResponseEntity<List<ScheduleConflictDto>> Description: POST /check-conflicts. Dry-run conflict detection used to fill the availability column.
saveSchedule	Input: @RequestBody @Valid SaveScheduleDto dto Output: ResponseEntity<WeeklyScheduleDto> Description: POST endpoint saving shift assignments (UC08 main flow).

4.9.2.2.2 WorkScheduleServiceImpl Class

Class	WorkScheduleServiceImpl
Description	Service implementation expanding a schedule request into daily assignments, detecting conflicts and persisting the roster.
Base Class	None (implements WorkScheduleService)
Constructor	+WorkScheduleServiceImpl(ShiftTemplateRepository templateRepo, ShiftAssignmentRepository assignmentRepo, EmployeeRepository employeeRepo, ScheduleConflictChecker conflictChecker, AuditLogService auditLogService)
Prototype	public class WorkScheduleServiceImpl implements WorkScheduleService
Source File	WorkScheduleServiceImpl.java
Namespace	com.scms.service.impl
Attributes	
Name	**Type & Description**
templateRepo	ShiftTemplateRepository - Reads shift templates.
assignmentRepo	ShiftAssignmentRepository - Persists and queries shift assignments.
employeeRepo	EmployeeRepository - Validates that employees are active and belong to the branch.
conflictChecker	ScheduleConflictChecker - Detects overlapping assignments (BR-15).
auditLogService	AuditLogService - Logs SCHEDULE_SAVE.

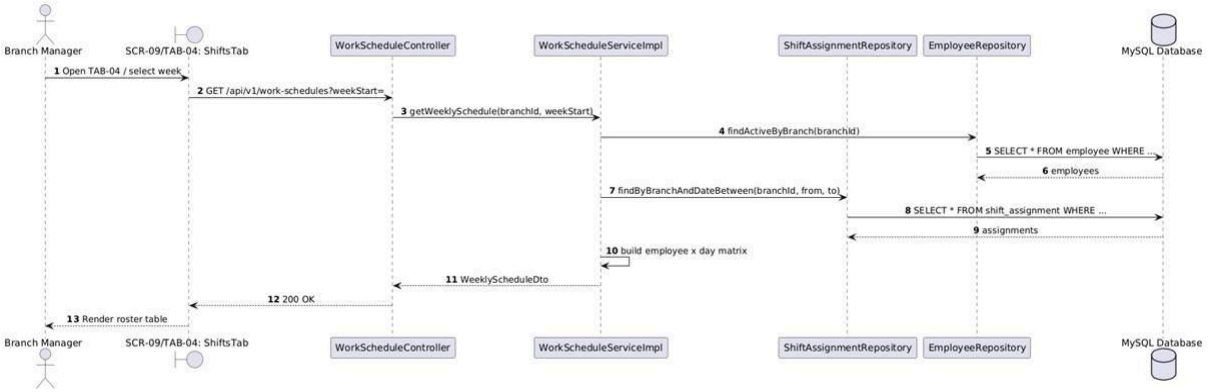
Methods	
Name	**Input / Output / Description**
saveSchedule	Input: SaveScheduleDto dto, Long branchId Output: WeeklyScheduleDto Description: @Transactional. Validates time frame and date range, expands days x employees into ShiftAssignment candidates, calls conflictChecker.findConflicts(); throws ScheduleConflictException with the conflict list (AT1) or saveAll() and logs audit.
checkConflicts	Input: SaveScheduleDto dto, Long branchId Output: List<ScheduleConflictDto> Description: Same expansion without persisting; returns conflicts for display.
getWeeklySchedule	Input: Long branchId, LocalDate weekStart Output: WeeklyScheduleDto Description: Builds the employee x day matrix for the week.

4.9.2.2.3 ScheduleConflictChecker Class

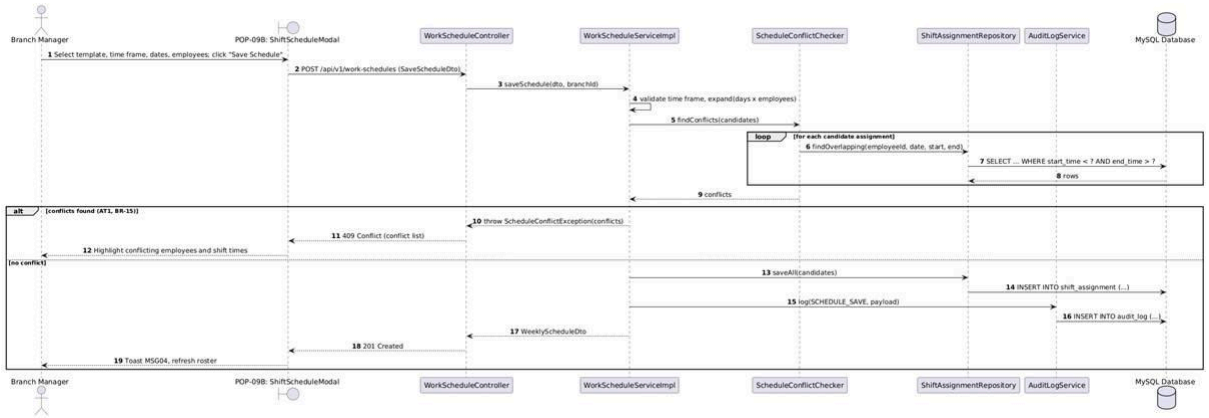
Class	ScheduleConflictChecker
Description	Component that checks overlapping work shifts.
Base Class	None (@Component)
Constructor	+ScheduleConflictChecker(ShiftAssignmentRepository assignmentRepo)
Prototype	public class ScheduleConflictChecker
Source File	ScheduleConflictChecker.java
Namespace	com.scms.service.support
Attributes	
Name	**Type & Description**
assignmentRepo	ShiftAssignmentRepository - Queries existing assignments.
Methods	
Name	**Input / Output / Description**
findConflicts	Input: List<ShiftAssignment> candidates Output: List<ScheduleConflictDto> Description: For each candidate queries existing assignments of the same employee on the same date where start < candidate.end AND end > candidate.start; also detects duplicates inside the request.

4.9.3 Object Interactions (Sequence Diagrams)

4.9.3.1 SD-01: View Weekly Shift Schedule (UC08)



4.9.3.2 SD-02: Create Shift Schedule & Assign Employees (UC08)



4.10 Manage Menu Categories

This section presents the detailed design of the Manage Menu Categories feature on SCR-11 Master Menu & Recipe. The COO centrally manages the master menu structure (BR-16): creates categories, edits name, description, display sequence and status, assigns and reorders products inside a category, and deactivates categories. A category that still contains active products cannot be deleted or deactivated (AT1). Changes are stored in the master database and reach branches only after being published via the Chain Sync Center (UC14, BR-23).

4.10.1 Screen Design

4.10.1.1 Menu Categories Screen (SCR-11)

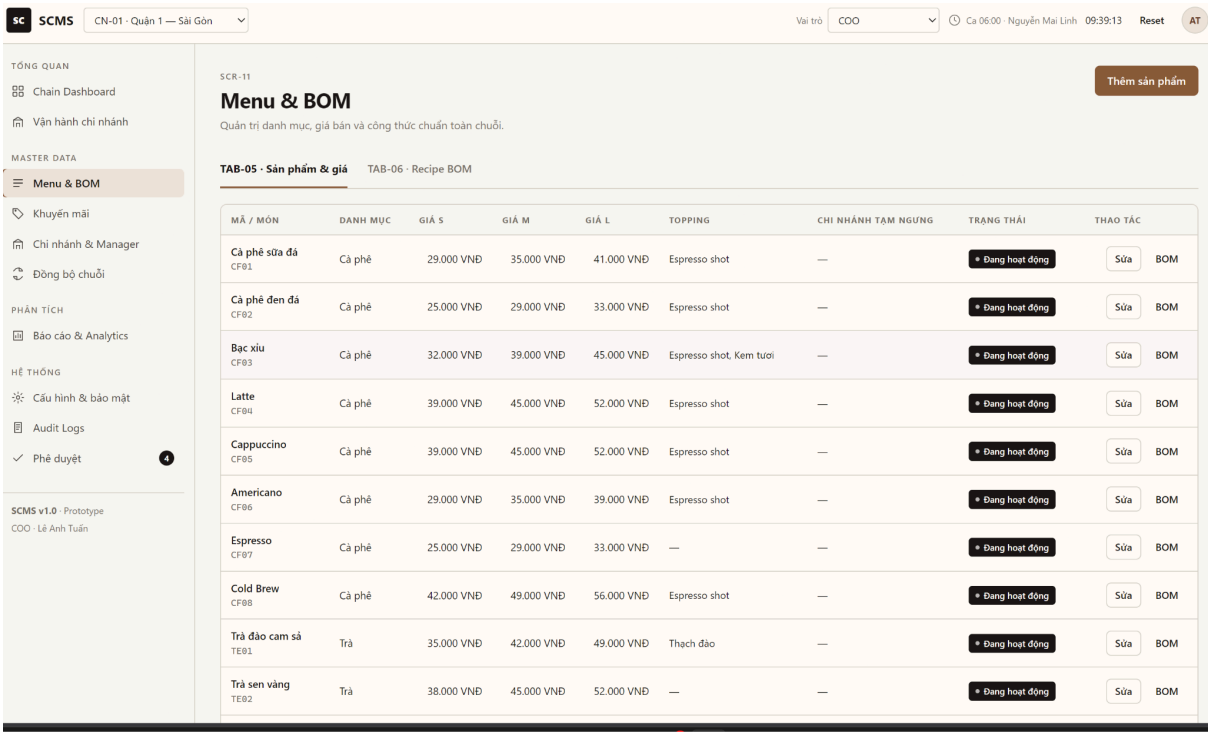


Table 4-13: Menu Categories Screen (SCR-11) Definition

No	Object/Control Name	Type	Required	Length	Description
1	btnCreateCategory	Button	No	-	Opens POP-11A in create mode.
2	tabSwitcher	Tabs	-	-	Menu Categories / TAB-05 Products & Pricing by Size / TAB-06 Standard Recipe BOM.
3	txtSearchCategory	Textbox	No	Max 50	Live search by category code or name.
4	cboStatusFilter	Dropdown	No	-	All / Active / Inactive.

5	tblCategories	Data Grid	Yes	-	Display Sequence, Code, Name, Description, number of products, Status, Action; drag-and-drop reorder calls PUT /api/v1/menu-categories/sequence.
6	btnEditCategory	Button	No	-	Opens POP-11A pre-filled with the category and its products.
7	btnDeactivateCategory	Button	No	-	Confirmation popup then PATCH /api/v1/menu-categories/{id}/status.

4.10.1.2 Create / Edit Menu Category Popup (POP-11A)

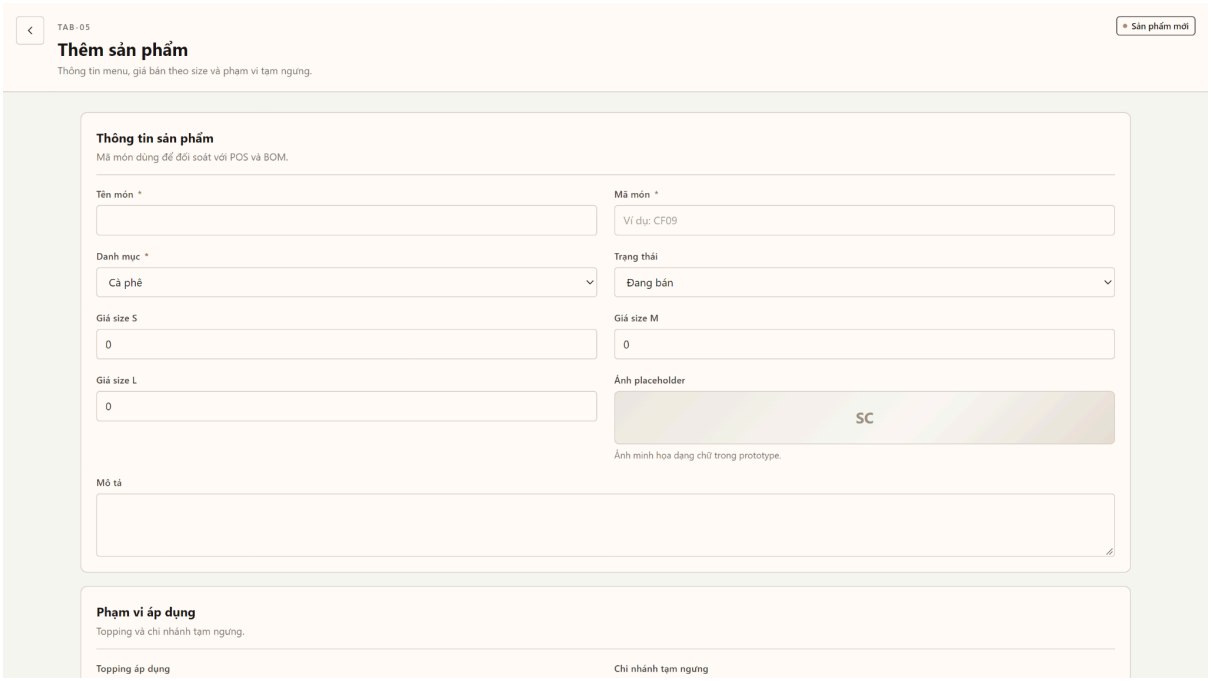


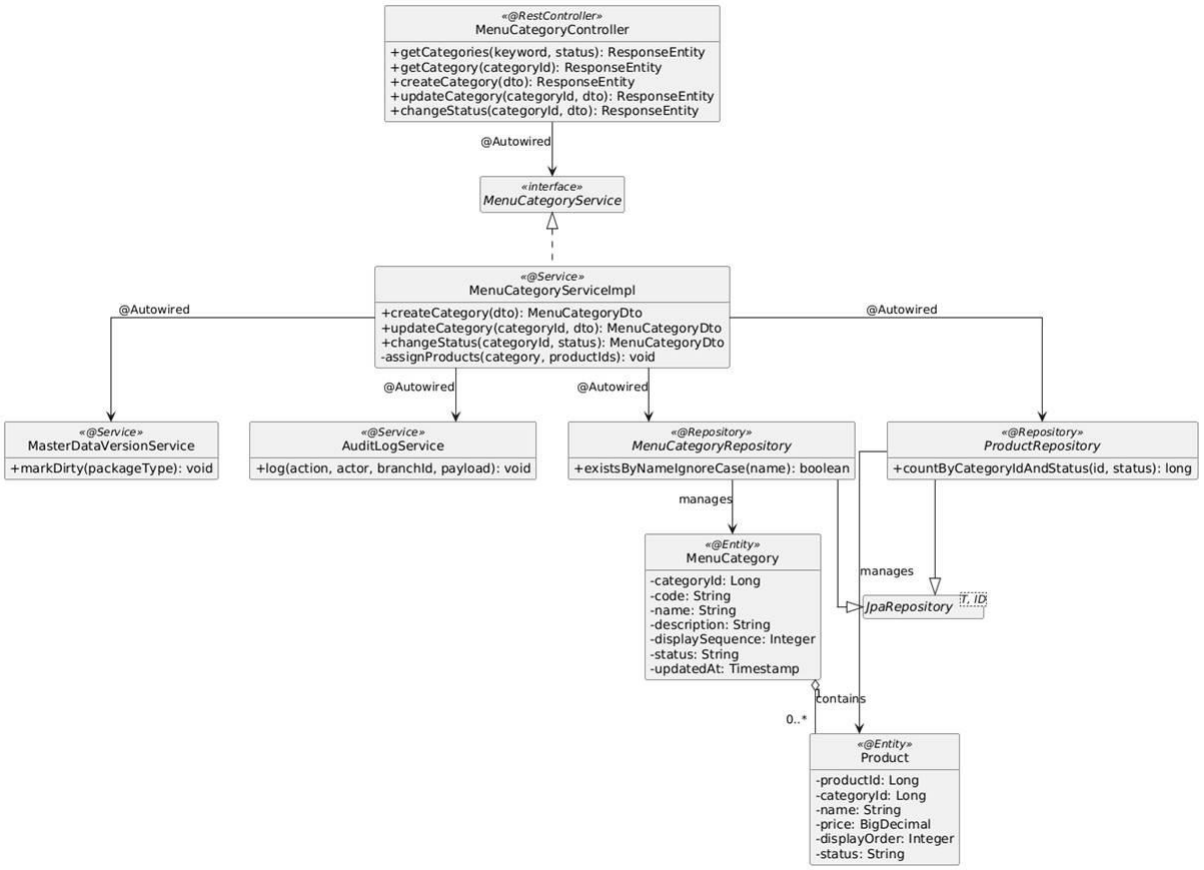
Table 4-14: Create / Edit Menu Category Popup (POP-11A) Definition

No	Object/Control Name	Type	Required	Length	Description
1	txtCategoryCode	Textbox	No	10	Auto-generated code (CATnn), read-only.

2	txtCategoryName	Textbox	Yes	Max 100	Unique category name.
3	txtDisplaySequence	Number	Yes	3	Positive integer ordering of the category on POS.
4	cboActiveStatus	Dropdown	Yes	-	Active / Inactive.
5	txtDescription	Textarea	No	Max 255	Category description.
6	tblAssignedProducts	Data Grid	No	-	Products in the category with drag handle for order; uncheck to remove.
7	cboAddProduct / btnAddProduct	Dropdown / Button	No	-	Searches products (GET /api/v1/products?keyword=) and adds them to the category.
8	btnSaveCategory	Button	Yes	-	POST /api/v1/menu-categories or PUT /api/v1/menu-categories/{id}; shows MSG04 / MSG03.
9	btnCancel	Button	No	-	Closes without saving.

4.10.2 Class/Component Design

4.10.2.1 Classes/Component Structure



4.10.2.2 Classes/Component Description

4.10.2.2.1 MenuCategoryController Class

Class	MenuCategoryController
Description	Spring REST Controller for master menu category management (COO only, BR-16).
Base Class	None (@RestController, @RequestMapping("/api/v1/menu-categories"))
Constructor	+MenuCategoryController(MenuCategoryService categoryService)
Prototype	public class MenuCategoryController
Source File	MenuCategoryController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
categoryService	MenuCategoryService - Service interface injected via @Autowired.
Methods	
Name	**Input / Output / Description**

getCategories	Input: @RequestParam String keyword, String status Output: ResponseEntity<List<MenuCategoryDto>> Description: GET endpoint listing categories ordered by display sequence with product count.
getCategory	Input: @PathVariable Long categoryId Output: ResponseEntity<MenuCategoryDetailDto> Description: GET /{categoryId}. Category with its ordered products.
createCategory	Input: @RequestBody @Valid SaveMenuCategoryDto dto Output: ResponseEntity<MenuCategoryDto> Description: POST endpoint creating a category and product mapping (UC09 main flow).
updateCategory	Input: @PathVariable Long categoryId, @RequestBody @Valid SaveMenuCategoryDto dto Output: ResponseEntity<MenuCategoryDto> Description: PUT endpoint updating details and product assignment/order.
changeStatus	Input: @PathVariable Long categoryId, @RequestBody ChangeStatusDto dto Output: ResponseEntity<MenuCategoryDto> Description: PATCH /{categoryId}/status. Activates / deactivates (AT1 check).

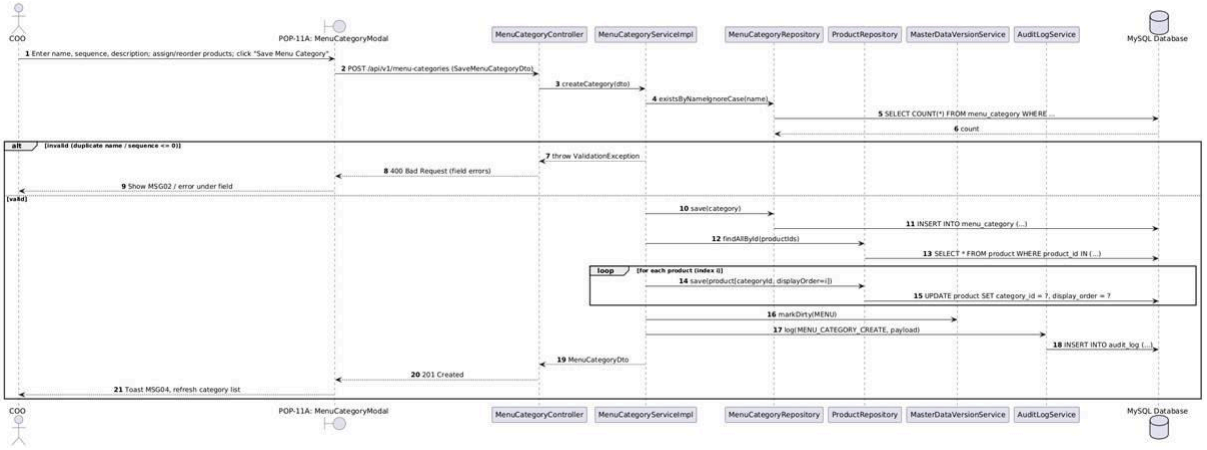
4.10.2.2.2 MenuCategoryServiceImpl Class

Class	MenuCategoryServiceImpl
Description	Service implementation validating the menu structure and persisting categories and category-product mappings.
Base Class	None (implements MenuCategoryService)
Constructor	+MenuCategoryServiceImpl(MenuCategoryRepository categoryRepo, ProductRepository productRepo, AuditLogService auditLogService, MasterDataVersionService syncVersionService)
Prototype	public class MenuCategoryServiceImpl implements MenuCategoryService
Source File	MenuCategoryServiceImpl.java
Namespace	com.scms.service.impl
Attributes	
Name	**Type & Description**
categoryRepo	MenuCategoryRepository - Persists categories.
productRepo	ProductRepository - Loads products and updates their category / display order.
auditLogService	AuditLogService - Logs MENU_CATEGORY_CREATE / UPDATE / STATUS.
syncVersionService	MasterDataVersionService - Marks master menu data as changed (pending publish in UC14).

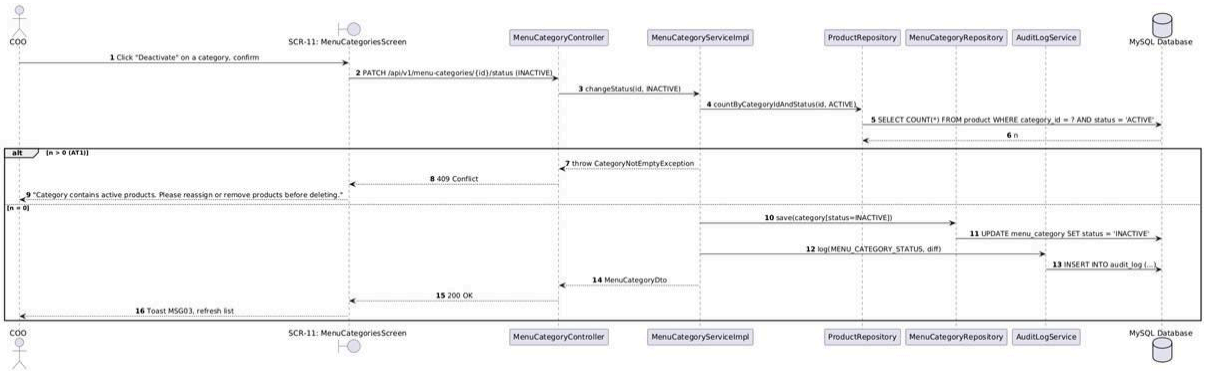
Methods	
Name	**Input / Output / Description**
createCategory	Input: SaveMenuCategoryDto dto Output: MenuCategoryDto Description: @Transactional. Validates unique name and positive sequence, generates code, saves the category, assigns products in the given order, marks master data dirty, logs audit.
updateCategory	Input: Long categoryId, SaveMenuCategoryDto dto Output: MenuCategoryDto Description: Updates details, re-assigns products (removed products move to 'Uncategorized'), reorders, logs diff.
changeStatus	Input: Long categoryId, String status Output: MenuCategoryDto Description: Before deactivation counts active products (countByCategoryIdAndStatus); if > 0 throws CategoryNotEmptyException (AT1), else updates status.

4.10.3 Object Interactions (Sequence Diagrams)

4.10.3.1 SD-01: Create / Update Menu Category (UC09)



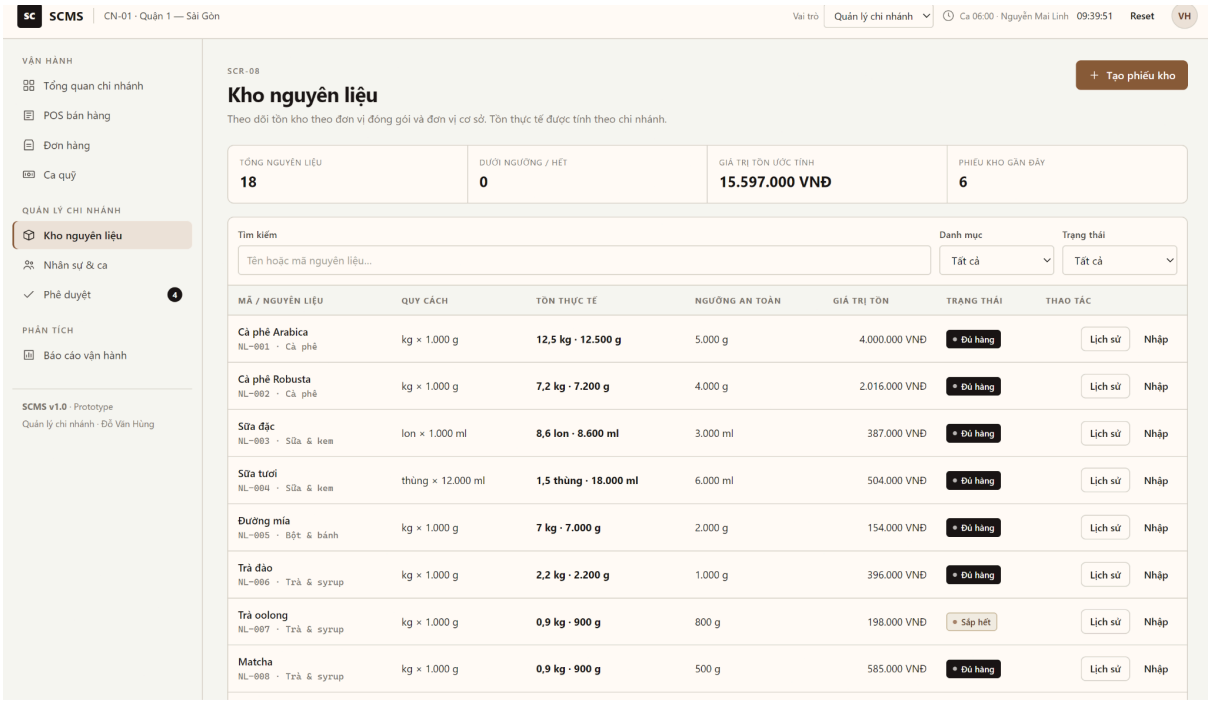
4.10.3.2 SD-02: Deactivate Menu Category (UC09)



4.11 Raw Material Inventory Management Subsystem

4.11.1 Screen Design

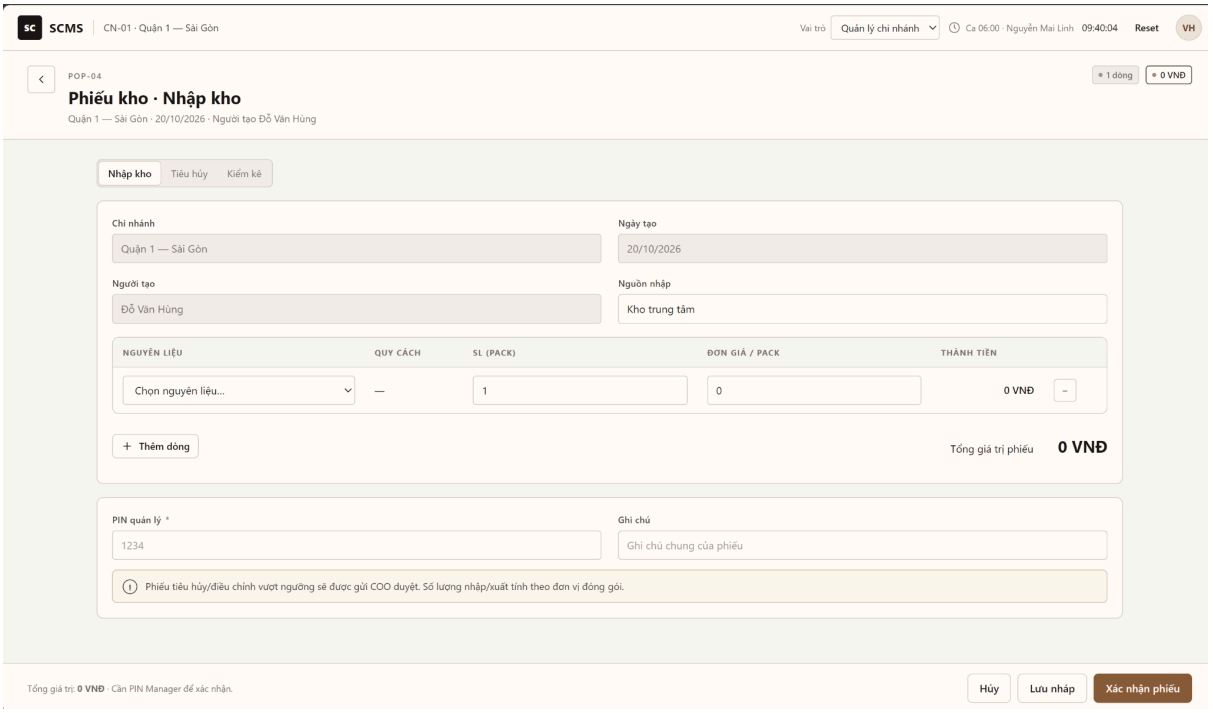
4.11.1.1 Branch Inventory Management Screen



No	Object/Control Name	Type	Required	Length	Description
1	pnlLowStockAlert	Alert Box	Dynamic	-	Prominent visual container automatically rendered when current on-hand stock is at or below minimum safety stock threshold (BR-40, UC33).
2	btnQuickRestock	Button	No	-	Quick action button navigating directly to POP-04A with nearly depleted items pre-selected for supplier restocking.
3	txtSearchMaterial	Textbox	No	Max 50	Real-time search input filtering inventory records by ingredient code or material name.
4	cboCategoryFilter	Dropdown	No	-	Filters catalog items by category: All Categories, Coffee Beans, Dairy & Milk, Syrups & Flavoring, Packaging.

5	cboStockStatusFilter	Dropdown	No	-	Filters materials by condition: All Conditions, Low Stock (Below Minimum), Sufficient Stock.
6	tblInventoryList	Data Grid	Yes	-	Master inventory ledger displaying code, name, packaging specification, base unit, minimum safety stock, on-hand balance, holding value, and status badges (BR-40).
7	btnOpenStockSlip	Button	No	-	Primary header button opening the multi-purpose stock voucher creation menu (Receive, Issue, Dispose, Stocktake).
8	btnExportStockCard	Button	No	-	Exports store inventory ledger and lot transaction history to a formatted Excel spreadsheet.
9	tblRecentStockSlips	Data Grid	Yes	-	Audit table listing the 4 most recent stock vouchers (Slip Code, Type, Timestamp, Authorizer, Destination, Amount, Status) for operational tracking.
10	btnNavBranchDash	Button	No	-	Sidebar shortcut returning to the primary Branch Dashboard screen (SCR-07).

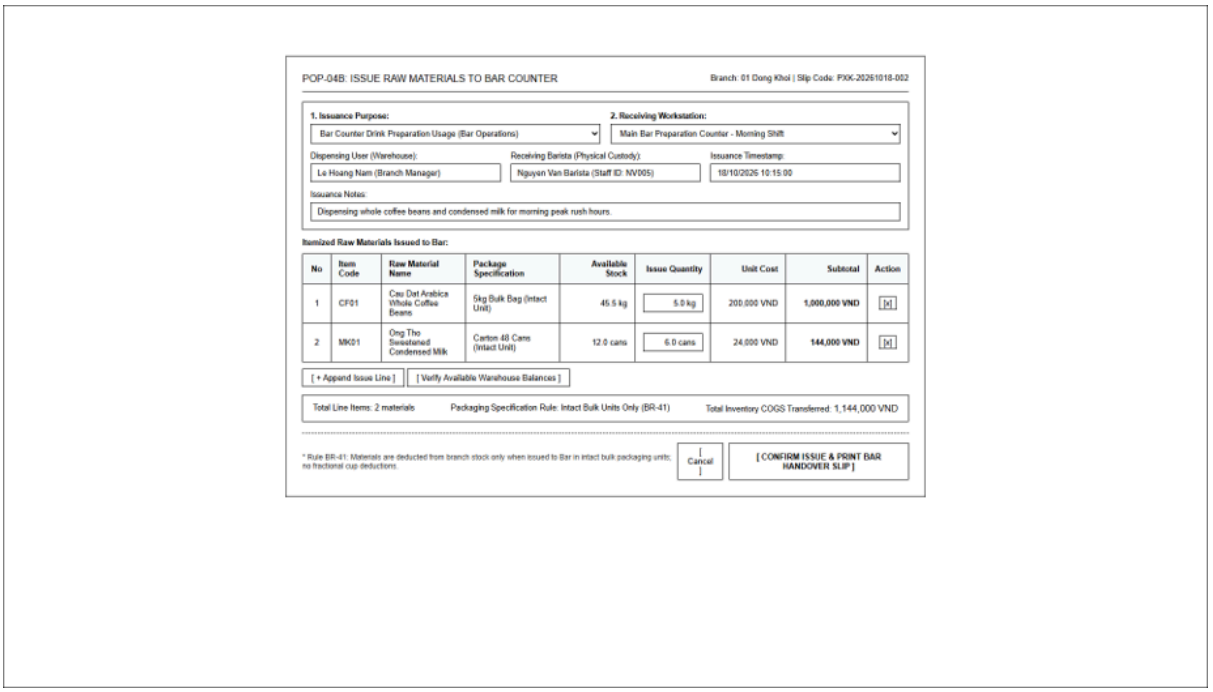
4.2.1.2 Receive Stock Delivery Slip



No	Object/Control Name	Type	Required	Length	Description
1	cboReceiveSource	Dropdown	Yes	-	Selects inbound origin: Supplier Delivery, Internal Commissary Transfer, Post-Stocktake Surplus Adjustment.
2	cboSupplier	Dropdown	Yes	-	Selects certified vendor: Cau Dat Coffee Co., Dalat Milk JSC, Monin Vietnam Syrups Co., Vinamilk.
3	txtSlipNumber	Textbox	Yes	20	Auto-generated inbound transaction code (e.g., PNK-20261018-001), read-only.
4	txtInvoiceRef	Textbox	No	Max 30	Vendor delivery note / purchase order invoice reference number.
5	txtReceiverName	Textbox	Yes	Max 50	Logged-in Branch Manager inspecting and accepting delivery custody, read-only.

6	txtReceiptDate	Textbox	Yes	19	Receipt timestamp, defaulted to current system datetime.
7	tblImportItems	Data Grid	Yes	-	Inbound item grid: No, Item Code, Raw Material Name, Packaging Specification, Inbound Qty, Unit Cost, Subtotal, Expiry Date, Delete Action.
8	btnAddItemRow	Button	No	-	Appends an empty row allowing addition of another material line.
9	lblTotalSlipValue	Label	Yes	-	Computed sum of all line item costs including 10% VAT payable to vendor.
10	btnConfirmReceive	Button	Yes	-	Validates bulk packaging units, creates stock transaction, increments branch inventory (BR-40), and records audit log (BR-51).
11	btnCancel	Button	No	-	Closes modal and aborts receipt without altering inventory records.

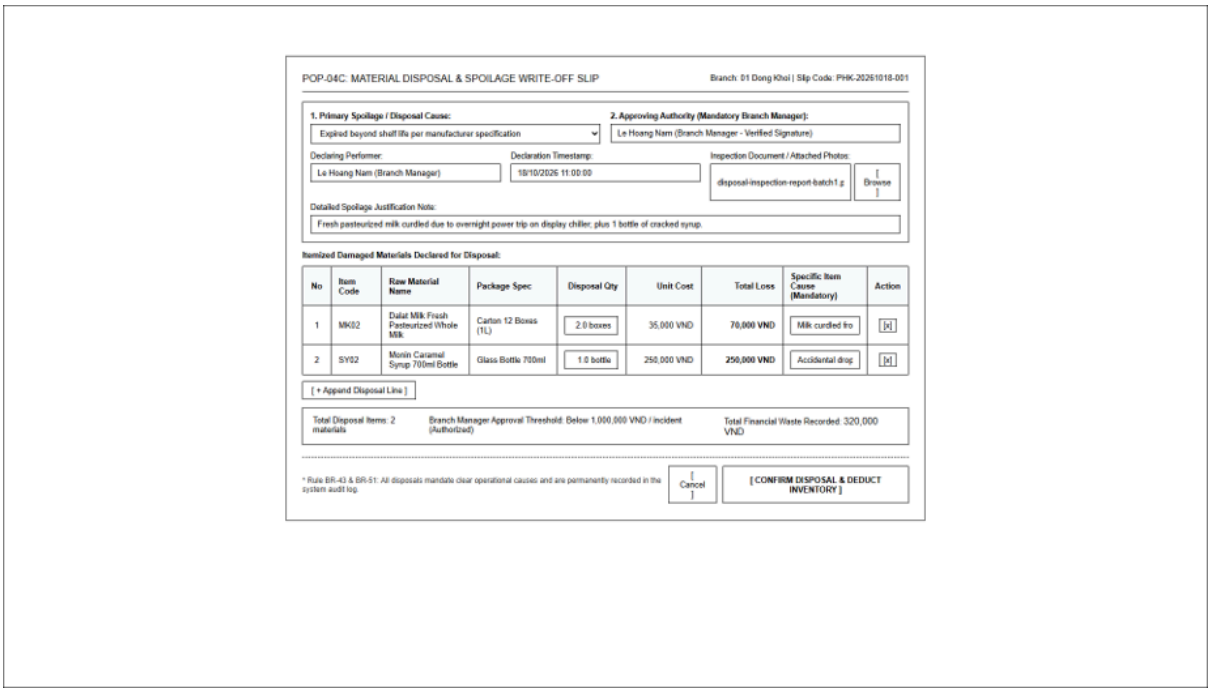
4.2.1.3 Issue Stock to Bar Counter Slip



No	Object/Control Name	Type	Required	Length	Description
1	cboIssuePurpose	Dropdown	Yes	-	Operational purpose: Barista Counter Drink Preparation, Sister Branch Transfer, Staff Training / R&D.
2	cboReceivingBar	Dropdown	Yes	-	Destination station: Main Bar Preparation Counter - Morning Shift, Main Bar - Afternoon Shift, POS Front Counter.
3	txtSlipNumber	Textbox	Yes	20	Auto-generated issue slip code (e.g., PXK-20261018-002), read-only.
4	txtBaristaSign	Textbox	Yes	Max 50	Barista on shift accepting physical custody of raw materials.
5	txtIssuerName	Textbox	Yes	Max 50	Branch Manager dispensing materials from warehouse, read-only.
6	tblIssueItems	Data Grid	Yes	-	Line items: No, Item Code, Name, Intact Package Spec, Available Stock, Issue

					Quantity, Unit Cost, Subtotal, Delete Action.
7	btnAddIssueItem	Button	No	-	Appends an empty row to requisition line items.
8	btnConfirmIssue	Button	Yes	-	Validates issueQty <= onHand (BR-42) and intact packaging unit compliance (BR-41), deducts warehouse stock, and prints bar transfer voucher.
9	btnCancel	Button	No	-	Closes modal without dispensing materials.

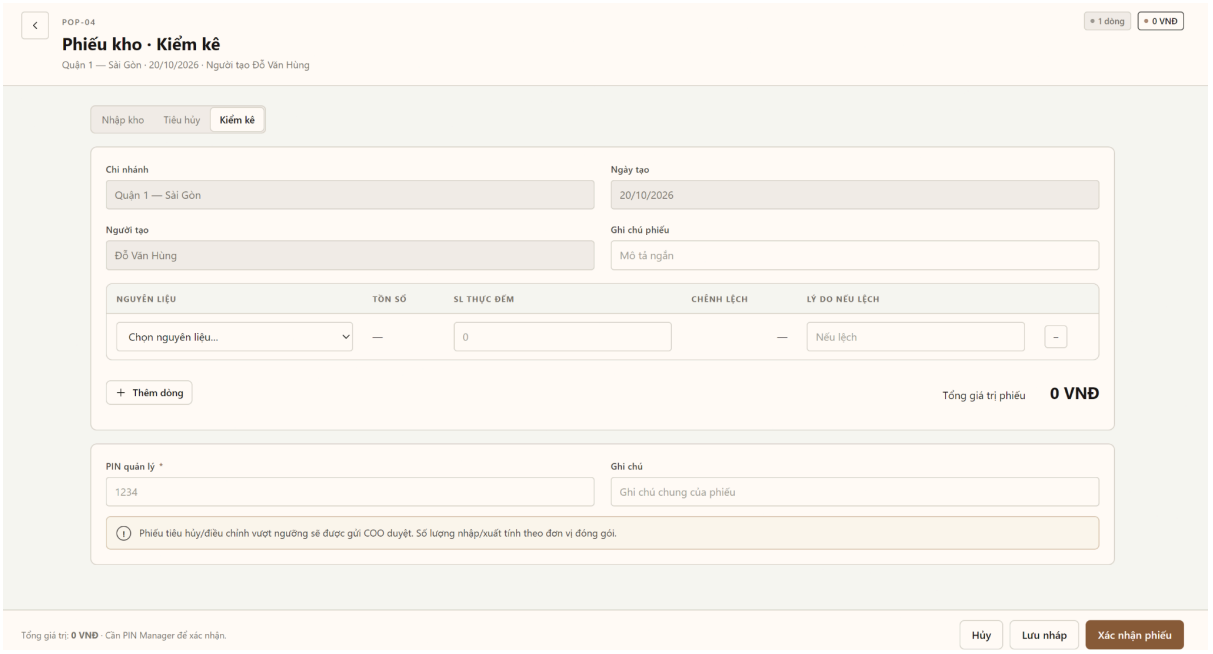
4.2.1.4 Raw Material Disposal & Write-Off Slip



No	Object/Control Name	Type	Required	Length	Description
1	cboDisposalReason	Dropdown	Yes	-	Predefined disposal classification: Expired beyond shelf life, Packaging leakage / transit damage, Refrigerator cooling failure, Ingredient degradation (BR-43).
2	txtSlipNumber	Textbox	Yes	20	Auto-generated disposal code (e.g., PHK-20261018-001), read-only.
3	txtManagerApproval	Textbox	Yes	Max 50	Authorizing Branch Manager verified signature, read-only.
4	txtCreatedDate	Textbox	Yes	19	Timestamp of disposal verification.
5	fileDamageReport	File Upload	No	-	Upload control for physical inspection report or damaged stock photo evidence.

6	tblDisposalItems	Data Grid	Yes	-	Details: No, Item Code, Material Name, Package Spec, Disposal Qty, Cost Price, Total Financial Loss, Mandatory Item Cause (BR-43), Action.
7	lblTotalLoss	Label	Yes	-	Cumulative financial waste value recorded to store loss ledger.
8	btnConfirmDisposal	Button	Yes	-	Enforces disposal reason (BR-43), validates Branch Manager approval limit (BR-51), deducts store stock, and logs audit record.
9	btnCancel	Button	No	-	Closes modal without executing write-off.

4.2.1.5 Store Inventory Audit & Reconciliation Slip

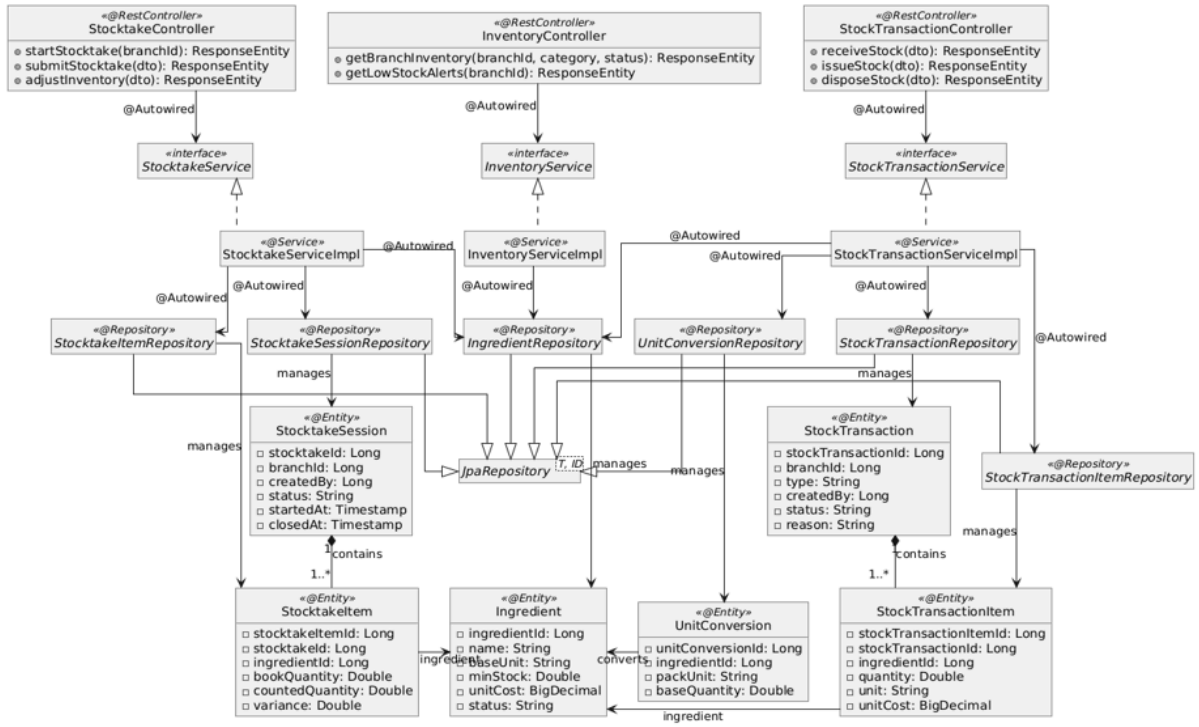


No	Object/Control Name	Type	Required	Length	Description
1	cboStocktakeRound	Dropdown	Yes	-	Audit session type: Periodic End-of-Day Store Closing Audit, Spot Audit by Management, Shift Handover Audit.
2	txtSlipNumber	Textbox	Yes	20	Auto-generated audit session code (e.g., PKK-20261017-001), read-only.
3	txtInspectorName	Textbox	Yes	Max 50	Auditing personnel (Branch Manager and Barista witness), read-only.
4	txtStocktakeTime	Textbox	Yes	19	Timestamp of count finalization.
5	tblStocktakeItems	Data Grid	Yes	-	Reconciliation grid: No, Item Code, Name, Package Spec, Book Stock, Counted Qty, Variance Qty (BR-44), Variance Value, Mandatory Justification Reason.
6	lblDiscrepancySummary	Label	Yes	-	Net shrinkage and discrepancy summary in VND.

7	btnConfirmStocktake	Button	Yes	-	Reconciles actual count, automatically realigns book balance to physical count (BR-44), closes audit session, and captures audit log (BR-51).
8	btnCancel	Button	No	-	Exits stocktake mode without altering inventory records.

4.11.2 Class/Component Design

4.11.2.1 Classes/Component Structure



4.11.2.2 Class/Communication Description

InventoryController Class

Class	InventoryController
Description	Spring REST Controller exposing HTTP endpoints for store inventory inquiries, filtering, and safety stock threshold monitoring.
Base Class	None (@RestController, @RequestMapping("/api/v1/inventory"))
Constructor	+InventoryController(InventoryService inventoryService)
Prototype	public class InventoryController
Source File	InventoryController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
inventoryService	InventoryService - Service interface injected via @Autowired managing inventory queries and alerts.
Methods	
Name	**Input / Output / Description**
getBranchInventory	Input: Long branchId, String category, String status Output: ResponseEntity<List<IngredientStockDto>> Description: GET endpoint retrieving active ingredients and on-hand balances for branch.
getLowStockAlerts	Input: Long branchId Output: ResponseEntity<List<LowStockAlertDto>> Description: GET endpoint returning ingredients where onHand <= minStock (BR-40, UC33).

StockTransactionController Class

Class	StockTransactionController
Description	Spring REST Controller handling multi-type warehouse transactions: Receiving from suppliers (UC28), Issuing to bar counter (UC29), and Disposing of damaged stock (UC30).
Base Class	None (@RestController, @RequestMapping("/api/v1/stock-transactions"))
Constructor	+StockTransactionController(StockTransactionService stockTxService)
Prototype	public class StockTransactionController
Source File	StockTransactionController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
stockTxService	StockTransactionService - Service interface orchestrating inbound, outbound, and disposal voucher operations.
Methods	
Name	**Input / Output / Description**

receiveStock	Input: @RequestBody @Valid CreateReceiveSlipDto dto Output: ResponseEntity<StockTxResponseDto> Description: POST endpoint creating goods receipt and incrementing inventory by bulk package units (BR-40, UC28).
issueStock	Input: @RequestBody @Valid CreateIssueSlipDto dto Output: ResponseEntity<StockTxResponseDto> Description: POST endpoint creating bar issue voucher and deducting store inventory in intact units (BR-41, BR-42, UC29).
disposeStock	Input: @RequestBody @Valid CreateDisposalSlipDto dto Output: ResponseEntity<StockTxResponseDto> Description: POST endpoint writing off spoiled/expired stock with mandatory explanation (BR-43, BR-51, UC30).

StocktakeController Class

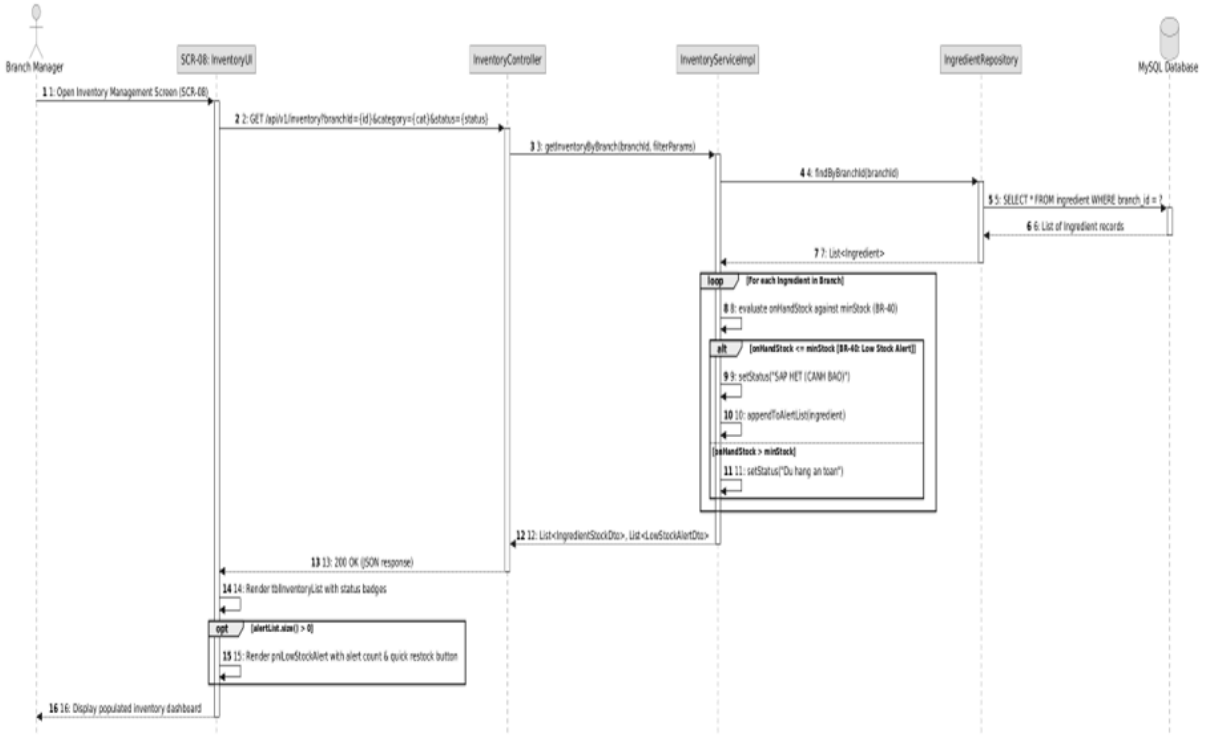
Class	StocktakeController
Description	Spring REST Controller managing store stocktaking sessions, discrepancy computation, and physical count balance realignment.
Base Class	None (@RestController, @RequestMapping("/api/v1/stocktake"))
Constructor	+StocktakeController(StocktakeService stocktakeService)
Prototype	public class StocktakeController
Source File	StocktakeController.java
Namespace	com.scms.controller
Attributes	
Name	**Type & Description**
stocktakeService	StocktakeService - Service interface executing periodic inventory audits and ledger adjustments.
Methods	
Name	**Input / Output / Description**
createStocktakeSession	Input: Long branchId Output: ResponseEntity<StocktakeSessionDto> Description: POST endpoint opening a new audit session and snapshotting book balances (UC31).
adjustInventory	Input: @RequestBody @Valid CreateStocktakeAdjustmentDto dto Output: ResponseEntity<StocktakeResponseDto> Description: POST endpoint calculating variance (counted - book), realigning on-hand stock, and logging audit trail (BR-44, BR-51, UC32).

StockTransactionServiceImpl Class

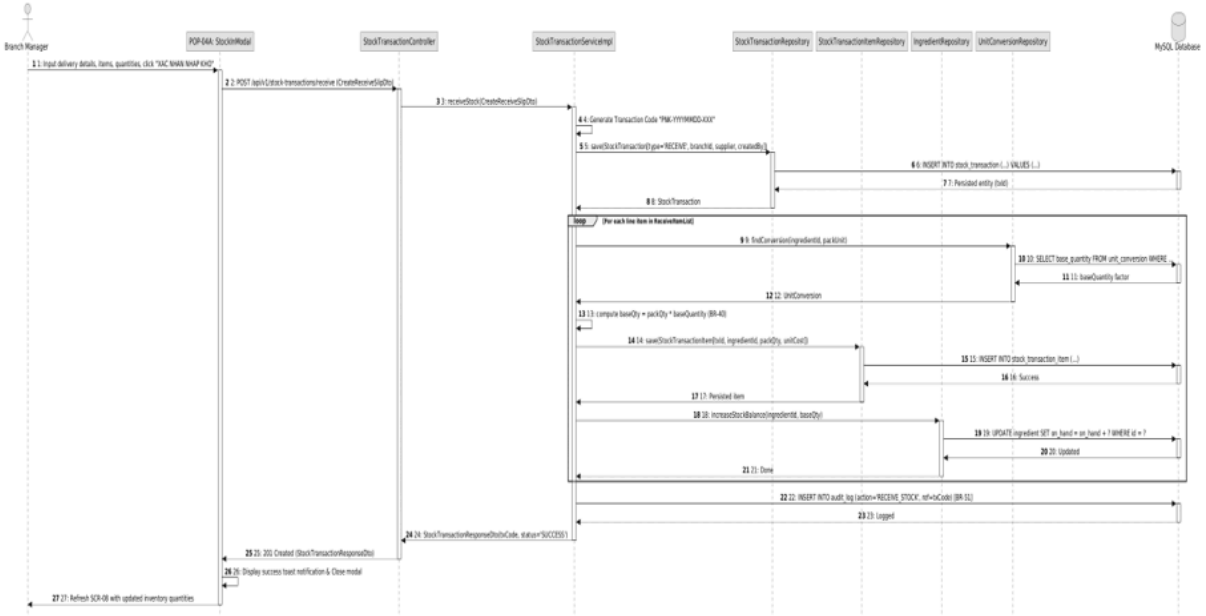
Class	StockTransactionServiceImpl
Description	Service implementation orchestrating inventory transaction validation, unit conversion calculations, and ledger persistence.
Base Class	None (implements StockTransactionService)
Constructor	+StockTransactionServiceImpl(StockTransactionRepository txRepo, StockTransactionItemRepository itemRepo, IngredientRepository ingRepo, UnitConversionRepository convRepo)
Prototype	public class StockTransactionServiceImpl implements StockTransactionService
Source File	StockTransactionServiceImpl.java
Namespace	com.scms.service.impl
Attributes	
Name	**Type & Description**
txRepo	StockTransactionRepository - Repository persisting transaction master records.
itemRepo	StockTransactionItemRepository - Repository persisting transaction line items.
ingRepo	IngredientRepository - Repository updating ingredient on-hand quantities.
convRepo	UnitConversionRepository - Repository querying packaging-to-base unit conversion ratios.
Methods	
Name	**Input / Output / Description**
receiveStock	Input: CreateReceiveSlipDto dto Output: StockTransaction Description: Validates bulk packaging unit, converts to base quantity, increments on-hand stock, logs audit entry.
issueStockToBar	Input: CreateIssueSlipDto dto Output: StockTransaction Description: Checks onHand >= requested, enforces intact packaging units, deducts stock, persists issue slip.
disposeStock	Input: CreateDisposalSlipDto dto Output: StockTransaction Description: Enforces mandatory reason, validates Branch Manager approval limit, deducts stock balance.

4.11.2.3 Object Interactions (Sequence Diagrams)

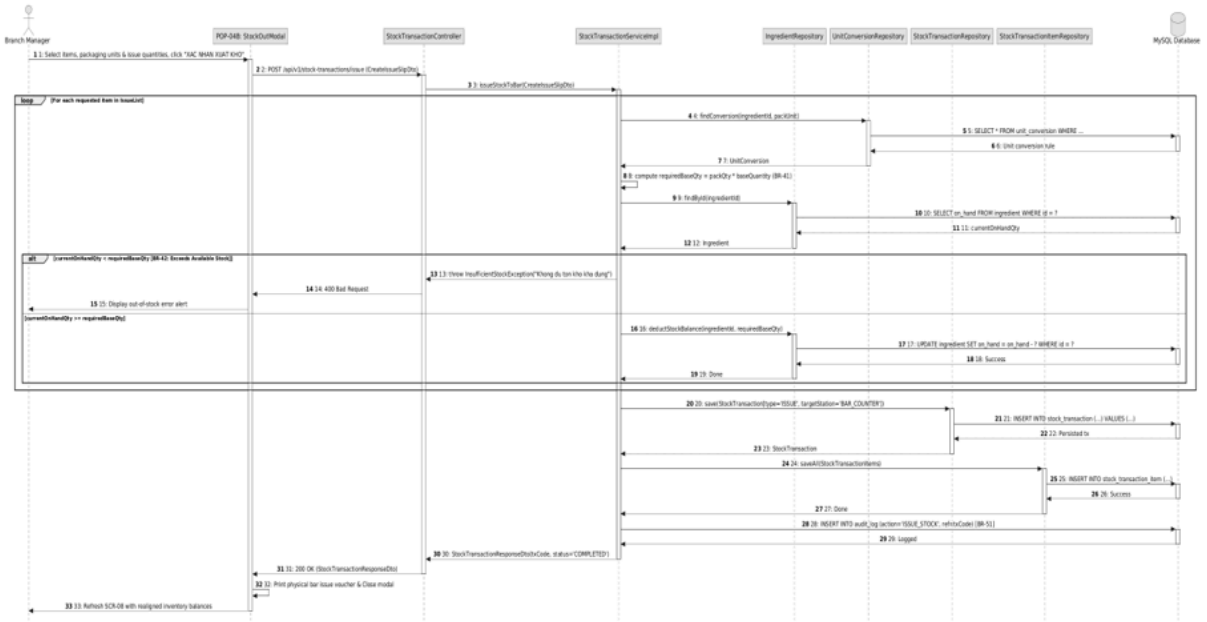
4.11.2.3.1: (SD-01) View Branch Inventory & Safety Stock Alert (UC33)



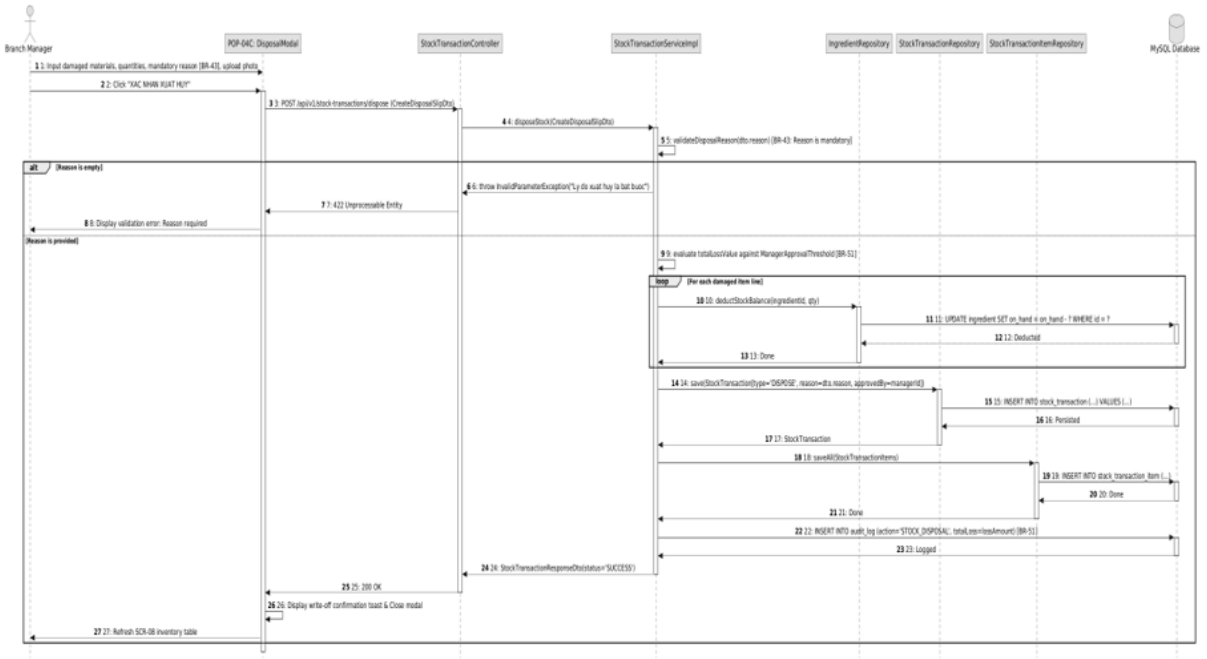
4.2.3.2 : SD-Receive Stock Delivery from



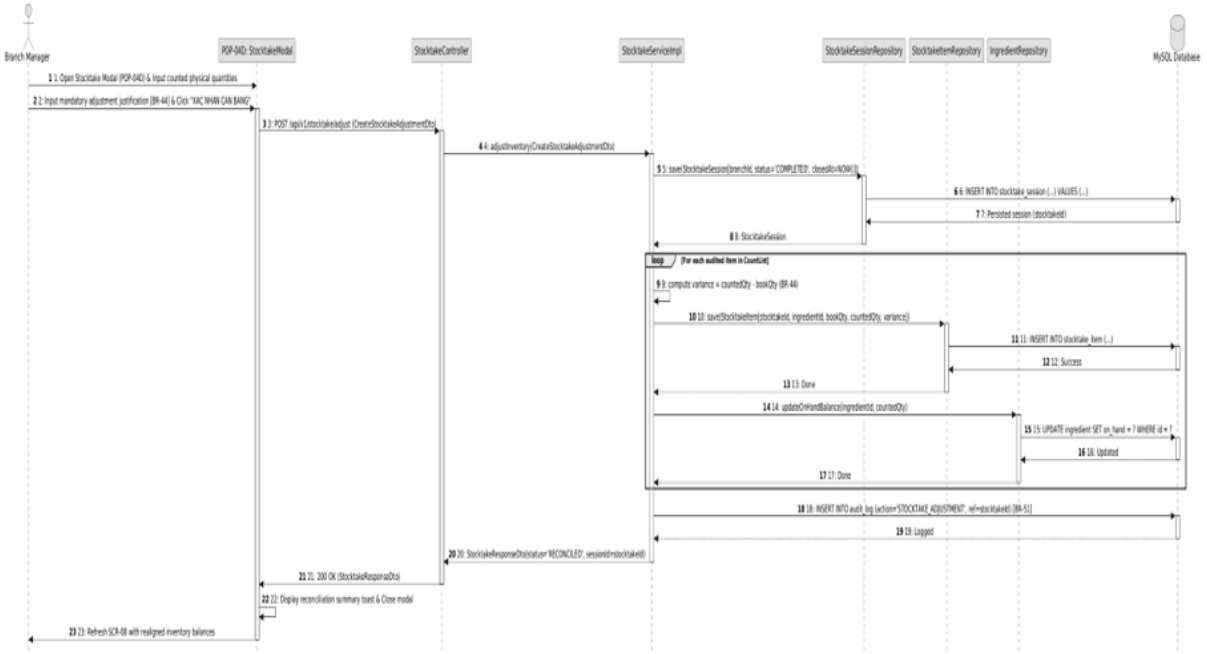
4.2.3.3: SD-03: Issue Stock to Bar Counter



4.2.3.4 SD-04: Record Damaged / Disposed Materials

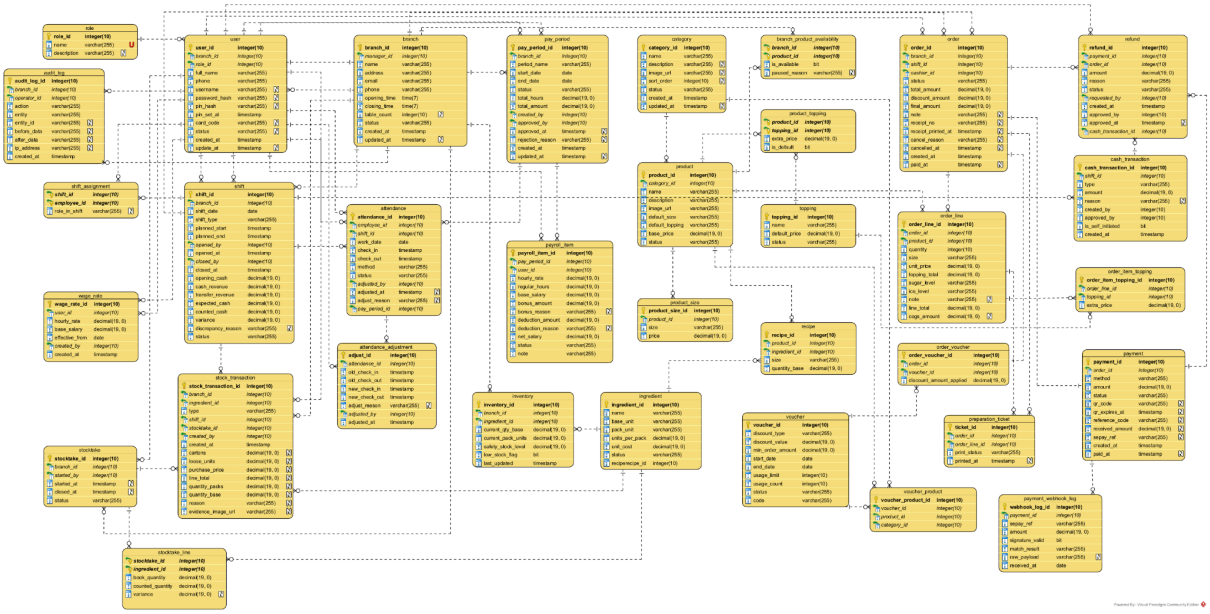


4.2.3.5 SD-05: Periodic Stocktake & Inventory Adjustment



5. Database Design

5.1 Database Design



5.1.1. branch

Description: Stores information about each branch in the chain.

Group: A. Organization & Access Control

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	branch_id	INT		No	Yes	PK	Branch identifier (primary key)
2	manager_id	INT		No	No	FK→user(user_id)	Current branch manager
3	name	VARCHAR	255	No	Yes		Branch name
4	address	VARCHAR	255	No	Yes		Physical address
5	email	VARCHAR	255	No	No		Contact email
6	phone	VARCHAR	255	No	No		Contact phone number
7	opening_time	TIME		No	Yes		Daily opening time
8	closing_time	TIME		No	Yes		Daily closing time
9	table_count	INT		No	No		Number of tables
10	status	VARCHAR	20	No	Yes		OPEN / TEMP_CLOSED / MAINTENANCE
11	created_at	TIMESTAMP		No	Yes		Creation timestamp
12	updated_at	TIMESTAMP		No	No		Last update timestamp

5.1.2. role

Description: Defines the user roles available in the system.

Group: A. Organization & Access Control

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	role_id	INT		No	Yes	PK	Role identifier
2	name	VARCHAR	50	Yes	Yes		COO / Branch Manager / Cashier / Staff
3	description	VARCHAR	255	No	No		Description of the role scope

5.1.3. user

Description: Employee profile and login account. Each user belongs to one branch (except COO) and has one role.

Group: A. Organization & Access Control

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	user_id	INT		No	Yes	PK	User identifier
2	branch_id	INT		No	No	FK→branch(branch_id)	Assigned branch; NULL for COO
3	role_id	INT		No	Yes	FK→role(role_id)	Assigned role
4	full_name	VARCHAR	255	No	Yes		Full name
5	phone	VARCHAR	20	No	Yes		Phone number
6	username	VARCHAR	50	Yes	No		Login username (required for COO / Branch Manager)
7	password_hash	VARCHAR	255	No	No		BCrypt password hash
8	pin_hash	VARCHAR	255	No	No		Hashed 4-digit PIN used at the kiosk and for counter approvals
9	pin_set_at	TIMESTAMP		No	No		Timestamp of the most recent PIN change
10	card_code	VARCHAR	50	Yes	No		RFID/NFC card code (BR-05: one card per active employee)
11	status	VARCHAR	20	No	Yes		ACTIVE / INACTIVE (BR-13: hard delete prohibited)
12	created_at	TIMESTAMP		No	Yes		Creation timestamp
13	update_at	TIMESTAMP		No	No		Last update timestamp

5.1.4. category

Description: Menu categories, centrally managed by the COO (UC09).

Group: B. Menu & Products

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	category_id	INT		No	Yes	PK	Category identifier
2	name	VARCHAR	100	No	Yes		Category name
3	description	VARCHAR	500	No	No		Description
4	image_url	VARCHAR	255	No	No		Category image URL
5	sort_order	INT		No	No		Display order
6	status	VARCHAR	20	No	Yes		ACTIVE / INACTIVE
7	created_at	TIMESTAMP		No	Yes		Creation timestamp
8	updated_at	TIMESTAMP		No	No		Last update timestamp

5.1.5. product

Description: Products sold at the counter. Selling price is stored per size in product_size (BR-18).

Group: B. Menu & Products

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	product_id	INT		No	Yes	PK	Product identifier
2	category_id	INT		No	Yes	FK→category(category_id)	Category
3	name	VARCHAR	150	No	Yes		Product name
4	description	VARCHAR	1000	No	No		Description
5	image_url	VARCHAR	500	No	No		Product image URL
6	default_size	VARCHAR	10	No	No		Default size (S/M/L)

7	default_topping	VARCHAR	255	No	No		Default topping
8	base_price	DECIMAL	19,2	No	Yes		Reference base price
9	status	VARCHAR	20	No	Yes		AVAILABLE / UNAVAILABLE / INACTIVE (BR-17)

5.1.6. product_size

Description: Selling price of a product for each cup size.

Group: B. Menu & Products | Additional constraints: (product_id, size) UNIQUE

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	product_size_id	INT		No	Yes	PK	Price row identifier
2	product_id	INT		No	Yes	FK→product(product_id)	Product
3	size	VARCHAR	10	No	Yes		S / M / L
4	price	DECIMAL	19, 2	No	Yes		Selling price for this size

5.1.7. topping

Description: Master list of toppings.

Group: B. Menu & Products

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	topping_id	INT		No	Yes	PK	Topping identifier
2	name	VARCHAR	100	Yes	Yes		Topping name
3	default_price	DECIMAL	19,2	No	Yes		Default unit price
4	status	VARCHAR	20	No	Yes		ACTIVE / INACTIVE

5.1.8. product_topping

Description: Toppings allowed for each product (many-to-many).

Group: B. Menu & Products | Additional constraints: Composite PK (product_id, topping_id)

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes

1	product_id	INT		No	No	PK + FK→product(product_id)	Product
2	topping_id	INT		No	No	PK + FK→topping(topping_id)	Topping
3	extra_price	DECIMAL	19,2	No	Yes		Surcharge when this topping is selected for the product
4	is_default	BIT		No	Yes		Whether this topping is selected by default

5.1.9. branch_product_availability

Description: Per-branch availability override for products (many-to-many).

Group: B. Menu & Products | Additional constraints: Composite PK (branch_id, product_id)

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	branch_id	INT		No	No	PK + FK→branch(branch_id)	Branch
2	product_id	INT		No	No	PK + FK→product(product_id)	Product
3	is_available	BIT		No	Yes		Whether the product is currently sellable at this branch
4	paused_reason	VARCHAR	500	No	No		Reason why the product is paused

5.1.10. recipe

Description: Bill of materials (BOM) defining ingredient quantities per product and size (BR-19).

Group: B. Menu & Products | Additional constraints: (product_id, ingredient_id, size) UNIQUE

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	recipe_id	INT		No	Yes	PK	Recipe line identifier

2	product_id	INT		No	Yes	FK→product(product_id)	Product
3	ingredient_id	INT		No	Yes	FK→ingredient(ingredient_id)	Ingredient
4	size	VARCHAR	10	No	No		Applicable size; NULL means all sizes
5	quantity_base	DECIMAL	19,4	No	Yes		Required quantity in base unit

5.1.11. ingredient

Description: Raw materials used for beverage and food preparation.

Group: C. Inventory & Ingredients

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	ingredient_id	INT		No	Yes	PK	Ingredient identifier
2	name	VARCHAR	150	No	Yes		Ingredient name
3	base_unit	VARCHAR	20	No	Yes		Base unit of measure (ml, g, can)
4	pack_unit	VARCHAR	20	No	No		Bulk packaging unit (carton, bag) - BR-40
5	units_per_pack	DECIMAL	19,4	No	No		Number of base units contained in one pack unit
6	unit_cost	DECIMAL	19,2	No	No		Reference cost per base unit
7	status	VARCHAR	20	No	Yes		ACTIVE / INACTIVE

5.1.12. inventory

Description: Current stock level of each ingredient at each branch.

Group: C. Inventory & Ingredients | Additional constraints: (branch_id, ingredient_id) UNIQUE

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	inventory_id	INT		No	Yes	PK	Inventory row identifier
2	branch_id	INT		No	Yes	FK→branch(branch_id)	Branch

3	ingredient_id	INT		No	Yes	FK→ingredient (ingredient_id)	Ingredient
4	current_qty_base	DECIMAL	19,4	No	Yes		On-hand quantity in base units
5	current_pack_units	DECIMAL	19,4	No	Yes		On-hand quantity in pack units
6	safety_stock_level	DECIMAL	19,4	No	Yes		Safety stock threshold per branch (BR-45)
7	low_stock_flag	BIT		No	Yes		TRUE when on-hand quantity is at or below the threshold
8	last_updated	TIMESTAMP		No	Yes		Last update timestamp

5.1.13. stocktake

Description: Physical inventory count session at a branch (UC31).

Group: C. Inventory & Ingredients

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	stocktake_id	INT		No	Yes	PK	Stocktake session identifier
2	branch_id	INT		No	Yes	FK→branch (branch_id)	Branch
3	started_by	INT		No	Yes	FK→user(user_id)	Employee who started the session
4	started_at	TIMESTAMP		No	Yes		Start timestamp
5	closed_at	TIMESTAMP		No	No		Close timestamp
6	status	VARCHAR	20	No	Yes		OPEN / ADJUSTED / CLOSED

5.1.14. stocktake_line

Description: Counted quantity per ingredient within a stocktake session (UC32, BR-44).

Group: C. Inventory & Ingredients | Additional constraints: Composite PK (stocktake_id, ingredient_id)

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	stocktake_id	INT		No	No	PK + FK→stocktake(stocktake_id)	Stocktake session
2	ingredient_id	INT		No	No	PK + FK→ingredient(ingredient_id)	Ingredient
3	book_quantity	DECIMAL	19,4	No	Yes		System (book) quantity at count time
4	counted_quantity	DECIMAL	19,4	No	Yes		Physically counted quantity
5	variance	DECIMAL	19,4	No	Yes		Variance = counted - book

5.1.15. stock_transaction

Description: Stock documents: receiving, issue to bar, damage/disposal, adjustment (UC28-UC30, UC32).

Group: C. Inventory & Ingredients

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	stock_transaction_id	INT		No	Yes	PK	Stock transaction identifier
2	branch_id	INT		No	Yes	FK→branch(branch_id)	Branch
3	ingredient_id	INT		No	Yes	FK→ingredient(ingredient_id)	Ingredient
4	type	VARCHAR	20	No	Yes		RECEIVE / ISSUE / DISPOSAL / ADJUST
5	shift_id	INT		No	No	FK→shift(shift_id)	Shift in which the transaction occurred
6	stocktake_id	INT		No	No	FK→stocktake(stocktake_id)	Source stocktake session (when type = ADJUST)
7	created_by	INT		No	Yes	FK→user(user_id)	Employee who created the record

8	created_at	TIMESTAMP		No	Yes		Creation timestamp
9	cartons	DECIMAL	19,4	No	No		Number of cartons received (BR-40)
10	loose_units	DECIMAL	19,4	No	No		Number of loose units received
11	purchase_price	DECIMAL	19,2	No	No		Purchase price per carton (RECEIVE only)
12	line_total	DECIMAL	19,2	No	No		Line total = quantity x price (RECEIVE only)
13	quantity_packs	DECIMAL	19,4	No	No		Quantity in pack units (issue / disposal)
14	quantity_base	DECIMAL	19,4	No	No		Quantity converted to base units
15	reason	VARCHAR	500	No	No		Reason - mandatory for DISPOSAL (BR-43)
16	evidence_image_url	VARCHAR	500	No	No		Evidence photo URL

5.1.16. shift

Description: Work shift, which is also the cash drawer session. Unit of cash and revenue reconciliation (UC24-UC27).

Group: D. Shifts, Attendance & Payroll

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	shift_id	INT		No	Yes	PK	Shift identifier
2	branch_id	INT		No	Yes	FK→branch(branch_id)	Branch
3	shift_date	DATE		No	Yes		Working date

4	shift_type	VARCHAR	20	No	Yes		MORNING / EVENING / NIGHT
5	planned_start	TIMESTAMP		No	Yes		Planned start time
6	planned_end	TIMESTAMP		No	Yes		Planned end time
7	opened_by	INT		No	Yes	FK→user(user_id)	Employee who opened the shift
8	opened_at	TIMESTAMP		No	No		Actual open timestamp
9	closed_by	INT		No	No	FK→user(user_id)	Employee who closed the shift
10	closed_at	TIMESTAMP		No	No		Actual close timestamp
11	opening_cash	DECIMAL	19,2	No	Yes		Declared opening cash (BR-36)
12	cash_revenue	DECIMAL	19,2	No	Yes		Cash revenue collected during the shift
13	transfer_revenue	DECIMAL	19,2	No	Yes		Bank transfer revenue collected during the shift
14	expected_cash	DECIMAL	19,2	No	No		Theoretical cash balance (BR-39)
15	counted_cash	DECIMAL	19,2	No	No		Physically counted cash at closing (BR-38)
16	variance	DECIMAL	19,2	No	No		Variance = counted - expected
17	discrepancy_reason	VARCHAR	500	No	No		Explanation - mandatory when variance is not zero
18	status	VARCHAR	20	No	Yes		OPEN / CLOSED

5.1.17. shift_assignment

Description: Assignment of employees to work shifts (UC08, BR-15).

Group: D. Shifts, Attendance & Payroll | Additional constraints: Composite PK (shift_id, employee_id)

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	shift_id	INT		No	No	PK + FK→shift(shift_id)	Shift
2	employee_id	INT		No	No	PK + FK→user(user_id)	Employee
3	role_in_shift	VARCHAR	50	No	No		Role performed in this shift

5.1.18. attendance

Description: Clock-in / clock-out records per shift (UC03, UC04).

Group: D. Shifts, Attendance & Payroll | Additional constraints: (employee_id, shift_id) UNIQUE

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	attendance_id	INT		No	Yes	PK	Attendance identifier
2	employee_id	INT		No	Yes	FK→user(user_id)	Employee
3	shift_id	INT		No	Yes	FK→shift(shift_id)	Shift
4	work_date	DATE		No	Yes		Working date
5	check_in	TIMESTAMP		No	Yes		Clock-in timestamp
6	check_out	TIMESTAMP		No	No		Clock-out timestamp
7	method	VARCHAR	20	No	Yes		PIN / RFID
8	status	VARCHAR	20	No	Yes		PRESENT / LATE / EARLY_LEAVE / ADJUSTED
9	adjusted_by	INT		No	No	FK→user(user_id)	Manager who adjusted the record (BR-10)
10	adjusted_at	TIMESTAMP		No	No		Adjustment timestamp

11	adjust_reason	VARCHAR	500	No	No		Adjustment reason (BR-10)
12	pay_period_id	INT		No	No	FK→pay_period(pay_period_id)	Pay period that locked this record

5.1.19. attendance_adjustment

Description: History of attendance time adjustments (BR-10).

Group: D. Shifts, Attendance & Payroll

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	adjust_id	INT		No	Yes	PK	Adjustment identifier
2	attendance_id	INT		No	Yes	FK→attendance(attendance_id)	Attendance record
3	old_check_in	TIMESTAMP		No	No		Previous clock-in
4	old_check_out	TIMESTAMP		No	No		Previous clock-out
5	new_check_in	TIMESTAMP		No	No		New clock-in
6	new_check_out	TIMESTAMP		No	No		New clock-out
7	adjust_reason	VARCHAR	500	No	Yes		Reason for the adjustment
8	adjusted_by	INT		No	Yes	FK→user(user_id)	Manager who made the adjustment
9	adjusted_at	TIMESTAMP		No	Yes		Adjustment timestamp

5.1.20. wage_rate

Description: History of hourly wage rates (BR-14: a new rate only applies to shifts after the change).

Group: D. Shifts, Attendance & Payroll

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	wage_rate_id	INT		No	Yes	PK	Wage rate identifier
2	user_id	INT		No	Yes	FK→user(user_id)	Employee
3	hourly_rate	DECIMAL	19,2	No	Yes		Hourly wage

4	base_salary	DECIMAL	19,2	No	No		Base salary
5	effective_from	DATE		No	Yes		Effective date
6	created_by	INT		No	Yes	FK→user(user_id)	Manager who set the rate
7	created_at	TIMESTAMP		No	Yes		Creation timestamp

5.1.21. pay_period

Description: Payroll period of a branch (UC45, BR-53).

Group: D. Shifts, Attendance & Payroll | Additional constraints: CHECK (approved_by IS NULL OR approved_by <> created_by)

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	pay_period_id	INT		No	Yes	PK	Pay period identifier
2	branch_id	INT		No	Yes	FK→branch(branch_id)	Branch
3	period_name	VARCHAR	100	No	Yes		Period name (e.g. October 2026 - Cycle 1)
4	start_date	DATE		No	Yes		Period start date
5	end_date	DATE		No	Yes		Period end date
6	status	VARCHAR	20	No	Yes		DRAFT / SUBMITTED / APPROVED / REJECTED / PAID
7	total_hours	DECIMAL	19,2	No	Yes		Total verified working hours
8	total_amount	DECIMAL	19,2	No	Yes		Total payroll amount payable
9	created_by	INT		No	Yes	FK→user(user_id)	Branch Manager who created the period
10	approved_by	INT		No	No	FK→user(user_id)	COO who approved the period
11	approved_at	TIMESTAMP		No	No		Approval timestamp

12	rejection_reason	VARCHAR	255	No	No		Reason provided when rejected
13	created_at	TIMESTAMP		No	Yes		Creation timestamp
14	updated_at	TIMESTAMP		No	No		Last update timestamp

5.1.22. payroll_item

Description: Payroll statement of one employee within a pay period.

Group: D. Shifts, Attendance & Payroll | Additional constraints: (pay_period_id, user_id) UNIQUE

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	payroll_item_id	INT		No	Yes	PK	Payroll line identifier
2	pay_period_id	INT		No	Yes	FK→pay_period(pay_period_id)	Pay period
3	user_id	INT		No	Yes	FK→user(user_id)	Employee
4	hourly_rate	DECIMAL	19,2	No	Yes		Hourly rate snapshot captured when the period is closed
5	regular_hours	DECIMAL	19,2	No	Yes		Verified working hours in the period
6	base_salary	DECIMAL	19,2	No	Yes		Base wage = hours x hourly rate
7	bonus_amount	DECIMAL	19,2	No	Yes		Bonus amount
8	bonus_reason	VARCHAR	255	No	No		Bonus justification
9	deduction_amount	DECIMAL	19,2	No	Yes		Deduction amount
10	deduction_reason	VARCHAR	255	No	No		Deduction justification
11	net_salary	DECIMAL	19,2	No	Yes		Net payable = base + bonus - deduction

12	status	VARCHAR	20	No	Yes		PENDING / APPROVED / PAID
13	note	VARCHAR	255	No	No		General remarks

5.1.23. order

Description: Customer order placed at the counter (UC16). Data is frozen once the order is PAID (BR-26).

Group: E. Orders & Payments | Additional constraints: CHECK (final_amount >= 0)

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	order_id	INT		No	Yes	PK	Order identifier
2	branch_id	INT		No	Yes	FK→branch(branch_id)	Branch
3	shift_id	INT		No	Yes	FK→shift(shift_id)	Shift
4	cashier_id	INT		No	Yes	FK→user(user_id)	Cashier who created the order
5	status	VARCHAR	20	No	Yes		PENDING / PAID / PARTIALLY_REFUNDED / REFUNDED / CANCELLED
6	total_amount	DECIMAL	19,2	No	Yes		Order subtotal
7	discount_amount	DECIMAL	19,2	No	Yes		Total discount applied
8	final_amount	DECIMAL	19,2	No	Yes		Amount payable
9	note	VARCHAR	500	No	No		Order note
10	receipt_no	VARCHAR	30	No	No		Receipt number (incremented per day or per shift)
11	receipt_printed_at	TIMESTAMP		No	No		Receipt print timestamp
12	cancel_reason	VARCHAR	500	No	No		Cancellation reason (BR-26b)
13	cancelled_at	TIMESTAMP		No	No		Cancellation timestamp
14	created_at	TIMESTAMP		No	Yes		Creation timestamp
15	paid_at	TIMESTAMP		No	No		Payment timestamp

5.1.24. order_line

Description: Order line item (resolves the Order-Product many-to-many relationship).

Group: E. Orders & Payments

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	order_line_id	INT		No	Yes	PK	Order line identifier
2	order_id	INT		No	Yes	FK→order(order_id)	Order
3	product_id	INT		No	Yes	FK→product(product_id)	Product
4	quantity	INT		No	Yes		Quantity
5	size	VARCHAR	10	No	No		Size sold (snapshot)
6	unit_price	DECIMAL	19,2	No	Yes		Unit price at sale time (BR-23)
7	topping_total	DECIMAL	19,2	No	Yes		Total topping surcharge
8	sugar_level	VARCHAR	20	No	No		Sugar level (BR-25)
9	ice_level	VARCHAR	20	No	No		Ice level (BR-25)
10	note	VARCHAR	300	No	No		Preparation note
11	line_total	DECIMAL	19,2	No	Yes		Line total
12	cogs_amount	DECIMAL	19,2	No	No		Cost of goods sold for this line (BR-48)

5.1.25. order_item_topping

Description: Toppings selected by the customer on an order line (many-to-many).

Group: E. Orders & Payments | Additional constraints: (order_line_id, topping_id) UNIQUE

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	order_item_topping_id	INT		No	Yes	PK	Order topping identifier
2	order_line_id	INT		No	Yes	FK→order_line(order_line_id)	Order line

3	topping_id	INT		No	Yes	FK→topping(topping_id)	Topping
4	extra_price	DECIMAL	19,2	No	Yes		Surcharge snapshot

5.1.26. voucher

Description: Discount programmes and promotion vouchers (UC13, BR-20/21).

Group: E. Orders & Payments

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	voucher_id	INT		No	Yes	PK	Voucher identifier
2	code	VARCHAR	50	Yes	Yes		Voucher code entered at checkout
3	discount_type	VARCHAR	10	No	Yes		PERCENT / FIXED
4	discount_value	DECIMAL	19,2	No	Yes		Discount value
5	min_order_amount	DECIMAL	19,2	No	No		Minimum order amount required
6	start_date	DATE		No	Yes		Valid from
7	end_date	DATE		No	Yes		Valid until
8	usage_limit	INT		No	No		Maximum number of redemptions
9	usage_count	INT		No	Yes		Number of redemptions so far
10	status	VARCHAR	20	No	Yes		ACTIVE / EXPIRED / INACTIVE

5.1.27. voucher_product

Description: Products or categories a voucher applies to (UC13 applicable products).

Group: E. Orders & Payments | Additional constraints: CHECK (product_id IS NOT NULL OR category_id IS NOT NULL)

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	voucher_product_id	INT		No	Yes	PK	Voucher scope identifier
2	voucher_id	INT		No	Yes	FK→voucher(voucher_id)	Voucher

3	product_id	INT		No	No	FK→product(product_id)	Applicable product
4	category_id	INT		No	No	FK→category(category_id)	Applicable category

5.1.28. order_voucher

Description: Voucher applied to an order (BR-21: at most one voucher per order).

Group: E. Orders & Payments

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	order_voucher_id	INT		No	Yes	PK	Redemption identifier
2	order_id	INT		Yes	Yes	FK→order(order_id)	Order
3	voucher_id	INT		No	Yes	FK→voucher(voucher_id)	Voucher
4	discount_amount_applied	DECIMAL	19,2	No	Yes		Discount amount actually applied

5.1.29. payment

Description: Payment of an order (UC19, UC20, UC43, UC44).

Group: E. Orders & Payments

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	payment_id	INT		No	Yes	PK	Payment identifier
2	order_id	INT		Yes	Yes	FK→order(order_id)	Order
3	method	VARCHAR	20	No	Yes		CASH / VIETQR
4	amount	DECIMAL	19,2	No	Yes		Amount to be paid
5	status	VARCHAR	20	No	Yes		PENDING / SUCCESS / FAILED / MISMATCH
6	qr_code	VARCHAR	1000	No	No		VietQR payload (BR-28)
7	qr_expires_at	TIMESTAMP		No	No		QR expiration timestamp

8	reference_code	VARCHAR	100	No	No		Reconciliation reference code
9	received_amount	DECIMAL	19,2	No	No		Amount reported by the payment gateway
10	sepay_ref	VARCHAR	100	No	No		SePay transaction reference
11	created_at	TIMESTAMP		No	Yes		Creation timestamp
12	paid_at	TIMESTAMP		No	No		Payment confirmation timestamp

5.1.30. refund

Description: Refund request for a paid order (UC23, BR-32/33/34/35).

Group: F. Refunds & Cash Drawer

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	refund_id	INT		No	Yes	PK	Refund identifier
2	payment_id	INT		No	Yes	FK→payment(payment_id)	Original payment
3	order_id	INT		No	Yes	FK→order(order_id)	Order
4	cash_transaction_id	INT		No	No	FK→cash_transaction(cash_transaction_id)	Cash drawer entry that settled the refund (BR-34)
5	amount	DECIMAL	19,2	No	Yes		Refund amount (must not exceed the amount paid)
6	reason	VARCHAR	500	No	Yes		Refund reason (mandatory - BR-32)

7	status	VARCHAR	20	No	Yes		PENDING / APPROVED / REJECTED / DONE
8	requested_by	INT		No	Yes	FK→user(user_id)	Cashier who requested the refund
9	created_at	TIMESTAMP		No	Yes		Request timestamp
10	approved_by	INT		No	No	FK→user(user_id)	Manager or COO who approved (BR-32/35)
11	approved_at	TIMESTAMP		No	No		Approval timestamp

5.1.31. cash_transaction

Description: Cash movements inside a shift (UC25/UC26, BR-37).

Group: F. Refunds & Cash Drawer

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	cash_transaction_id	INT		No	Yes	PK	Cash transaction identifier
2	shift_id	INT		No	Yes	FK→shift(shift_id)	Shift
3	type	VARCHAR	20	No	Yes		OPEN / CASH_IN / CASH_OUT / REFUND_OUT / CLOSE
4	amount	DECIMAL	19,2	No	Yes		Amount moved
5	reason	VARCHAR	500	No	Yes		Business reason
6	created_by	INT		No	Yes	FK→user(user_id)	Employee who created the entry
7	approved_by	INT		No	No	FK→user(user_id)	Manager who approved by PIN (BR-32)
8	is_self_initiated	BIT		No	Yes		TRUE when a Branch Manager created it themselves (BR-37)

9	created_at	TIMESTAMP		No	Yes		Creation timestamp

5.1.32. payment_webhook_log

Description: Immutable log of inbound SePay webhooks (BR-29/30).

Group: G. System Configuration & Audit | Additional constraints: UNIQUE(sepay_ref) - prevents duplicate processing

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	webhook_log_id	INT		No	Yes	PK	Log identifier
2	payment_id	INT		No	No	FK→payment(payment_id)	Matched payment
3	sepay_ref	VARCHAR	100	No	Yes		SePay transaction reference
4	amount	DECIMAL	19,2	No	Yes		Amount reported by the webhook
5	signature_valid	BIT		No	Yes		Whether the HMAC signature is valid
6	match_result	VARCHAR	20	No	Yes		MATCH / MISMATCH
7	raw_payload	VARCHAR	4000	No	No		Raw webhook payload
8	received_at	TIMESTAMP		No	Yes		Receive timestamp

5.1.33. audit_log

Description: Immutable audit trail of sensitive operations (BR-51).

Group: G. System Configuration & Audit | Additional constraints: UPDATE and DELETE prohibited at database level

No	Field Name	Type	Size	Unique	Not Null	PK/FK	Notes
1	audit_log_id	INT		No	Yes	PK	Audit log identifier

2	branch_id	INT		No	No	FK→branch(branch_id)	Branch in which the action occurred
3	operator_id	INT		No	Yes	FK→user(user_id)	User who performed the action
4	operator_role	VARCHAR	50	No	No		Role at the time of the action (snapshot)
5	action	VARCHAR	50	No	Yes		LOGIN / PRICE_EDIT / ORDER_CANCEL / REFUND / CASH_IN / CASH_OUT / BOM_EDIT / INVENTORY_ADJUST / PERMISSION_ASSIGN
6	entity	VARCHAR	50	No	Yes		Table affected by the action
7	entity_id	VARCHAR	50	No	No		Identifier of the affected record
8	before_data	VARCHAR	4000	No	No		Previous values (JSON)
9	after_data	VARCHAR	4000	No	No		New values (JSON)
10	ip_address	VARCHAR	50	No	No		Client IP address
11	created_at	TIMESTAMP		No	Yes		Action timestamp

5.2 Data File Design

The following files are used to store, access and exchange data in the Raw Material Inventory Management Subsystem (section 4.3). Entity classes map to the tables defined in 5.1, repositories access them through Spring Data JPA, and DTO classes carry data between the REST controllers and the client.

#	File Name	Type	Notes

1	Ingredient.java	Entity	Maps to table ingredient. Raw material with name, base unit, minimum stock and unit cost; referenced by stock voucher lines, stocktake lines and unit conversions.
2	UnitConversion.java	Entity	Maps to table unit_conversion. Conversion ratio between a pack unit and the base unit of an ingredient (BR-40, BR-41).
3	StockTransaction.java	Entity	Maps to table stock_transaction. Master stock voucher (RECEIVE/ISSUE/DISPOSAL/ADJUST); contains 1..* StockTransactionItem.
4	StockTransactionItem.java	Entity	Maps to table stock_transaction_item. Line of a stock voucher: ingredient, quantity, unit and cost.
5	StocktakeSession.java	Entity	Maps to table stocktake_session. Stocktake session of a branch; contains 1..* StocktakeItem.
6	StocktakeItem.java	Entity	Maps to table stocktake_item. Stocktake line with book quantity, counted quantity and variance.
7	IngredientRepository.java	Repository	Spring Data JpaRepository for Ingredient. Used by InventoryServiceImpl, StockTransactionServiceImpl and StocktakeServiceImpl.
8	UnitConversionRepository.java	Repository	Spring Data JpaRepository for UnitConversion. Used by StockTransactionServiceImpl.
9	StockTransactionRepository.java	Repository	Spring Data JpaRepository for StockTransaction. Used by StockTransactionServiceImpl.
10	StockTransactionItemRepository.java	Repository	Spring Data JpaRepository for StockTransactionItem. Used by StockTransactionServiceImpl.
11	StocktakeSessionRepository.java	Repository	Spring Data JpaRepository for StocktakeSession. Used by StocktakeServiceImpl.

12	StocktakeItemRepository.java	Repository	Spring Data JpaRepository for StocktakeItem. Used by StocktakeServiceImpl.
13	IngredientStockDto.java	DTO	Response item of getBranchInventory: ingredient and its on-hand balance for a branch.
14	LowStockAlertDto.java	DTO	Response item of getLowStockAlerts: ingredients where on-hand quantity ≤ minimum stock (BR-45).
15	CreateReceiveSlipDto.java	DTO	Request body of receiveStock: goods receipt by pack units (UC28, BR-40).
16	CreateIssueSlipDto.java	DTO	Request body of issueStock: issue to the bar counter (UC29, BR-41, BR-42).
17	CreateDisposalSlipDto.java	DTO	Request body of disposeStock: write-off with a mandatory reason (UC30, BR-43).
18	StockTxResponseDto.java	DTO	Response of the receive, issue and dispose endpoints.
19	StocktakeSessionDto.java	DTO	Response of createStocktakeSession: the newly opened session with book balances (UC31).
20	CreateStocktakeAdjustmentDto.java	DTO	Request body of adjustInventory: counted quantities of a stocktake (UC32, BR-44).
21	StocktakeResponseDto.java	DTO	Response of adjustInventory: variances and the realigned stock.
22	Stock card export	Excel (.xlsx)	Generated by btnExportStockCard on the Branch Inventory screen (4.3.1.1): store inventory ledger and lot transaction history. File name pattern is not defined in the SDD.
23	Damage report attachment	Upload (file/photo)	Optional evidence uploaded through fileDamageReport on the Disposal slip (4.3.1.4): inspection report or photo of damaged stock. No column in 5.1 stores it, so the storage location is to be decided.